Octane Security Report

Security Analysis of SailMoney: Protocol
6/26/2026

About Octane

Octane is a security intelligence platform purpose-built for modern software teams. We help engineering and security teams identify, investigate, and triage vulnerabilities faster—without disrupting the development process.

Combining automated static analysis with human-readable explanations, Octane bridges the gap between high-signal tooling and real-world developer workflows. Our platform provides contextualized vulnerability reports, clustered findings, and AI-enhanced guidance to make security reviews faster, clearer, and more actionable.

Octane was designed in collaboration with top auditors and engineers to meet the unique demands of web3 and next-gen software systems. We work closely with security researchers, protocol teams, and developer-first organizations to continuously improve our threat detection models and surface what actually matters.

Our clients include leading teams across DeFi, L2 protocols, infrastructure tooling, and security research. Octane integrates seamlessly with developer workflows—from GitHub automation to custom feedback tooling—so teams can reduce noise, eliminate risk, and ship with confidence.

Octane is backed by deep domain knowledge in static analysis, program synthesis, and exploit modeling, with capabilities designed for both technical depth and usability.

To learn more or request a demo, visit www.octane.security, or reach out to us at info@octane.security.

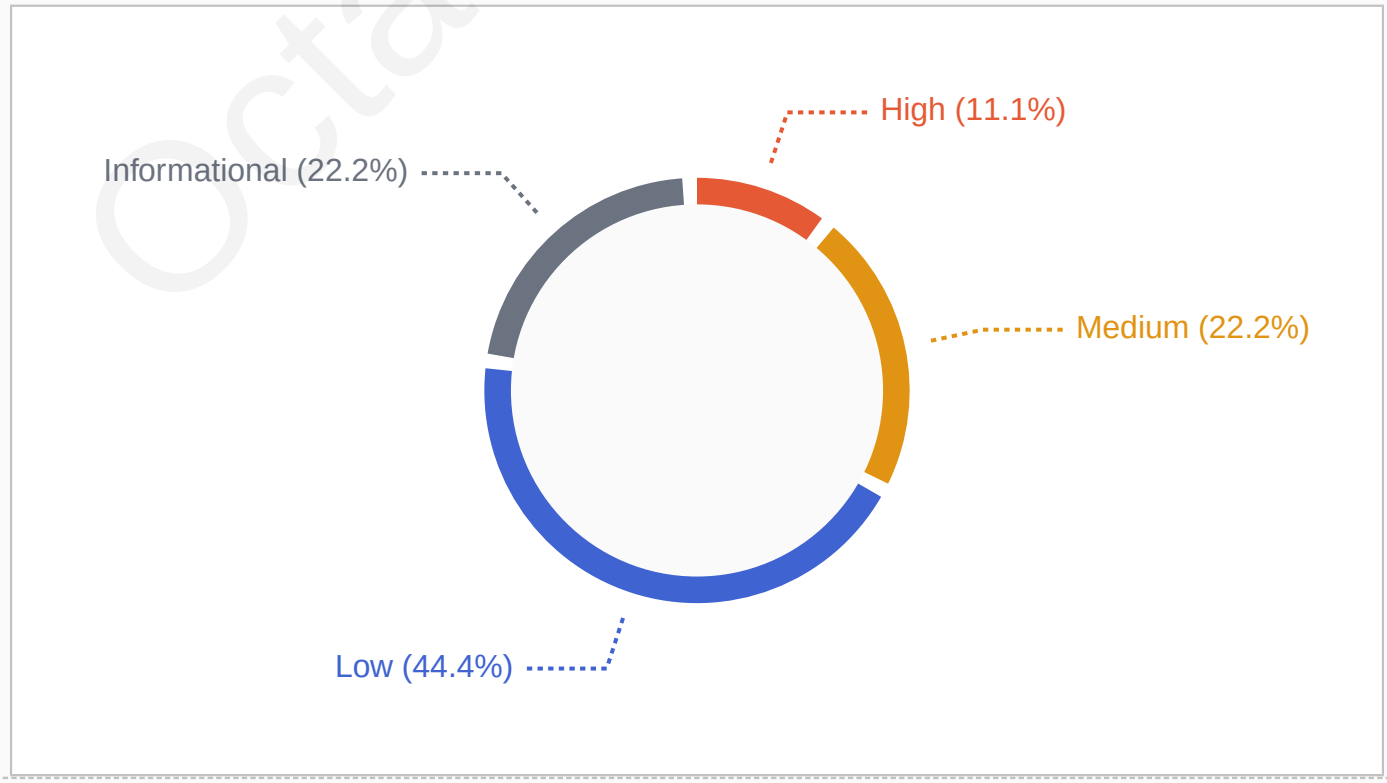
Octane Security
Distributed team with a global presence
www.octane.security

"Summary of findings"

This analysis reviewed the SailMoney: Protocol smart contracts using Octane's automated analysis and included team feedback on findings.

The analysis identified a total of 9 issues. After triage and review, 9 of these issues have been resolved or acknowledged.

Vulnerabilities Confirmed	09/09
Critical	00/00
High	01/01
Medium	02/02
Low	04/04
Informational	02/02



Vulnerabilities & Warnings

Vulnerability #1

Missing initData authentication in MandateFactory.deployAndAttach causes malicious permission initialization and fund theft

Severity: HIGH | Likelihood: MEDIUM | Impact: HIGH | Status: Resolved

Resolved in PR #71. The MandateFactory clone salt now binds keccak256(initData): keccak256(abi.encode(msg.sender, account, salt, keccak256(initData))). Different initData yields a different clone address, so a registration signature bound to a predicted address cannot be reused with substituted initData — the substituted payload deploys at a different address the signature does not authorize. predictCloneAddress folds initData identically so prediction matches deployment. Fix is confined to the untrusted MandateFactory; the kernel's RegisterPermission scheme is unchanged.

MandateFactory.deployAndAttach deterministically deploys a clone using a salt derived only from (msg.sender, account, salt) and then initializes it with arbitrary initData that is not authenticated or bound to the clone address or to the kernel's registration signature. SailKernel.registerPermission authenticates only the permission address, not its initialization. In deployments using a shared/public forwarder, a frontrunner can reuse the victim's (account, impl, salt, kernelSig), substitute different initData, deploy and initialize the same clone address first, and register it to the victim account. This can attach a maliciously-initialized permission enabling the manager to steal funds.

In MandateFactory.deployAndAttach, the clone address is derived via Clones.cloneDeterministic using a namespaced salt keccak256(abi.encode(msg.sender, account, salt)). The factory then calls the clone with initData and only checks that CloneInitializable.initialized() returns true (a liveness guard). No signature or binding includes initData; it is neither added to the salt nor verified by the factory. Next, the factory calls SailKernel.registerPermission(account, clone, deadline, kernelSig), where the EIP-712 struct binds only to (account, permission address, nonce, deadline). Because the kernel signature does not include initData (or a hash of it), and the factory does not bind initData into the clone salt, an attacker who can reproduce msg.sender at the factory (e.g., via the same shared/public forwarder) can front-run the victim's transaction, deploy the same clone address with attacker-chosen initData, and then reuse the victim's kernelSig to register the attacker-initialized permission on the victim's account. The included reference templates are configure-based and safe from this vector, but deployAndAttach is explicitly intended for third-party initialize-based templates. Under such usage, the attacker's malicious initial-

ization can broaden permission behavior (e.g., allow arbitrary approvals or transfers), and the adversarial manager can then dispatch calls that drain funds.

file: contracts/factory/MandateFactory.sol

```
226   uint256 preBalance = address(this).balance - msg.value;
227
228 - bytes32 namespacedSalt = keccak256(abi.encode(msg.sender,
    account, salt));

229   clone = Clones.cloneDeterministic(impl, namespacedSalt);

...

248  ///         and with the same `account`, because the factory
    namespaces the salt
249  ///         by msg.sender and account.
250 - function predictCloneAddress(address impl, address account,
    bytes32 salt) external view returns (address) {
251 - bytes32 namespacedSalt = keccak256(abi.encode(msg.sender,
    account, salt));

252   return Clones.predictDeterministicAddress(impl, namespaced-
    Salt, address(this));
253  }
```

```
226   uint256 preBalance = address(this).balance - msg.value;
227
228 +// Bind initData into the salt so the predicted clone
    address is tied to the exact initialization payload.
229 + bytes32 namespacedSalt = keccak256(abi.encode(msg.sender,
    account, salt, keccak256(initData)));
230   clone = Clones.cloneDeterministic(impl, namespacedSalt);

...

249  ///         and with the same `account`, because the factory
    namespaces the salt
250  ///         by msg.sender and account.
251 + function predictCloneAddress(address impl, address account,
    bytes32 salt, bytes calldata initData) external view returns
    (address) {
252 +// Must use the identical initData bytes that will be
    passed to deployAndAttach.
```

```
253 + bytes32 namespacedSalt = keccak256(abi.encode(msg.sender,
    account, salt, keccak256(initData)));
254   return Clones.predictDeterministicAddress(impl, namespaced-
    Salt, address(this));
255  }
```

Severity

Impact Explanation: [High] In each valid scenario the attacker's substituted initialization widens the registered permission's behavior, enabling the adversarial manager to execute approvals or transfers that directly and materially drain the victim's Safe assets.

Likelihood Explanation: [Medium] Exploitation requires significant but realistic conditions: use of a shared/public forwarder (so msg.sender can be replicated), a third-party initialize-based template without inner signature binding, and public mempool visibility for frontrunning. These constraints are not rare/exceptional but are not universal, yielding medium likelihood.

Exploitation

Exploitation Scenarios:

Scenario 1.
Shared forwarder + initialize-based "Approve" permission: Attacker front-runs deployAndAttach via the same forwarder and salt, initializes the clone to allow approvals to an attacker-controlled spender, registers it with the victim's kernelSig, and the manager then approves the attacker spender, enabling token pull-drain via transferFrom.
Preconditions / Assumptions
(a). Account A is registered in SailKernel with permissionSigner S and manager M
(b). A third-party permission template T is initialize-based and uses CloneInitializable
(c). A shared/public forwarder F is used to call MandateFactory (factory sees msg.sender=F)
(d). PermissionSigner S has signed kernelSig over (account=A, predictedCloneAddress, nonce, deadline)
(e). The transaction is visible in the public mempool (attacker can front-run and copy kernelSig)
(f). Template initializer does not require an embedded permissionSigner signature (no inner auth binding)
(g). Manager M may be adversarial within registered permissions (per scope)

(h). Template T's evaluate() allows approvals if token/spender are stored in its initialized allowlists

Scenario 2.
Shared forwarder + initialize-based "Transfer" permission: Attacker front-runs, initializes the clone with permissive recipient rules, registers it with the victim's kernelSig, and the manager dispatches token.transfer calls to the attacker, resulting in direct outflows.
Preconditions / Assumptions
(a). Account A is registered in SailKernel with permissionSigner S and manager M
(b). A third-party permission template U is initialize-based and uses CloneInitializable
(c). A shared/public forwarder F is used to call MandateFactory (factory sees msg.sender=F)
(d). PermissionSigner S has signed kernelSig over (account=A, predictedCloneAddress, nonce, deadline)
(e). The transaction is visible in the public mempool (attacker can front-run and copy kernelSig)
(f). Template initializer does not require an embedded permissionSigner signature (no inner auth binding)
(g). Manager M may be adversarial within registered permissions (per scope)
(h). Template U's evaluate() allows token.transfer to recipients per initialized rules

Scenario 3.
Shared forwarder + initialize-based permission with a bypass flag: Attacker front-runs and sets a bypass=true flag during initialization, registers with the victim's kernelSig, and the manager gains near-total ability to dispatch arbitrary calls moving funds.
Preconditions / Assumptions
(a). Account A is registered in SailKernel with permissionSigner S and manager M
(b). A third-party permission template V is initialize-based and uses CloneInitializable
(c). A shared/public forwarder F is used to call MandateFactory (factory sees msg.sender=F)
(d). PermissionSigner S has signed kernelSig over (account=A, predictedCloneAddress, nonce, deadline)
(e). The transaction is visible in the public mempool (attacker can front-run and copy kernelSig)
(f). Template initializer does not require an embedded permissionSigner signature (no inner auth binding)
(g). Manager M may be adversarial within registered per-

missions (per scope)
(h). Template V includes a bypass flag in initialize that, when set, makes evaluate() always return true

Vulnerability #2

Missing config-bound checks in SailKernel replace/register allow mempool replay of signer ops before MandateFactory.configure, causing unconfigured-permission DoS and nonce-epoch rotation

Severity: MEDIUM | Likelihood: MEDIUM | Impact: MEDIUM | Status: Resolved

Resolved in PR #69 (same change as #5). A bounded scan over the static signature region rejects any v==1 approved-hash entry before checkSignatures is called, so a setup-time delegate cannot authorize registration without a genuine owner signature. The scan is bounded to the static region so contract-signature tails are never misread.

SailKernel.replacePermission and registerPermission accept any valid permissionSigner EIP-712 signature without verifying that the incoming permission is already configured. MandateFactory sequences configure() before calling these kernel ops in the same transaction. A mempool observer can replay only the kernel operation first, consuming the signer nonce and altering the registry so the factory's later kernel call reverts and rolls back configure(). The result is a removed old permission or a newly registered but unconfigured permission (deny-by-default), and for replacement, manager/batch nonce epochs are rotated, invalidating queued manager-signed actions.

MandateFactory.replace and attach bundle template.configure(...) followed by a SailKernel permission-registry operation (replacePermission or registerPermission). Each inner call is authenticated independently: configure() verifies the permissionSigner's EIP-712 signature against the template, while the kernel verifies a permissionSigner EIP-712 signature for the registry change. SailKernel.replacePermission/registerPermission do not assert that the incoming permission is configured for the account, and accept calls from any msg.sender holding a valid signer signature.

If the factory call is broadcast to a public mempool, an observer can copy the kernel signature and submit SailKernel.replacePermission or registerPermission before the factory transaction mines. For replacePermission, this swaps old→new, bumps the removed side's registration epoch, and rotates manager/batch nonce epochs by a large step. When the factory transaction executes later, its kernel step now reverts (new permission already registered or signer nonce consumed), which reverts the entire factory transaction and rolls back its earlier configure(). The final state is: the old permission is removed; the new permission is registered but unconfigured (all included ConfigurablePermission templates deny by default until configured); and, in the replace case, all

previously queued manager-signed operations are invalidated by epoch rotation. For attach, the attacker's early registerPermission causes the factory's register step to revert, also rolling back configure(), leaving the permission registered but unconfigured. The attacker must pay the kernel's registration fee but gains no direct on-chain profit; the impact is an operational DoS and disruption of queued actions, not fund loss.

file: contracts/core/SailKernel.sol

```
1  // SPDX-License-Identifier: GPL-2.0-or-later
2  pragma solidity 0.8.26;
3
4  import {IPermission, Context}                from
   "../interfaces/IPermission.sol";
5  import {IBatchPermission, Call, BatchContext} from "../inter-
   faces/IBatchPermission.sol";
6  import {IPermissionIntrospection}            from "../inter-
   faces/IPermissionIntrospection.sol";
7  import {IFeePolicy}                           from
   "../interfaces/IFeePolicy.sol";
8  import {SailGovernance}      from "../governance/SailGover-
   nance.sol";
9  import {ECDSA}               from "@openzeppelin/con-
   tracts/utils/cryptography/ECDSA.sol";
10 import {EIP712}              from "@openzeppelin/con-
   tracts/utils/cryptography/EIP712.sol";
11 import {ReentrancyGuard}     from "@openzeppelin/con-
   tracts/utils/ReentrancyGuard.sol";
12 import {IERC1271}            from "@openzeppelin/contracts/in-
   terfaces/IERC1271.sol";
13 import {IERC20}              from "@openzeppelin/contracts/to-
   ken/ERC20/IERC20.sol";
14 import {Math}                from "@openzeppelin/con-
   tracts/utils/math/Math.sol";
15
16 /// @dev Minimal Safe factory interface — used only in `createAc-
   count`.
17 interface ISafeFactory {
18  function createProxyWithNonce(address singleton, bytes
   calldata initializer, uint256 saltNonce)
19  external
20  returns (address proxy);
21
22 /// @notice The proxy creation bytecode the factory deploys
   (excludes the appended
```

```
23  ///         singleton constructor arg). For Safe v1.4.1 this
    is `type(SafeProxy).creationCode`.
24  /// @dev    The kernel uses this to predict the CREATE2 proxy
    address locally (see
25  ///         `createAccount`), so it can adopt an already-deployed
    proxy (idempotency /
26  ///         front-run resilience) without a factory-side
    predictor. Safe v1.4.1's factory
27  ///         exposes `proxyCreationCode()` but NOT a view
    address predictor —
28  ///         `calculateCreateProxyWithNonceAddress` was a
    revert-based simulator removed after v1.3.0.
29   function proxyCreationCode() external pure returns (bytes
    memory);
30  }
31
32  /// @dev Minimal Safe module interface — used for executing
    transactions and fee transfers.
33  /// @dev DEPLOY ASSUMPTION: only Safe v1.4.1-style proxies may be
    governance-codehash-allowlisted —
34  ///     i.e. proxies that (a) intercept masterCopy() (0xa619486e)
    from storage slot 0 in their own
35  ///     fallback, (b) expose nonce(), and (c) implement Safe-core
    checkSignatures(bytes32,bytes,bytes).
36  ///     registerAccount's #9 singleton pin and #4 owner-signature
    gate rely on all three.
37  interface ISafe {
38   function execTransactionFromModule(address to, uint256 value,
    bytes calldata data, uint8 operation)
39   external
40   returns (bool success);
41
42   function execTransactionFromModuleReturnData(address to,
    uint256 value, bytes calldata data, uint8 operation)
43   external
44   returns (bool success, bytes memory returnData);
45
46   function isModuleEnabled(address module) external view returns
    (bool);
47
48  /// @notice The Safe's transaction nonce. Incremented by
    `execTransaction` (before the inner
49  ///         call runs) and never by `setup` — so a value of 0
    proves the Safe has not yet
50  ///         executed an owner-approved transaction (used to
    reject setup-time registration).
51   function nonce() external view returns (uint256);
```

```
52
53  /// @notice The Safe singleton (master copy) the proxy delegates
    to. On a genuine SafeProxy
54  ///         v1.4.1 this is intercepted by the proxy's own
    fallback (selector 0xa619486e) and
55  ///         read from storage slot 0, so it cannot be forged
    by a hostile singleton.
56   function masterCopy() external view returns (address);
57
58  /// @notice Validate `signatures` over `dataHash` against the
    Safe's owner set and threshold.
59  ///         Safe-core method (not the fallback handler's
    ERC-1271 entrypoint), so it carries no
60  ///         dependency on which fallbackHandler is configured.
    Reverts (GS0xx) on any failure;
61  ///         returns nothing on success. `data` is only consulted
    for legacy contract-signature
62  ///         (v==0) entries (requires keccak256(data)==dataHash);
    empty for EOA / approved-hash.
63   function checkSignatures(bytes32 dataHash, bytes calldata
    data, bytes calldata signatures) external view;
64  }
65
66  /// @title  SailKernel
67  /// @notice Central execution kernel for the Sail protocol.
68  ///
69  ///         The kernel manages a registry of Safe accounts, each
    with a set of
70  ///         permission contracts. When a manager submits a signed
    transaction, the
71  ///         kernel:
72  ///           1. Verifies the manager's EIP-712 signature and
    nonce.
73  ///           2. Evaluates the named registered permission (only
    the selected permission must return true).
74  ///           3. Executes the transaction via Safe's module
    interface.
75  ///
76  ///         Key design properties:
77  ///           • Deny-by-default: accounts with no registered
    permissions cannot dispatch.
78  ///           • Permission cap: governance-tunable limit (1–100)
    per account to bound
79  ///             gas usage in the dispatch loop; read from
    SailGovernance at registration time.
80  ///           • Salt binding: `createAccount` binds the CREATE2
    salt to msg.sender to
```

```
81  ///           prevent front-running attacks on Safe registration.
82  ///           • Two-tier nonces: manager nonces gate dispatch;
    signer nonces gate
83  ///             permission-registry operations, preventing
    cross-operation replay.
84  ///         • ERC-1271 support: both manager and permissionSigner
    may be smart contracts.
85  ///
86  /// @custom:security-contact security@sail.money
87  contract SailKernel is EIP712, ReentrancyGuard {
88  //
    ----------------------------------------------------------------------
89  // Constants
90  //
    ----------------------------------------------------------------------
91
92  /// @notice Gas budget allocated to each permission's `evaluate`
    staticcall.
93  ///         A revert or gas exhaustion within a permission is
    treated as a false return.
94  ///         Increased to 150_000 to accommodate templates
    (e.g. GMXPerpPermission,
95  ///         AzuroPredictionPermission, LimitlessPrediction-
    Permission) that use
96  ///         `try this._decode*(...)` external calls within
    their evaluate paths.
97   uint256 public constant PERMISSION_GAS_CAP = 150_000;
98
99  /// @notice Maximum number of subcalls in a single batch
    dispatch.
100 /// @dev    Bounds gas consumption in the subcall execution
    loop. A manager that needs
101 ///         more than this can split into multiple consecutive
    batch dispatches.
102  uint256 public constant MAX_BATCH_LENGTH = 16;
103
104 /// @notice Gas budget allocated to a batch permission's
    `evaluateBatch` staticcall.
105 /// @dev    Higher than PERMISSION_GAS_CAP because the batch
    evaluator must inspect
106 ///         every subcall's calldata; a revert, OOG, or malformed
    return is treated
107 ///         as a false return (fail-closed).
108  uint256 public constant BATCH_EVAL_GAS_CAP = 1_000_000;
109
110 /// @dev Gas budget for the `isBatchPermission()` type-detec-
    tion staticcall. A view
```

```
111 ///      function returning a bool should comfortably fit; a
    larger budget would
112 ///      enlarge the attack surface without benefit.
113  uint256 private constant BATCH_DETECT_GAS_CAP = 20_000;
114
115 /// @dev Large nonce step applied to managerNonces/batchNonces
    when a signer restriction op
116 ///      (revokePermission, replacePermission, revokeSession,
    revokePermissions) takes effect.
117 ///      Invalidates any outstanding manager-signed dispatches
    that pre-date the restriction
118 ///      without requiring separate nonce-rotation messages
    (Octane finding #7 related).
119  uint256 private constant NONCE_EPOCH_INCREMENT = 1 << 128;
120
121 /// @dev ERC-1271 magic value returned by `isValidSignature`
    for a valid signature.
122  bytes4  private constant ERC1271_MAGIC           = 0x1626ba7e;
123
124 /// @dev Exact calldata length of `setManager(address)`:
    selector (4) + 1 static arg (32).
125 ///      Used by the W1 fallback-relay guard — a Safe
    FallbackManager relay appends the
126 ///      original caller's 20 bytes, so any deviation from
    this length flags a relayed call.
127  uint256 private constant SET_MANAGER_CALLDATA_LEN  = 36;
128
129 /// @dev Exact calldata length of `collectFees(ad-
    dress,uint256,uint256,address)`:
130 ///      selector (4) + 4 static args (128). See
    `SET_MANAGER_CALLDATA_LEN` for the W1 rationale.
131  uint256 private constant COLLECT_FEES_CALLDATA_LEN = 132;
132
133 //
    ------------------------------------------------------------------------
134 // EIP-712 type hashes
135 //
    ------------------------------------------------------------------------
136
137 /// @notice EIP-712 type hash for manager dispatch authorisa-
    tion.
138 ///         Type string: "Dispatch(address account,address
    permission,address target,uint256 value,bytes32 dataHash,uint256
    nonce,uint256 deadline)"
139 /// @dev BREAKING CHANGE from v1: `permission` field added.
    Any pre-signed dispatch
140
```

```
///      messages created against the v1 typehash are permanently invalid after this upgrade.
///      Off-chain systems (keeper bots, SDK integrations) must re-generate signatures.
 bytes32 public constant DISPATCH_TYPEHASH = keccak256(
"Dispatch(address account,address permission,address target,uint256 value,bytes32 dataHash,uint256 nonce,uint256 deadline)"
);

/// @notice EIP-712 type hash for single-permission registration.
///         Type string: "RegisterPermission(address account,address permission,uint256 nonce,uint256 deadline)"
 bytes32 public constant REGISTER_PERMISSION_TYPEHASH = keccak256(
"RegisterPermission(address account,address permission,uint256 nonce,uint256 deadline)"
);

/// @notice EIP-712 type hash for single-permission revocation.
///         Type string: "RevokePermission(address account,address permission,uint256 nonce,uint256 deadline)"
 bytes32 public constant REVOKE_PERMISSION_TYPEHASH = keccak256(
"RevokePermission(address account,address permission,uint256 nonce,uint256 deadline)"
);

/// @notice EIP-712 type hash for atomic permission replacement.
///         Type string: "ReplacePermission(address account,address oldPermission,address newPermission,uint256 nonce,uint256 deadline)"
 bytes32 public constant REPLACE_PERMISSION_TYPEHASH = keccak256(
"ReplacePermission(address account,address oldPermission,address newPermission,uint256 nonce,uint256 deadline)"
);

/// @notice EIP-712 type hash for atomic batch permission replacement.
///         Type string: "ReplacePermissions(address account,address[] oldPermissions,address[] newPermissions,uint256 nonce,uint256 deadline)"
 bytes32 public constant REPLACE_PERMISSIONS_TYPEHASH = keccak256(
"ReplacePermissions(address account,address[] oldPermissions,address[] newPermissions,uint256 nonce,uint256 deadline)"
```

```
168  );
169
170  /// @notice EIP-712 type hash for session revocation.
171  ///         Type string: "RevokeSession(address account,uint256
     nonce,uint256 deadline)"
172   bytes32 public constant REVOKE_SESSION_TYPEHASH = keccak256(
173  "RevokeSession(address account,uint256 nonce,uint256
     deadline)"
174  );
175
176  /// @notice EIP-712 type hash for session re-activation.
177  ///         Type string: "ActivateSession(address account,uint256
     nonce,uint256 deadline)"
178   bytes32 public constant ACTIVATE_SESSION_TYPEHASH = keccak256(
179  "ActivateSession(address account,uint256 nonce,uint256
     deadline)"
180  );
181
182  /// @notice EIP-712 type hash for fee policy updates.
183  ///         Type string: "SetFeePolicy(address account,address
     newFeePolicy,address feeAsset,uint256 nonce,uint256 deadline)"
184   bytes32 public constant SET_FEE_POLICY_TYPEHASH = keccak256(
185  "SetFeePolicy(address account,address newFeePolicy,address
     feeAsset,uint256 nonce,uint256 deadline)"
186  );
187
188  /// @notice EIP-712 type hash for batch permission registration.
189  ///         Type string: "RegisterPermissions(address
     account,address[] permissions,uint256 nonce,uint256 deadline)"
190  ///         The `address[]` field is encoded as keccak256 of
     the ABI-packed padded addresses
191  ///         per EIP-712 §4 (see `_hashAddressArray`).
192   bytes32 public constant REGISTER_PERMISSIONS_TYPEHASH =
     keccak256(
193  "RegisterPermissions(address account,address[] permis-
     sions,uint256 nonce,uint256 deadline)"
194  );
195
196  /// @notice EIP-712 type hash for batch permission revocation.
197  ///         Type string: "RevokePermissions(address
     account,address[] permissions,uint256 nonce,uint256 deadline)"
198   bytes32 public constant REVOKE_PERMISSIONS_TYPEHASH =
     keccak256(
199  "RevokePermissions(address account,address[] permis-
     sions,uint256 nonce,uint256 deadline)"
200  );
201
```

```
202  /// @notice EIP-712 type hash for manager batch-dispatch
     authorisation.
203  ///         Type string: "DispatchBatch(address account,address
     permission,bytes32 callsHash,uint256 nonce,uint256 deadline)"
204  ///         `callsHash` = keccak256(abi.encode(calls)) — see
     `dispatchBatch` for the encoding.
205   bytes32 public constant DISPATCH_BATCH_TYPEHASH = keccak256(
206  "DispatchBatch(address account,address permission,bytes32
     callsHash,uint256 nonce,uint256 deadline)"
207  );
208
209  /// @notice EIP-712 type hash for owner-authorised self-reg-
     istration via `registerAccount`.
210  ///         Type string: "RegisterAccount(address account,ad-
     dress permissionSigner,address manager,address feePolicy,address
     feeAsset,uint256 deadline)"
211  /// @dev    The owner-set+threshold signature over this struct
     is the robust gate for the
212  ///         self-registration path: it cannot be produced by
     a Safe.setup delegatecall helper
213  ///         (which has no owner keys). No nonce field —
     registration is one-shot (the
214  ///         `registered[account]` latch is never cleared, so
     a replayed signature reverts with
215  ///         AccountAlreadyRegistered) and the EIP-712 domain
     pins chainId against cross-chain replay.
216   bytes32 public constant REGISTER_ACCOUNT_TYPEHASH = keccak256(
217  "RegisterAccount(address account,address permission-
     Signer,address manager,address feePolicy,address feeAsset,uint256
     deadline)"
218  );
219
220  //
     --------------------------------------------------------------------------
221  // Account state
222  //
     --------------------------------------------------------------------------
223
224  /// @notice Per-account configuration set at registration and
     updatable via signed ops.
225   struct AccountConfig {
226  /// @dev Address whose signatures authorise permission-reg-
     istry operations.
227   address permissionSigner;
228  /// @dev Address whose signatures authorise dispatch calls.
229   address manager;
230
```

```
    /// @dev Fee policy contract; address(0) means no fee
    policy is configured.
231  address feePolicy;
232 /// @dev Canonical fee settlement token; address(0) =
    native ETH.
233  address feeAsset;
234 /// @dev When false, all dispatch calls for this account
    are blocked.
235  bool    sessionActive;
236 }
237
238 /// @notice Enriched permission descriptor returned by
    getPermissionsWithInfo.
239  struct PermissionInfo {
240 /// @dev Contract address of the permission.
241  address permission;
242 /// @dev True if the permission implements IBatchPermis-
    sion.isBatchPermission().
243  bool    isBatch;
244 /// @dev True if the permission implements IPermissionIn-
    trospection.
245  bool    hasIntrospection;
246 /// @dev From IPermissionIntrospection.permissionId();
    bytes32(0) if not supported.
247  bytes32 permissionId;
248 /// @dev From IPermissionIntrospection.permissionVersion();
    bytes32(0) if not supported.
249  bytes32 permissionVersion;
250 }
251
252 /// @notice Per-account configuration.
253  mapping(address account => AccountConfig)                    pub-
    lic  configs;
254
255 /// @notice Whether an account has been registered with the
    kernel.
256  mapping(address account => bool)                           pub-
    lic  registered;
257
258 /// @dev Ordered list of permission addresses per account.
    Maintained as a packed array
259 ///      with swap-and-pop removal to keep indices compact.
    Max length: governance.maxPermissionsPerAccount().
260  mapping(address account => address[])                      pri-
    vate _permissions;
261
262
```

```
    /// @dev Index-plus-one of each permission in `_permissions[ac-
    count]`.
263 ///      Zero means the permission is not registered. Stored
    as index+1 to distinguish
264 ///      "registered at slot 0" from "not registered".
265  mapping(address account => mapping(address permission =>
    uint256)) private _permissionIndex;
266
267 /// @notice Per-account nonces for manager dispatch signatures.
268 ///         Separate from signerNonces to prevent cross-operation
    replay.
269  mapping(address account => uint256) public managerNonces;
270
271 /// @notice Per-account nonces for manager batch-dispatch
    signatures.
272 /// @dev    Lives in a separate namespace from `managerNonces`
    so that single
273 ///         dispatches and batch dispatches cannot replay
    across each other.
274 ///         Consuming a batch nonce does not advance dispatch
    nonces, and vice versa.
275  mapping(address account => uint256) public batchNonces;
276
277 /// @notice Per-account nonces for permissionSigner operations
278 ///         (register, revoke, replace, session, feePolicy).
279 /// @dev    All signer operations share a single nonce counter
    per account.
280 ///         Concurrent independent operations must use batch
    variants (registerPermissions,
281 ///         revokePermissions) rather than multiple single-op
    calls. Submitting two
282 ///         single-op calls simultaneously will cause one to
    fail due to nonce conflict.
283  mapping(address account => uint256) public signerNonces;
284
285 /// @notice Per-(account, permission) registration epoch. A
    plain monotonic counter, bumped
286 ///         every time a permission LEAVES an account's registry
    (revoke, the removed side of a
287 ///         replace, or a manager-rotation clear). It is NOT
    bumped on registration: a freshly
288 ///         registered permission keeps its epoch so the
    configure-then-register flow (see
289 ///         MandateFactory) stamps the same epoch the dispatch
    later reads.
290 /// @dev    Pushed into Context/BatchContext at dispatch time
    so a ConfigurablePermission can
```

```
291 ///         compare it against the epoch it stamped at
    configure() time and fail closed when a
292 ///         stale configuration survives a revoke 're-register
    cycle (Octane #2 / #8). Distinct
293 ///         from the packed manager/batch nonce epochs
    (NONCE_EPOCH_INCREMENT) — this is a clean
294 ///         standalone +1 counter per (account, permission).
295  mapping(address account => mapping(address permission =>
    uint256)) public registrationEpoch;
296
297 //
    ------------------------------------------------------------------------
298 // Principal tracking
299 //
    ------------------------------------------------------------------------
300
301 /// @notice Cumulative deposit amount recorded for each account
    (informational).
302 ///         Written by the permissionSigner via `recordDeposit`.
303  mapping(address account => uint256) public cumulativeDeposits;
304
305 /// @notice Cumulative withdrawal amount recorded for each
    account (informational).
306 ///         Written by the permissionSigner via
    `recordWithdrawal`.
307  mapping(address account => uint256) public cumulativeWith-
    drawals;
308
309 /// @dev Records the single fee asset a policy instance has
    been bound to for an account.
310 ///      `bound` is an explicit flag (not asset != 0) because
    address(0) is a valid fee
311 ///      asset (native ETH), so it cannot double as the "unbound"
    sentinel.
312  struct PolicyAssetBinding {
313  bool    bound;
314  address asset;
315 }
316
317 /// @notice Per-(account, policy) binding pinning a fee-policy
    instance to the single fee
318 ///         asset it was first used with for that account.
    Reusing the SAME policy instance
319 ///         with a DIFFERENT asset would leave the policy's
    persisted high-water mark in
320 ///         stale units, inflating the next performance fee
    on a one-time over-collection.
```

```
321 ///         To change the fee asset, point the account at a
    fresh policy instance, which
322 ///         carries its own fresh per-account state.
323  mapping(address account => mapping(address policy => PolicyAs-
    setBinding)) private _policyAssetBinding;
324
325 //
    ------------------------------------------------------------------------
326 // Protocol references
327 //
    ------------------------------------------------------------------------
328
329 /// @notice The governance contract that stores fee parameters
    and the protocol cut.
330  SailGovernance public immutable governance;
331
332 /// @notice Runtime codehash of the audited, immutable
    SafeModuleEnabler this kernel pins as
333 ///         the ONLY permissible Safe.setup delegatecall `to`
    target (W2). Captured at
334 ///         construction from the deployed helper, so it
    matches the launch build by
335 ///         construction — no hand-copied literal, no
    recompile-mismatch risk. `createAccount`
336 ///         requires every non-zero `setupTarget` to carry
    exactly this codehash, so a
337 ///         governance mistake allowlisting a mutable/look-alike
    helper cannot open the
338 ///         setup-delegatecall takeover surface.
339  bytes32         public immutable EXPECTED_SETUP_CODEHASH;
340
341 /// @notice Address that receives the protocol's share of
    collected fees.
342  address         public treasury;
343
344 //
    ------------------------------------------------------------------------
345 // Events
346 //
    ------------------------------------------------------------------------
347
348 /// @notice Emitted when a new account is registered.
349 /// @param  account         The Safe address that was registered.
350 /// @param  permissionSigner Address authorised to manage
    permissions.
351 /// @param  manager          Address authorised to dispatch
    transactions.
```

```
352   event AccountRegistered(address indexed account, address
      indexed permissionSigner, address indexed manager);
353
354   /// @notice Emitted when a permission is added to an account.
355   /// @param  account    The Safe account.
356   /// @param  permission The permission contract address.
357   event PermissionRegistered(address indexed account, address
      indexed permission);
358
359   /// @notice Emitted when a permission is removed from an
      account.
360   /// @param  account    The Safe account.
361   /// @param  permission The permission contract address that
      was removed.
362   event PermissionRevoked(address indexed account, address
      indexed permission);
363
364   /// @notice Emitted when a permission is atomically replaced.
365   /// @param  account        The Safe account.
366   /// @param  oldPermission  Permission that was removed.
367   /// @param  newPermission  Permission that was added in its
      place.
368   event PermissionReplaced(address indexed account, address
      indexed oldPermission, address indexed newPermission);
369
370   /// @notice Emitted when the manager session is suspended for
      an account.
371   /// @param  account The Safe account whose session was revoked.
372   event SessionRevoked(address indexed account);
373
374   /// @notice Emitted when a previously revoked session is
      re-activated.
375   /// @param  account The Safe account whose session was
      activated.
376   event SessionActivated(address indexed account);
377
378   /// @notice Emitted when the fee policy for an account is
      updated.
379   /// @param  account       The Safe account.
380   /// @param  newFeePolicy  The new fee policy contract address
      (address(0) = cleared).
381   event FeePolicyUpdated(address indexed account, address indexed
      newFeePolicy);
382
383   /// @notice Emitted when an account's manager (delegated
      signer) is rotated.
384
```

```
    /// @dev    Rotation clears the account's permission set, so
    a `ManagerChanged` is
385 ///         accompanied by a `PermissionRevoked` for each
    previously-registered mandate.
386 /// @param  account    The Safe account whose manager changed.
387 /// @param  oldManager The previous manager address.
388 /// @param  newManager The new manager address.
389  event ManagerChanged(address indexed account, address indexed
    oldManager, address indexed newManager);
390
391 /// @notice Emitted on each successful dispatch.
392 /// @dev    `dataHash` is keccak256(calldata) — the raw bytes
    are recoverable from the tx.
393 /// @param  account     The Safe account that executed the
    transaction.
394 /// @param  permission  The registered permission that
    authorised the call.
395 /// @param  target      The call target.
396 /// @param  selector    Leading 4 bytes of calldata; bytes4(0)
    if calldata is shorter than 4 bytes.
397 /// @param  value       Native ETH forwarded with the call
    (wei).
398  event Dispatched(
399  address indexed account,
400  address indexed permission,
401  address target,
402  bytes4  selector,
403  uint256 value
404 );
405
406 /// @notice Emitted on each successful batch dispatch.
407 /// @param  account    The Safe account that executed the
    batch.
408 /// @param  permission The batch-aware permission that
    authorised the batch.
409 /// @param  batchHash  keccak256(abi.encode(calls)) — stable
    identifier for the call sequence.
410 /// @param  callCount  Number of subcalls in the batch.
411  event BatchDispatched(
412  address indexed account,
413  address indexed permission,
414  bytes32 batchHash,
415  uint256 callCount
416 );
417
418 /// @notice Emitted on each successful fee collection.
419 /// @param  account        The Safe account that was charged.
```

```
420 /// @param  feeToken       ERC-20 token used for fee payment;
    address(0) = native ETH.
421 /// @param  grossFee       Total fee collected.
422 /// @param  protocolCut    Portion forwarded to the treasury.
423 /// @param  distributorCut Portion forwarded to the fee policy's
    distributor.
424 /// @param  managerTake    Remainder forwarded to the manager's
    recipient.
425  event FeesCollected(
426  address indexed account,
427  address indexed feeToken,
428  uint256 grossFee,
429  uint256 protocolCut,
430  uint256 distributorCut,
431  uint256 managerTake
432 );
433
434 /// @notice Emitted when a deposit is recorded for an account.
435 /// @param  account    The Safe account.
436 /// @param  amount     Amount deposited in this call.
437 /// @param  cumulative Running cumulative deposit total after
    this call.
438  event DepositRecorded(address indexed account, uint256 amount,
    uint256 cumulative);
439
440 /// @notice Emitted when a withdrawal is recorded for an
    account.
441 /// @param  account    The Safe account.
442 /// @param  amount     Amount withdrawn in this call.
443 /// @param  cumulative Running cumulative withdrawal total
    after this call.
444  event WithdrawalRecorded(address indexed account, uint256
    amount, uint256 cumulative);
445
446 /// @notice Emitted when the treasury address is updated.
447 /// @param  oldTreasury Previous treasury address.
448 /// @param  newTreasury New treasury address.
449  event TreasuryUpdated(address indexed oldTreasury, address
    indexed newTreasury);
450
451 //
    ------------------------------------------------------------------------
452 // Errors
453 //
    ------------------------------------------------------------------------
454
455
```

```
    /// @dev Thrown by `_registerAccount` when the account is
    already registered.
456  error AccountAlreadyRegistered(address account);
457
458 /// @dev Thrown by `_requireRegistered` when the account has
    not been registered.
459  error AccountNotRegistered(address account);
460
461 /// @dev Thrown by `dispatch` when `sessionActive` is false
    for the account.
462  error SessionInactive(address account);
463
464 /// @dev Thrown when `block.timestamp` exceeds the provided
    deadline.
465  error DeadlineExpired(uint256 deadline, uint256 current);
466
467 /// @dev Thrown when an EIP-712 manager signature cannot be
    verified.
468  error InvalidManagerSignature();
469
470 /// @dev Thrown when an EIP-712 permissionSigner signature
    cannot be verified.
471  error InvalidSignerSignature();
472
473 /// @dev Thrown when a permission returns false (or reverts)
    during dispatch.
474  error PermissionDenied(address permission);
475
476 /// @dev Thrown when `ISafe.execTransactionFromModule` returns
    false.
477  error SafeExecutionFailed();
478
479 /// @dev Thrown by `registerPermission` / `registerPermissions`
    when the permission
480 ///      is already in the account's permission set.
481  error PermissionAlreadyRegistered(address permission);
482
483 /// @dev Thrown by operations that require a permission to be
    registered when it is not.
484  error PermissionNotRegistered(address permission);
485
486 /// @dev Thrown when adding permissions would exceed
    governance.maxPermissionsPerAccount().
487  error TooManyPermissions(address account, uint256 limit);
488
489 /// @dev Thrown when the ETH sent with a registration call is
    below the required fee.
```

```
490   error InsufficientFee(uint256 required, uint256 provided);
491
492   /// @dev Thrown by `collectFees` when no fee policy is set
      for the account.
493   error FeePolicyNotSet();
494
495   /// @dev Thrown by `collectFees` when `grossFee` exceeds the
      policy's computed maximum.
496   error FeeTooLarge(uint256 requested, uint256 maxAllowed);
497
498   /// @dev Thrown when an ETH or ERC-20 fee transfer via the
      Safe fails.
499   error FeeTransferFailed();
500
501   /// @dev Thrown when the caller of a manager-only function is
      not the account's manager.
502   error NotManager(address caller, address expected);
503
504   /// @dev Thrown by `onlyGovernance` when caller is not the
      current governance address.
505   error NotGovernance();
506
507   /// @dev Thrown by `onlyTimelock` when caller is not the
      governance 48-hour timelock.
508   error NotTimelock();
509
510   /// @dev Thrown when a caller other than the account's
      permissionSigner invokes a guarded op.
511   error NotPermissionSigner();
512
513   /// @dev Thrown when a required address argument is the zero
      address.
514   error ZeroAddress();
515
516   /// @dev Thrown by `setManager` when the new manager equals
      the current one — a no-op
517   ///      rotation would needlessly clear mandates and bump
      the nonce epoch.
518   error ManagerUnchanged();
519
520   /// @dev Thrown by permission registration when the supplied
      address has no deployed code.
521   error NotAContract(address addr);
522
523   /// @dev Thrown by `collectFees` when `distributorBps` returned
      by the policy exceeds 10 000.
524   error DistributorBpsTooLarge(uint256 bps);
```

```
525
526 /// @dev Thrown by `collectFees` when the fee token does not
    match the account's configured canonical asset.
527  error FeeTokenMismatch(address provided, address expected);
528
529 /// @dev Thrown by `collectFees` when `grossFee` is zero.
530  error ZeroFee();
531
532 /// @dev Thrown by `dispatch` / `collectFees` when the protocol
    is paused.
533  error ProtocolPaused();
534
535 /// @dev Thrown when `createAccount` deploys a Safe that does
    not have this kernel enabled as a module.
536  error ModuleNotEnabled();
537
538 /// @dev Thrown by `registerAccount` when the caller Safe has
    not finalized setup (nonce == 0),
539 ///      blocking a setup-delegatecall helper from registering
    attacker-chosen principals (Octane #4).
540  error SetupNotFinalized();
541
542 /// @dev Thrown by Safe-authorized functions (`registerAc-
    count`, `setManager`, `collectFees`) when
543 ///      msg.data is not the exact static length — a Safe
    fallback relay appends the caller's 20
544 ///      bytes, so a length mismatch flags a fallbackHan-
    dler-relayed call (Octane W1).
545  error UnexpectedCalldataLength();
546
547 /// @dev Thrown by `createAccount` when the provided Safe
    factory is not in governance's trusted allowlist.
548  error UntrustedFactory(address factory);
549
550 /// @dev Thrown by `createAccount` when the provided Safe
    singleton is not in governance's trusted allowlist.
551  error UntrustedSingleton(address singleton);
552
553 /// @dev Thrown by `createAccount` when `safeInitializer` is
    too short to contain the
554 ///      Safe.setup `to` field (selector + owners offset +
    threshold + to = 100 bytes).
555  error InvalidInitializer();
556
557 /// @dev Thrown by `createAccount` when the Safe.setup
    delegatecall `to` target is not
558 ///      in governance's trusted module-setup allowlist.
```

```
559   error UntrustedModuleSetup(address setup);
560
561   /// @dev Thrown by `createAccount` when the Safe.setup
      delegatecall `to` target's runtime
562   ///      codehash does not match the immutable, audited
      SafeModuleEnabler pinned at deploy
563   ///      (`EXPECTED_SETUP_CODEHASH`). Defends against a
      governance mistake allowlisting a
564   ///      MUTABLE helper at a trusted address: even an
      address-allowlisted target is rejected
565   ///      unless its deployed bytecode is byte-identical to
      the pinned immutable helper (W2).
566   error UntrustedModuleSetupCodehash(address setup);
567
568   /// @dev Thrown by `registerAccount` when the caller's runtime
      codehash is not in
569   ///      governance's trusted Safe-proxy-codehash allowlist.
570   error UntrustedProxyCodehash(bytes32 codehash);
571
572   /// @dev Thrown by `dispatchBatch` when the calls array is
      empty.
573   error EmptyBatch();
574
575   /// @dev Thrown by `dispatchBatch` when the calls array exceeds
      MAX_BATCH_LENGTH.
576   error BatchTooLong(uint256 length);
577
578   /// @dev Thrown by `dispatchBatch` when the named permission
      does not implement IBatchPermission
579   ///      (detected via try/catch on `isBatchPermission()`).
580   error PermissionNotBatchAware(address permission);
581
582   /// @dev Thrown by `dispatchBatch` when the named permission's
      `evaluateBatch`
583   ///      returns false, reverts, runs out of gas, or returns
      malformed data.
584   error BatchPermissionDenied();
585
586   /// @dev Thrown by `dispatchBatch` when one of the batched
      subcalls reverts
587   ///      (Safe's execTransactionFromModule returns false).
588   error BatchSubcallFailed(uint256 index, address target);
589
590   /// @dev Thrown by `dispatchBatch` when a subcall targets the
      kernel itself —
591   ///      a defensive guard preventing self-targeted reentrancy
      attempts.
```

```
592   error KernelSelfTarget(uint256 index);
593
594   /// @dev Thrown by `dispatchBatch` when a subcall targets the
      zero address.
595   error BatchZeroTarget(uint256 index);
596
597   /// @dev Thrown by `dispatch`/`dispatchBatch` when a call
      targets the Safe account itself.
598   ///      Prevents module-triggered self-calls that satisfy
      Safe's onlySelf guard and could
599   ///      enable owner/threshold changes, module manipulation,
      or guard/fallback overwrites.
600   error AccountSelfTarget();
601
602   /// @dev Thrown by `_registerAccount` or `setFeePolicy` when
      the provided fee policy is not
603   ///      in governance's trusted allowlist.  Prevents
      upgradeable/metamorphic policies.
604   error UntrustedFeePolicy(address policy);
605
606   /// @dev Thrown by `setFeePolicy` when a fee-policy instance
      already used for an account
607   ///      with one fee asset is reused with a different asset.
      Reusing an instance across
608   ///      denominations would leave its persisted high-water
      mark in stale units. To change
609   ///      the fee asset, point the account at a fresh policy
      instance.
610   error FeePolicyAssetMismatch(address policy, address expected,
      address provided);
611
612   /// @dev Thrown by `replacePermissions` when `oldPermissions`
      and `newPermissions` have different lengths.
613   error ArrayLengthMismatch();
614
615   //
      ------------------------------------------------------------------------
616   // Constructor
617   //
      ------------------------------------------------------------------------
618
619   /// @notice Deploy the kernel with a governance contract,
      treasury, and the Safe.setup helper.
620   /// @param  _governance    Address of the deployed SailGovernance
      contract.
621   /// @param  _treasury      Address that will receive the
      protocol's share of fees.
```

```
622  /// @param  _setupEnabler  The deployed, immutable SafeMod-
     uleEnabler. Its runtime codehash is
623  ///                       captured into `EXPECTED_SETUP_CODEHASH`
     and pinned as the only
624  ///                       permissible Safe.setup delegatecall
     target (W2). MUST be the genuine
625  ///                       immutable helper at launch; it must
     already be deployed when the
626  ///                       kernel is constructed so its
     codehash can be read here.
627   constructor(address _governance, address _treasury, address
     _setupEnabler) EIP712("SailKernel", "1") {
628   if (_governance == address(0) || _treasury == address(0)
     || _treasury == address(this)) revert ZeroAddress();
629   governance             = SailGovernance(_governance);
630   treasury               = _treasury;
631   EXPECTED_SETUP_CODEHASH = _setupEnabler.codehash;
632  }
633
634  //
     ------------------------------------------------------------------------
635  // Modifiers
636  //
     ------------------------------------------------------------------------
637
638  /// @dev Reverts with NotGovernance when caller is not the
     current governance address.
639   modifier onlyGovernance() {
640   if (msg.sender != governance.governance()) revert
     NotGovernance();
641  _;
642  }
643
644  /// @dev Reverts with NotTimelock when caller is not the
     governance timelock.
645  ///      Used for high-impact setters that must observe the
     48-hour delay.
646   modifier onlyTimelock() {
647   if (msg.sender != address(governance.timelock())) revert
     NotTimelock();
648  _;
649  }
650
651  /// @dev Reverts with ProtocolPaused when governance.isPaused()
     returns true.
652   modifier whenNotPaused() {
653   if (governance.isPaused()) revert ProtocolPaused();
```

```
654  _;
655  }
656
657  //
     ------------------------------------------------------------------------
658  // Governance
659  //
     ------------------------------------------------------------------------
660
661  /// @notice Update the treasury address that receives the
     protocol's fee share.
662  /// @dev    Enforces the 48-hour timelock so that a compromised
     governance key cannot
663  ///         instantly redirect all protocol fee flows. Schedule
     via governance.timelock().
664  /// @param  newTreasury New treasury address. Must not be the
     zero address.
665   function setTreasury(address newTreasury) external onlyTime-
     lock {
666   if (newTreasury == address(0)) revert ZeroAddress();
667   if (newTreasury == address(this)) revert ZeroAddress();
668   address old = treasury;
669   treasury = newTreasury;
670   emit TreasuryUpdated(old, newTreasury);
671  }
672
673  //
     ------------------------------------------------------------------------
674  // 1. Account instantiation
675  //
     ------------------------------------------------------------------------
676
677  /// @notice Deploy a new Safe via factory and register it with
     the kernel in one transaction.
678  /// @dev    The salt passed to the factory is derived from
     `keccak256(saltNonce, msg.sender,
679  ///         permissionSigner, manager, feePolicy)` so that a
     front-runner supplying different
680  ///         principals lands at a different CREATE2 address
     and cannot squat the registration
681  ///         (Octane #4 / #16). The Safe.setup delegatecall
     `to` target embedded in
682  ///         `safeInitializer` is validated against the trusted
     module-setup allowlist, removing
683  ///         the arbitrary-delegatecall surface (Octane #1).
     If a proxy already exists at the
684
```

```
    ///         predicted address (legitimate pre-deploy or retry)
    the factory call is skipped.
685 /// @param  safeFactory       Address of the Safe proxy factory
    contract.
686 /// @param  safeSingleton     Address of the Safe singleton
    (implementation) contract.
687 /// @param  safeInitializer   Calldata for the Safe's `setup`
    call during deployment.
688 /// @param  saltNonce         Caller-chosen nonce; combined
    with msg.sender + principals into the salt.
689 /// @param  permissionSigner  Address that will sign
    permission-registry operations.
690 /// @param  manager           Address that will sign dispatch
    calls.
691 /// @param  feePolicy         Fee policy contract; address(0)
    = no fee policy.
692 /// @param  feeAsset          Canonical fee settlement token;
    address(0) = native ETH.
693 /// @return account           Address of the deployed (or
    pre-existing) Safe proxy.
694  function createAccount(
695  address safeFactory,
696  address safeSingleton,
697  bytes calldata safeInitializer,
698  uint256 saltNonce,
699  address permissionSigner,
700  address manager,
701  address feePolicy,
702  address feeAsset
703 ) external returns (address account) {
704  if (!governance.trustedSafeFactory(safeFactory))     revert
    UntrustedFactory(safeFactory);
705  if (!governance.trustedSafeSingleton(safeSingleton)) revert
    UntrustedSingleton(safeSingleton);
706
707 // Enforce that the delegatecall target inside Safe.setup
    is an allowlisted helper.
708 // setup() ABI layout: selector(4) + owners_offset(32) +
    threshold(32) + to(32) + ...
709 // 'to' is at bytes [68:100].
710  if (safeInitializer.length < 100) revert InvalidInitial-
    izer();
711  address setupTarget = ad-
    dress(uint160(uint256(bytes32(safeInitializer[68:100]))));
712 // address(0) means no delegatecall (vanilla Safe.setup)
    — always safe, no allowlist check needed.
713
```

```
     if (setupTarget != address(0) && !governance.trustedMod-
    uleSetup(setupTarget)) revert UntrustedModuleSetup(setupTarget);
714  // W2: address-allowlisting alone is not enough — pin the
    target's runtime codehash to the
715  // audited immutable SafeModuleEnabler captured at deploy.
    This converts the operational rule
716  // ("only ever allowlist an IMMUTABLE helper") into a code
    guarantee: a governance mistake
717  // allowlisting a mutable/upgradeable look-alike at a
    trusted address cannot satisfy the pin.
718  // Gated on setupTarget != address(0) exactly like the
    address check, so the no-setup path
719  // (vanilla Safe.setup) is unaffected.
720   if (setupTarget != address(0) && setupTarget.codehash !=
    EXPECTED_SETUP_CODEHASH) {
721   revert UntrustedModuleSetupCodehash(setupTarget);
722  }
723
724   uint256 boundSalt = uint256(keccak256(abi.encode(saltNonce,
    msg.sender, permissionSigner, manager, feePolicy)));
725
726  // Predict the proxy address with the same CREATE2 formula
    SafeProxyFactory uses, so a
727  // proxy already deployed at that address (a retry, or a
    same-config front-runner) is
728  // adopted instead of reverting. Computed locally rather
    than via a factory predictor:
729  // Safe v1.4.1's factory exposes no view predictor (only
    proxyCreationCode()).
730  //   salt              = keccak256(keccak256(initializer),
    saltNonce)
731  //   deploymentData = proxyCreationCode() ++
    uint256(uint160(singleton))
732   bytes32 create2Salt = keccak256(abi.encodePacked(kec-
    cak256(safeInitializer), boundSalt));
733   bytes32 initCodeHash = keccak256(
734   abi.encodePacked(ISafeFactory(safeFactory).proxyCre-
    ationCode(), uint256(uint160(safeSingleton)))
735  );
736   address predicted = address(
737   uint160(uint256(keccak256(abi.encodePacked(bytes1(0xff),
    safeFactory, create2Salt, initCodeHash))))
738  );
739   if (predicted.code.length == 0) {
740   account = ISafeFactory(safeFactory).createProxyWith-
    Nonce(safeSingleton, safeInitializer, boundSalt);
741  } else {
```

```
742   account = predicted;
743  }
744
745   if (!ISafe(account).isModuleEnabled(address(this))) revert
     ModuleNotEnabled();
746
747  // Parity with registerAccount: the resulting proxy must
     carry an allowlisted Safe-proxy
748  // runtime codehash. The trusted factory + CREATE2 prediction
     path already implies this,
749  // so the check is redundant for a legitimately-created
     proxy — it makes the codehash trust
750  // anchor explicit and symmetric across both account-entry
     paths.
751   bytes32 accountCodehash;
752   assembly { accountCodehash := extcodehash(account) }
753   if (!governance.trustedSafeProxyCodehash(accountCodehash))
     revert UntrustedProxyCodehash(accountCodehash);
754
755  _registerAccount(account, permissionSigner, manager,
     feePolicy, feeAsset);
756  }
757
758  /// @notice Register an existing Safe that has already added
     this kernel as a module.
759  /// @dev    MUST be called by the Safe itself (msg.sender ==
     Safe). The caller must be a
760  ///         genuine Safe proxy (verified by runtime codehash
     against the trusted allowlist,
761  ///         blocking arbitrary-contract self-registration —
     finding #4a) and must already
762  ///         have this kernel enabled as a module.
763  ///
764  ///         #4 OWNER AUTHORISATION (the robust gate):
     registration is bound to an EIP-712
765  ///         signature over the principals, verified through
     the Safe's own owner-set+threshold
766  ///         check (`checkSignatures`). This defeats the
     Safe.setup-delegatecall attack that a
767  ///         view-only heuristic could not: during setup every
     Safe-storage signal (nonce, module
768  ///         state, slot-0 singleton) is attacker-forgeable,
     but a setup helper cannot produce the
769  ///         owners' signatures over this digest. If an attacker
     rewrites the owner set to sign with
770  ///         their own key, they own the Safe — there is no
     victim. (The same setup-controlling
```

```
771  ///         attacker could also enable a draining module of
     their own, so this boundary is the most
772  ///         a registration check can defend; see the PR notes.)
     The `nonce()==0` check below is
773  ///         kept purely as cheap defense-in-depth on top of
     the signature.
774  ///
775  ///         createAccount does NOT carry an owner signature:
     it routes through the internal
776  ///         `_registerAccount` (never this public function),
     is kernel-orchestrated, validates the
777  ///         singleton/factory/setup-target against governance
     allowlists, and binds the principals
778  ///         into the CREATE2 salt — so it is protected by
     construction and unaffected by this gate.
779  ///
780  ///         INVOCATION: owners sign the RegisterAccount digest
     off-chain; an owner-approved Safe
781  ///         `execTransaction` then calls this function (so
     msg.sender == the Safe, satisfying the
782  ///         codehash gate) carrying that signature. msg.sender
     == account is preserved.
783  /// @dev    `checkSignatures` is called with empty `data`,
     which suits EOA and approved-hash owners.
784  ///         A Safe owner that is itself a contract relying on
     the legacy v==0 contract-signature path
785  ///         would require non-empty `data` (keccak256(data)==di-
     gest) — a nested-Safe-owner edge case.
786  /// @param  permissionSigner  Address that will sign
     permission-registry operations.
787  /// @param  manager           Address that will sign dispatch
     calls.
788  /// @param  feePolicy         Fee policy contract; address(0)
     = no fee policy.
789  /// @param  feeAsset          Canonical fee settlement token;
     address(0) = native ETH.
790  /// @param  deadline          Unix timestamp after which the
     owner signature is invalid.
791  /// @param  ownerSig          Safe owner-set+threshold
     signature(s) over the RegisterAccount digest.
792   function registerAccount(
793   address permissionSigner,
794   address manager,
795   address feePolicy,
796   address feeAsset,
797   uint256 deadline,
798   bytes calldata ownerSig
```

```
799  ) external {
800  // No exact-length W1 guard here (cf. setManager/collect-
     Fees): `ownerSig` is dynamic, so an
801  // exact length is undefined and a minimum-length check
     would not catch a +20-byte fallback
802  // relay. The owner-signature requirement closes the W1
     fallback vector for this function
803  // directly — a relay cannot produce the owners' signature
     over the digest.
804
805  // 1. Codehash (cheap/static; uses extcodehash, not an
     ISafe method call). Pins genuine
806  //    SafeProxy bytecode before any call into the proxy.
807   bytes32 codehash;
808   assembly { codehash := extcodehash(caller()) }
809   if (!governance.trustedSafeProxyCodehash(codehash)) revert
     UntrustedProxyCodehash(codehash);
810
811  // 2. #9 trusted singleton — checked BEFORE any other
     ISafe method runs. The codehash above
812  //    only pins proxy bytecode; a genuine proxy can still
     delegate to a hostile singleton that
813  //    forges module execution/return data. masterCopy()
     is the one ISafe call that may precede
814  //    this check: the proxy answers 0xa619486e from storage
     slot 0 in its own fallback, so a
815  //    malicious singleton cannot forge it. Confirming the
     singleton here means nonce(),
816  //    isModuleEnabled(), and checkSignatures() below are
     never invoked on an unverified singleton.
817   address singleton = ISafe(msg.sender).masterCopy();
818   if (!governance.trustedSafeSingleton(singleton)) revert
     UntrustedSingleton(singleton);
819
820  // 3. #4 defense-in-depth: setup() never bumps the Safe
     nonce; execTransaction increments it
821  //    before the inner call runs. A value of 0 therefore
     flags a not-yet-finalized Safe. This is
822  //    forgeable by a setup-delegatecall helper (nonce
     lives in slot 5), so it is NOT the gate —
823  //    the owner signature below is. Kept as a cheap early
     reject.
824   if (ISafe(msg.sender).nonce() == 0) revert SetupNotFinal-
     ized();
825
826  // 4. Module must be enabled.
827
```

```
     if (!ISafe(msg.sender).isModuleEnabled(address(this)))
    revert ModuleNotEnabled();
828
829  // 5. #4 owner authorisation (the robust gate). Bind the
    principals to an owner-set+threshold
830  //    signature so a setup helper — which holds no owner
    keys — cannot register. chainId is in
831  //    the EIP-712 domain; no nonce is needed because
    registration is one-shot (`registered[]`
832  //    never clears, so a replay reverts AccountAlreadyReg-
    istered in _registerAccount).
833   if (block.timestamp > deadline) revert DeadlineExpired(dead-
    line, block.timestamp);
834   bytes32 digest = _hashTypedDataV4(keccak256(abi.encode(
835   REGISTER_ACCOUNT_TYPEHASH,
836   msg.sender,
837   permissionSigner,
838   manager,
839   feePolicy,
840   feeAsset,
841   deadline
842  )));
843  // Reverts (GS0xx) unless `ownerSig` satisfies the Safe's
    owner set + threshold.
844   ISafe(msg.sender).checkSignatures(digest, "", ownerSig);
845
846  _registerAccount(msg.sender, permissionSigner, manager,
    feePolicy, feeAsset);
847  }
848
849  /// @dev Shared registration logic for `createAccount` and
    `registerAccount`.
850   function _registerAccount(address account, address permission-
    Signer, address manager, address feePolicy, address feeAsset)
851   internal
852  {
853   if (registered[account]) revert AccountAlreadyRegis-
    tered(account);
854   if (permissionSigner == address(0) || manager == address(0))
    revert ZeroAddress();
855   if (feePolicy != address(0) && !governance.trustedFeePol-
    icy(feePolicy)) revert UntrustedFeePolicy(feePolicy);
856   registered[account] = true;
857   configs[account] = AccountConfig({
858   permissionSigner: permissionSigner,
859   manager:          manager,
860   feePolicy:        feePolicy,
```

```
861   feeAsset:         feeAsset,
862   sessionActive:    true
863  });
864   emit AccountRegistered(account, permissionSigner, manager);
865  }
866
867  /// @notice Rotate the manager (delegated signer) for an
     account, clearing every
868  ///         attached mandate in the same transaction.
869  /// @dev    AUTHORIZATION: MUST be called by the Safe itself
     (`msg.sender == account`).
870  ///         Authorization therefore flows through the Safe's
     own owner threshold — the
871  ///         custody anchor — and replay protection comes from
     the Safe's nonce, so no
872  ///         kernel signature or nonce is needed. This mirrors
     `registerAccount`'s
873  ///         `msg.sender == Safe` trust model. `_requireRegistered`
     ensures only an
874  ///         already-registered account (a genuine Safe proxy,
     vetted at registration)
875  ///         can reach this path, and an account can only rotate
     its own manager.
876  ///
877  ///         MANDATE RESET: A rotated signer must never silently
     inherit authority the
878  ///         owner approved for the old one. Rather than leave
     mandates attached but
879  ///         inert, this clears the permission set outright
     (fail-closed: subsequent
880  ///         dispatches revert with `PermissionNotRegistered`
     until the owner re-approves
881  ///         each mandate via the normal `registerPermission(s)`
     flow — which binds them
882  ///         to the new manager). A `PermissionRevoked` is
     emitted per cleared mandate.
883  ///
884  ///         IN-FLIGHT OPS: `managerNonces`, `batchNonces`,
     and `signerNonces` are all
885  ///         bumped by `NONCE_EPOCH_INCREMENT` so any dispatch
     the old manager pre-signed,
886  ///         and any permission-signer op pre-signed against
     the old epoch, is invalidated —
887  ///         consistent with every other mandate-mutating op.
888  ///
889  ///         PAUSE: Intentionally exempt from `whenNotPaused`
     — losing the agent key is
```

```
890  ///         exactly the kind of incident during which recovery
     must remain possible, and
891  ///         rotation moves no funds. (See `revokeSession`/`re-
     vokePermission`.)
892  /// @param  newManager New address authorised to sign
     dispatches. Must be non-zero and
893  ///                    different from the current manager.
894   function setManager(address newManager) external nonReentrant
     {
895  // W1: reject Safe-fallback-relayed calls. A direct call
     is exactly selector + 1 static arg;
896  // a fallback relay appends the caller's 20 bytes (see
     SET_MANAGER_CALLDATA_LEN).
897   if (msg.data.length != SET_MANAGER_CALLDATA_LEN) revert
     UnexpectedCalldataLength();
898   address account = msg.sender;
899  _requireRegistered(account);
900   if (newManager == address(0)) revert ZeroAddress();
901   address oldManager = configs[account].manager;
902   if (newManager == oldManager) revert ManagerUnchanged();
903
904   configs[account].manager = newManager;
905  _clearPermissions(account);
906   managerNonces[account] += NONCE_EPOCH_INCREMENT;
907   batchNonces[account]   += NONCE_EPOCH_INCREMENT;
908   signerNonces[account]  += NONCE_EPOCH_INCREMENT;
909
910   emit ManagerChanged(account, oldManager, newManager);
911  }
912
913  /// @notice The address currently authorised to sign dispatches
     for an account.
914  /// @param  account The Safe account to query.
915  /// @return         The account's manager (delegated signer);
     address(0) if unregistered.
916   function getManager(address account) external view returns
     (address) {
917   return configs[account].manager;
918  }
919
920  //
     ------------------------------------------------------------------------
921  // 2. Permission registry
922  //
     ------------------------------------------------------------------------
923
924  /// @notice Register a single permission for an account.
```

```
925  ///         Requires a permissionSigner EIP-712 signature and
     an ETH registration fee.
926  /// @dev    PERMISSION TRUST: The kernel binds authorization
     to a permission address only.
927  ///         If a registered permission is upgradeable or uses
     a proxy, a change to its
928  ///         implementation requires no new kernel signature.
     Operators are responsible for
929  ///         registering only non-upgradeable or audited
     permission contracts.
930  ///         See Sail Protocol whitepaper §8.2.
931  /// @param  account    The registered Safe account.
932  /// @param  permission Address of the permission contract to
     register.
933  /// @param  deadline   Unix timestamp after which the signature
     is invalid.
934  /// @param  sig        EIP-712 signature over RegisterPermission
     struct by permissionSigner.
935   function registerPermission(address account, address permis-
     sion, uint256 deadline, bytes calldata sig)
936   external
937   payable
938   nonReentrant
939   whenNotPaused
940  {
941  _requireRegistered(account);
942   if (block.timestamp > deadline) revert DeadlineExpired(dead-
     line, block.timestamp);
943   if (permission == address(0)) revert ZeroAddress();
944   if (permission.code.length == 0) revert NotAContract(per-
     mission);
945   if (_permissionIndex[account][permission] != 0) revert
     PermissionAlreadyRegistered(permission);
946   uint256 limit = governance.maxPermissionsPerAccount();
947   if (_permissions[account].length >= limit)
948   revert TooManyPermissions(account, limit);
949
950   uint256 nonce = signerNonces[account];
951  _verifySignerSig(
952   account,
953   keccak256(abi.encode(REGISTER_PERMISSION_TYPEHASH,
     account, permission, nonce, deadline)),
954   sig
955  );
956   signerNonces[account] = nonce + 1;
957
958   uint256 fee = _calcPermissionFee();
```

```
959   if (msg.value < fee) revert InsufficientFee(fee, msg.value);
960
961   _permissions[account].push(permission);
962   _permissionIndex[account][permission] = _permissions[ac-
      count].length; // stored as index + 1
963
964   _collectRegistrationFee(fee);
965   emit PermissionRegistered(account, permission);
966   }
967
968   /// @notice Revoke a single permission from an account.
969   ///         Requires a permissionSigner EIP-712 signature.
970   /// @dev    Intentionally exempt from whenNotPaused — users
      must be able to revoke
971   ///         permissions even during a protocol pause to reduce
      their exposure.
972   /// @param  account    The registered Safe account.
973   /// @param  permission Address of the permission contract to
      revoke.
974   /// @param  deadline   Unix timestamp after which the signature
      is invalid.
975   /// @param  sig        EIP-712 signature over RevokePermission
      struct by permissionSigner.
976   function revokePermission(address account, address permission,
      uint256 deadline, bytes calldata sig) external nonReentrant {
977   _requireRegistered(account);
978   if (block.timestamp > deadline) revert DeadlineExpired(dead-
      line, block.timestamp);
979   uint256 nonce = signerNonces[account];
980   _verifySignerSig(
981   account,
982   keccak256(abi.encode(REVOKE_PERMISSION_TYPEHASH,
      account, permission, nonce, deadline)),
983   sig
984   );
985   signerNonces[account] = nonce + 1;
986   _removePermission(account, permission);
987   registrationEpoch[account][permission] += 1;
988   managerNonces[account] += NONCE_EPOCH_INCREMENT;
989   batchNonces[account]   += NONCE_EPOCH_INCREMENT;
990   emit PermissionRevoked(account, permission);
991   }
992
993   /// @notice Atomically replace one permission with another in
      a single signed operation.
994   ///         Requires a permissionSigner EIP-712 signature and
      an ETH registration fee.
```

```
995  /// @dev    PERMISSION TRUST: The kernel binds authorization
     to a permission address only.
996  ///         If a registered permission is upgradeable or uses
     a proxy, a change to its
997  ///         implementation requires no new kernel signature.
     Operators are responsible for
998  ///         registering only non-upgradeable or audited
     permission contracts.
999  ///         See Sail Protocol whitepaper §8.2.
1000 /// @param  account        The registered Safe account.
1001 /// @param  oldPermission  Permission to remove.
1002 /// @param  newPermission  Permission to add in its place.
1003 /// @param  deadline       Unix timestamp after which the
     signature is invalid.
1004 /// @param  sig          EIP-712 signature over ReplacePermission
     struct by permissionSigner.
1005  function replacePermission(
1006  address account,
1007  address oldPermission,
1008  address newPermission,
1009  uint256 deadline,
1010  bytes calldata sig
1011 ) external payable nonReentrant whenNotPaused {
1012 _requireRegistered(account);
1013  if (block.timestamp > deadline) revert DeadlineExpired(dead-
     line, block.timestamp);
1014  if (newPermission == address(0)) revert ZeroAddress();
1015  if (newPermission.code.length == 0) revert NotAContract(new-
     Permission);
1016  if (_permissionIndex[account][newPermission] != 0) revert
     PermissionAlreadyRegistered(newPermission);
1017
1018  uint256 nonce = signerNonces[account];
1019 _verifySignerSig(
1020  account,
1021  keccak256(abi.encode(REPLACE_PERMISSION_TYPEHASH,
     account, oldPermission, newPermission, nonce, deadline)),
1022  sig
1023 );
1024  signerNonces[account] = nonce + 1;
1025
1026  uint256 idx = _permissionIndex[account][oldPermission];
1027  if (idx == 0) revert PermissionNotRegistered(oldPermission);
1028
1029  uint256 fee = _calcPermissionFee();
1030  if (msg.value < fee) revert InsufficientFee(fee, msg.value);
1031
```

```
1032 _permissions[account][idx - 1] = newPermission;
1033  delete _permissionIndex[account][oldPermission];
1034 _permissionIndex[account][newPermission] = idx;
1035 // Bump the REMOVED side only. newPermission is register-like
     (its config is applied just
1036 // before this call in the bundled flow); bumping it would
     strand that fresh config.
1037  registrationEpoch[account][oldPermission] += 1;
1038
1039 _collectRegistrationFee(fee);
1040  managerNonces[account] += NONCE_EPOCH_INCREMENT;
1041  batchNonces[account]   += NONCE_EPOCH_INCREMENT;
1042  emit PermissionReplaced(account, oldPermission, newPer-
     mission);
1043 }
1044
1045 /// @notice Atomically replace N permissions with N new ones
     in a single signed operation.
1046 ///         Eliminates the front-running overlap window that
     arises when separate
1047 ///         `registerPermissions` + `revokePermissions` calls
     are used for NN migrations.
1048 ///         Requires a permissionSigner EIP-712 signature and
     a fee per swap.
1049 /// @dev    PERMISSION TRUST: The kernel binds authorization
     to a permission address only.
1050 ///         If a registered permission is upgradeable or uses
     a proxy, a change to its
1051 ///         implementation requires no new kernel signature.
     Operators are responsible for
1052 ///         registering only non-upgradeable or audited
     permission contracts.
1053 ///         See Sail Protocol whitepaper §8.2.
1054 /// @param  account         The registered Safe account.
1055 /// @param  oldPermissions  Permissions to remove (parallel
     to `newPermissions`).
1056 /// @param  newPermissions  Permissions to add in their place.
1057 /// @param  deadline        Unix timestamp after which the
     signature is invalid.
1058 /// @param  sig             EIP-712 signature over
     ReplacePermissions struct by permissionSigner.
1059  function replacePermissions(
1060  address account,
1061  address[] calldata oldPermissions,
1062  address[] calldata newPermissions,
1063  uint256 deadline,
1064  bytes calldata sig
```

```
1065 ) external payable nonReentrant whenNotPaused {
1066  if (oldPermissions.length != newPermissions.length) revert
     ArrayLengthMismatch();
1067  if (oldPermissions.length == 0) {
1068  if (msg.value > 0) _collectRegistrationFee(0); //
     refunds full msg.value to caller
1069  return;
1070 }
1071 _requireRegistered(account);
1072  if (block.timestamp > deadline) revert DeadlineExpired(dead-
     line, block.timestamp);
1073
1074  uint256 nonce = signerNonces[account];
1075 _verifySignerSig(
1076  account,
1077  keccak256(abi.encode(
1078  REPLACE_PERMISSIONS_TYPEHASH,
1079  account,
1080 _hashAddressArray(oldPermissions),
1081 _hashAddressArray(newPermissions),
1082  nonce,
1083  deadline
1084 )),
1085  sig
1086 );
1087  signerNonces[account] = nonce + 1;
1088
1089  uint256 fee = _calcPermissionFee() * oldPermissions.length;
1090  if (msg.value < fee) revert InsufficientFee(fee, msg.value);
1091
1092  for (uint256 i; i < oldPermissions.length; i++) {
1093  address oldPerm = oldPermissions[i];
1094  address newPerm = newPermissions[i];
1095  if (newPerm == address(0)) revert ZeroAddress();
1096  if (newPerm.code.length == 0) revert NotAContract(new-
     Perm);
1097  uint256 idx = _permissionIndex[account][oldPerm];
1098  if (idx == 0) revert PermissionNotRegistered(oldPerm);
1099  if (_permissionIndex[account][newPerm] != 0) revert
     PermissionAlreadyRegistered(newPerm);
1100 _permissions[account][idx - 1] = newPerm;
1101  delete _permissionIndex[account][oldPerm];
1102 _permissionIndex[account][newPerm] = idx;
1103  registrationEpoch[account][oldPerm] += 1; // bump the
     removed side only (see replacePermission)
1104  emit PermissionReplaced(account, oldPerm, newPerm);
1105 }
```

```
1106
1107 _collectRegistrationFee(fee);
1108  managerNonces[account] += NONCE_EPOCH_INCREMENT;
1109  batchNonces[account]   += NONCE_EPOCH_INCREMENT;
1110 }
1111
1112 /// @notice Suspend the manager session for an account. All
     `dispatch` calls will
1113 ///         revert with `SessionInactive` until `activateSession`
     is called.
1114 /// @param  account  The registered Safe account.
1115 /// @param  deadline Unix timestamp after which the signature
     is invalid.
1116 /// @param  sig      EIP-712 signature over RevokeSession
     struct by permissionSigner.
1117  function revokeSession(address account, uint256 deadline,
     bytes calldata sig) external nonReentrant {
1118 _requireRegistered(account);
1119  if (block.timestamp > deadline) revert DeadlineExpired(dead-
     line, block.timestamp);
1120  uint256 nonce = signerNonces[account];
1121 _verifySignerSig(
1122  account,
1123  keccak256(abi.encode(REVOKE_SESSION_TYPEHASH, account,
     nonce, deadline)),
1124  sig
1125 );
1126  signerNonces[account] = nonce + 1;
1127  configs[account].sessionActive = false;
1128  managerNonces[account] += NONCE_EPOCH_INCREMENT;
1129  batchNonces[account]   += NONCE_EPOCH_INCREMENT;
1130  emit SessionRevoked(account);
1131 }
1132
1133 /// @notice Re-activate a previously suspended session.
     Requires a fresh permissionSigner
1134 ///         signature to prove the key is still under the
     operator's control.
1135 /// @param  account  The registered Safe account.
1136 /// @param  deadline Unix timestamp after which the signature
     is invalid.
1137 /// @param  sig      EIP-712 signature over ActivateSession
     struct by permissionSigner.
1138  function activateSession(address account, uint256 deadline,
     bytes calldata sig) external nonReentrant {
1139 _requireRegistered(account);
1140
```

```
     if (block.timestamp > deadline) revert DeadlineExpired(dead-
    line, block.timestamp);
1141  uint256 nonce = signerNonces[account];
1142 _verifySignerSig(
1143  account,
1144  keccak256(abi.encode(ACTIVATE_SESSION_TYPEHASH, account,
    nonce, deadline)),
1145  sig
1146 );
1147  signerNonces[account] = nonce + 1;
1148  configs[account].sessionActive = true;
1149 // Rotate manager/batch nonce epochs on reactivation so
    any dispatch the manager
1150 // pre-signed while the session was suspended (current
    epoch) cannot execute once
1151 // the session is live again. Mirrors revokeSession's
    epoch bump — a session cycle
1152 // is a clean kill switch for outstanding signatures in
    both directions.
1153  managerNonces[account] += NONCE_EPOCH_INCREMENT;
1154  batchNonces[account]   += NONCE_EPOCH_INCREMENT;
1155  emit SessionActivated(account);
1156 }
1157
1158 /// @notice Replace the fee policy for an account. Requires a
    permissionSigner signature.
1159 ///         Setting `newFeePolicy = address(0)` clears the
    policy and blocks fee collection.
1160 /// @param  account      The registered Safe account.
1161 /// @param  newFeePolicy New fee policy contract address;
    address(0) = no fee policy.
1162 /// @param  feeAsset     Canonical fee settlement token for
    the new policy; address(0) = native ETH.
1163 /// @param  deadline     Unix timestamp after which the
    signature is invalid.
1164 /// @param  sig          EIP-712 signature over SetFeePolicy
    struct by permissionSigner.
1165  function setFeePolicy(address account, address newFeePolicy,
    address feeAsset, uint256 deadline, bytes calldata sig) external
    nonReentrant {
1166 _requireRegistered(account);
1167 // Allow clearing (newFeePolicy == 0) during pause so
    permissionSigner can disarm a compromised policy
1168 // before the pause lifts and collectFees becomes callable
    again.  Block non-zero updates while paused.
1169  if (governance.isPaused() && newFeePolicy != address(0))
    revert ProtocolPaused();
```

```
1170  if (newFeePolicy != address(0) && !governance.trustedFeeP-
     olicy(newFeePolicy)) revert UntrustedFeePolicy(newFeePolicy);
1171  if (block.timestamp > deadline) revert DeadlineExpired(dead-
     line, block.timestamp);
1172  uint256 nonce = signerNonces[account];
1173 _verifySignerSig(
1174  account,
1175  keccak256(abi.encode(SET_FEE_POLICY_TYPEHASH, account,
     newFeePolicy, feeAsset, nonce, deadline)),
1176  sig
1177 );
1178  signerNonces[account] = nonce + 1;
1179
1180 // Pin a policy instance to the single fee asset it is
     used with for this account, so the
1181 // policy's persisted (per-account) high-water mark can
     never be mixed across denominations.
1182 // First lazily bind the currently configured policy (e.g.
     one set at registration) to its
1183 // current asset, so a later swap to a different asset on
     the SAME instance is caught even
1184 // on the first setFeePolicy call.
1185  address currentPolicy = configs[account].feePolicy;
1186  if (currentPolicy != address(0)) {
1187  PolicyAssetBinding storage current = _policyAssetBind-
     ing[account][currentPolicy];
1188  if (!current.bound) { current.bound = true; current.asset
     = configs[account].feeAsset; }
1189 }
1190 // Then enforce the binding for the incoming policy: bind
     on first use, else require a match.
1191  if (newFeePolicy != address(0)) {
1192  PolicyAssetBinding storage binding = _policyAssetBind-
     ing[account][newFeePolicy];
1193  if (!binding.bound) { binding.bound = true; binding.asset
     = feeAsset; }
1194  else if (binding.asset != feeAsset) revert
     FeePolicyAssetMismatch(newFeePolicy, binding.asset, feeAsset);
1195 }
1196
1197  configs[account].feePolicy = newFeePolicy;
1198  configs[account].feeAsset  = (newFeePolicy == address(0))
     ? address(0) : feeAsset;
1199  emit FeePolicyUpdated(account, newFeePolicy);
1200
1201 // Lifecycle hook (after all kernel state writes — CEI):
     tell the new policy it has been
```

```
1202 // attached so it can re-anchor its per-account accounting. This closes the detach→reattach
1203 // over-collection (the same instance would otherwise bill management fees over the dormant
1204 // interval). The call passes ONLY `account` — the kernel never learns NAV/valuation; that
1205 // stays the policy/manager's responsibility. The new policy is governance-trusted (checked
1206 // above) and setFeePolicy is nonReentrant, so the external call is safe here.
1207  if (newFeePolicy != address(0)) IFeePolicy(newFeePolicy).onAttach(account);
1208 }
1209
1210 /// @notice Register multiple permissions atomically. One signer nonce is consumed for the
1211 ///         entire batch; the total fee equals the sum of individual permission fees.
1212 ///         Reverts atomically if any permission in the batch is already registered,
1213 ///         the cap would be exceeded, the fee is insufficient, or the signature is invalid.
1214 /// @dev    The `permissions` array is EIP-712 encoded as keccak256 of the ABI-packed
1215 ///         zero-padded addresses (see `_hashAddressArray`). Empty arrays are a no-op
1216 ///         and do not consume a nonce.
1217 /// @dev    PERMISSION TRUST: The kernel binds authorization to a permission address only.
1218 ///         If a registered permission is upgradeable or uses a proxy, a change to its
1219 ///         implementation requires no new kernel signature. Operators are responsible for
1220 ///         registering only non-upgradeable or audited permission contracts.
1221 ///         See Sail Protocol whitepaper §8.2.
1222 /// @param  account     The registered Safe account.
1223 /// @param  permissions Addresses of permission contracts to register.
1224 /// @param  deadline    Unix timestamp after which the signature is invalid.
1225 /// @param  sig         EIP-712 signature over RegisterPermissions struct by permissionSigner.
1226  function registerPermissions(
1227  address account,
1228  address[] calldata permissions,
1229  uint256 deadline,
```

```
1230  bytes calldata sig
1231 ) external payable nonReentrant whenNotPaused {
1232  if (permissions.length == 0) {
1233  if (msg.value > 0) _collectRegistrationFee(0); //
     refunds full msg.value to caller
1234  return;
1235 }
1236 _requireRegistered(account);
1237  if (block.timestamp > deadline) revert DeadlineExpired(dead-
     line, block.timestamp);
1238
1239 // Enforce the cap before consuming the nonce to avoid
     nonce burns on revert.
1240  uint256 limit = governance.maxPermissionsPerAccount();
1241  if (_permissions[account].length + permissions.length >
     limit)
1242  revert TooManyPermissions(account, limit);
1243
1244  uint256 nonce = signerNonces[account];
1245 _verifySignerSig(
1246  account,
1247  keccak256(abi.encode(
1248  REGISTER_PERMISSIONS_TYPEHASH,
1249  account,
1250 _hashAddressArray(permissions),
1251  nonce,
1252  deadline
1253 )),
1254  sig
1255 );
1256  signerNonces[account] = nonce + 1;
1257
1258 // Compute total fee before any state changes
1259  uint256 totalFee = _calcPermissionFee() * permis-
     sions.length;
1260  if (msg.value < totalFee) revert InsufficientFee(totalFee,
     msg.value);
1261
1262 // Add all permissions atomically — reverts if any duplicate
     or invalid address found
1263  for (uint256 i; i < permissions.length; i++) {
1264  address perm = permissions[i];
1265  if (perm == address(0)) revert ZeroAddress();
1266  if (perm.code.length == 0) revert NotAContract(perm);
1267  if (_permissionIndex[account][perm] != 0) revert
     PermissionAlreadyRegistered(perm);
1268 _permissions[account].push(perm);
```

```
1269 _permissionIndex[account][perm] = _permissions[ac-
     count].length;
1270  emit PermissionRegistered(account, perm);
1271 }
1272
1273 _collectRegistrationFee(totalFee);
1274 }
1275
1276 /// @notice Revoke multiple permissions atomically. One signer
     nonce is consumed for
1277 ///         the entire batch; no ETH fee is required.
1278 ///         Empty arrays are a no-op and do not consume a
     nonce.
1279 /// @dev    Intentionally exempt from whenNotPaused — users
     must be able to revoke
1280 ///         permissions even during a protocol pause to reduce
     their exposure.
1281 /// @param  account     The registered Safe account.
1282 /// @param  permissions Addresses of permission contracts to
     revoke.
1283 /// @param  deadline    Unix timestamp after which the signature
     is invalid.
1284 /// @param  sig         EIP-712 signature over RevokePermissions
     struct by permissionSigner.
1285  function revokePermissions(
1286  address account,
1287  address[] calldata permissions,
1288  uint256 deadline,
1289  bytes calldata sig
1290 ) external nonReentrant {
1291  if (permissions.length == 0) return;
1292 _requireRegistered(account);
1293  if (block.timestamp > deadline) revert DeadlineExpired(dead-
     line, block.timestamp);
1294
1295  uint256 nonce = signerNonces[account];
1296 _verifySignerSig(
1297  account,
1298  keccak256(abi.encode(
1299  REVOKE_PERMISSIONS_TYPEHASH,
1300  account,
1301 _hashAddressArray(permissions),
1302  nonce,
1303  deadline
1304 )),
1305  sig
1306 );
```

```
1307  signerNonces[account] = nonce + 1;
1308
1309  for (uint256 i; i < permissions.length; i++) {
1310 _removePermission(account, permissions[i]);
1311  registrationEpoch[account][permissions[i]] += 1;
1312  emit PermissionRevoked(account, permissions[i]);
1313 }
1314  managerNonces[account] += NONCE_EPOCH_INCREMENT;
1315  batchNonces[account]   += NONCE_EPOCH_INCREMENT;
1316 }
1317
1318 /// @notice Return the full list of registered permission
     addresses for an account.
1319 /// @param  account The Safe account to query.
1320 /// @return         Array of registered permission contract
     addresses.
1321  function getPermissions(address account) external view returns
     (address[] memory) {
1322  return _permissions[account];
1323 }
1324
1325 /// @notice Check whether a specific permission is registered
     for an account.
1326 /// @param  account    The Safe account to query.
1327 /// @param  permission The permission contract address to look
     up.
1328 /// @return            True if the permission is currently
     registered.
1329  function isPermissionRegistered(address account, address
     permission) external view returns (bool) {
1330  return _permissionIndex[account][permission] != 0;
1331 }
1332
1333 //
     ------------------------------------------------------------------------
1334 // 3. Manager dispatch
1335 //
     ------------------------------------------------------------------------
1336
1337 /// @notice Verify a manager signature, evaluate the named
     permission, and
1338 ///         execute the transaction via the Safe module
     interface.
1339 ///
1340 // SELECTIVE authorization semantics: the manager signature
     names one
1341
```

```
    // registered permission, and only that permission evaluates
    the call.
1342 // Changed from the prior conjunctive (AND) model where all
    registered
1343 // permissions had to approve. The new model enables
    multi-permission SMAs
1344 // where unrelated permissions (e.g., Uniswap, Aave, Transfer)
    coexist
1345 // on one account without falsely denying each other's calls.
    Layered
1346 // defense via permission composition is not supported here —
    a separate
1347 // guard mechanism may be added later if needed.
1348 ///
1349 /// @dev    The manager nonce is incremented before external
    interaction; however, any revert
1350 ///         will roll back this write, so failed attempts do
    not consume the nonce. To achieve
1351 ///         strict single-use semantics, convert post-nonce
    failures to non-reverting denials
1352 ///         or add an explicit cancel-nonce operation. The
    permission is evaluated via
1353 ///         staticcall with PERMISSION_GAS_CAP gas; a revert
    or gas exhaustion inside a
1354 ///         permission is treated as denial. The named permission
    must be pre-registered
1355 ///         on the account; this is the permissionSigner's
    trust anchor.
1356 /// @dev    CONFIG SEQUENCING: Dispatch authorizes by permission
    address only, not by
1357 ///         configuration state. To atomically tighten a
    permission's rules, use
1358 ///         replacePermission rather than reconfiguring in
    place. An in-place configure
1359 ///         is subject to a front-run window while the
    transaction is pending.
1360 /// @param  account     The registered Safe account to execute
    through.
1361 /// @param  permission  The registered permission that must
    authorise this call.
1362 /// @param  target      Call target address.
1363 /// @param  value       Native ETH to forward with the call
    (wei).
1364 /// @param  data        Calldata for the target call.
1365 /// @param  managerSig  EIP-712 signature over Dispatch struct
    by the account's manager.
1366
```

```
    /// @param  deadline    Unix timestamp after which the signature
    is invalid.
1367  function dispatch(
1368  address account,
1369  address permission,
1370  address target,
1371  uint256 value,
1372  bytes calldata data,
1373  bytes calldata managerSig,
1374  uint256 deadline
1375 ) external nonReentrant whenNotPaused {
1376 _requireRegistered(account);
1377
1378  AccountConfig storage cfg = configs[account];
1379  if (!cfg.sessionActive) revert SessionInactive(account);
1380 // Deadline is a user-supplied expiry, intentionally
    compared against block.timestamp.
1381 // Matches the pattern used by every other deadline-checking
    entry point in this contract.
1382 // forge-lint: disable-next-line(block-timestamp)
1383  if (block.timestamp > deadline) revert DeadlineExpired(dead-
    line, block.timestamp);
1384
1385 // O(1) membership check — revert if the named permission
    is not registered.
1386  if (_permissionIndex[account][permission] == 0) revert
    PermissionNotRegistered(permission);
1387
1388  uint256 nonce    = managerNonces[account];
1389  bytes32 dataHash = keccak256(data);
1390  bytes32 digest   = _hashTypedDataV4(keccak256(abi.encode(
1391  DISPATCH_TYPEHASH,
1392  account,
1393  permission,
1394  target,
1395  value,
1396  dataHash,
1397  nonce,
1398  deadline
1399 )));
1400  if (!_recoverOrERC1271(cfg.manager, digest, managerSig))
    revert InvalidManagerSignature();
1401  managerNonces[account] = nonce + 1;
1402
1403 // Prevent module-triggered self-calls: a call targeting
    the Safe itself satisfies
1404
```

```
    // Safe's onlySelf guard, enabling enableModule/set-
    Guard/owner changes without permission.
1405  if (target == account) revert AccountSelfTarget();
1406 // Parity with dispatchBatch's per-subcall guards: a single
    dispatch may not target the
1407 // zero address or the kernel itself. Re-entry is already
    blocked by nonReentrant; rejecting
1408 // these targets closes the class at the dispatch boundary.
    Single dispatch has no subcall
1409 // index, so it reuses the batch errors with index 0.
1410  if (target == address(0))    revert BatchZeroTarget(0);
1411  if (target == address(this)) revert KernelSelfTarget(0);
1412
1413  bytes4 sel = data.length >= 4 ? bytes4(data[:4]) : bytes4(0);
1414  Context memory ctx = Context({
1415  account:        account,
1416  manager:        cfg.manager,
1417  submitter:      msg.sender,
1418  target:         target,
1419  selector:       sel,
1420  value:          value,
1421  blockTimestamp: block.timestamp,
1422  blockNumber:    block.number,
1423  configEpoch:    registrationEpoch[account][permission]
1424 });
1425  if (!_evaluatePermission(permission, data, ctx)) revert
    PermissionDenied(permission);
1426
1427  if (!ISafe(account).execTransactionFromModule(target,
    value, data, 0)) revert SafeExecutionFailed();
1428
1429  emit Dispatched(account, permission, target, sel, value);
1430 }
1431
1432 /// @dev Invoke a single permission via staticcall with the
    configured gas cap.
1433 ///      Returns false on revert, out-of-gas, or malformed
    return data.
1434  function _evaluatePermission(address permission, bytes
    calldata data, Context memory ctx)
1435  internal
1436  view
1437  returns (bool)
1438 {
1439  bytes memory callData = abi.encodeCall(IPermission.evaluate,
    (data, ctx));
1440
```

```
    (bool success, bytes memory ret) = permission.static-
    call{gas: PERMISSION_GAS_CAP}(callData);
1441  if (!success || ret.length < 32) return false;
1442 // Decode as uint256 to avoid abi.decode revert on
    non-canonical bool words (e.g. 0x02).
1443  return abi.decode(ret, (uint256)) == 1;
1444 }
1445
1446 //
    ------------------------------------------------------------------------
1447 // 3b. Batch dispatch
1448 //
    ------------------------------------------------------------------------
1449
1450 /// @notice Execute a sequence of Safe module calls as a single
    atomic transaction,
1451 ///         gated by ONE named batch-aware permission.
1452 ///
1453 /// @dev    DIVERGENCE FROM `dispatch`: only the named
    `permission` is evaluated.
1454 ///         Other IPermissions registered on the account are
    NOT consulted during
1455 ///         batch dispatch. The batch-aware permission owns
    full responsibility for
1456 ///         validating every subcall and any cross-call
    invariants (e.g. matching
1457 ///         approve/consume amounts, mandatory reset-to-zero
    cleanup).
1458 ///
1459 ///         The `permission` MUST still be registered on the
    account — registration
1460 ///         is the permissionSigner's trust anchor. Choosing
    it for a batch then
1461 ///         requires only the manager's signature.
1462 ///
1463 ///         Atomicity: if any subcall returns false from
    execTransactionFromModule,
1464 ///         the entire dispatch reverts, rolling back all
    prior subcalls in the batch.
1465 ///
1466 ///         Operation type: every subcall executes with
    `operation = 0` (CALL).
1467 ///         DELEGATECALL is never used. The kernel does not
    depend on Safe MultiSend.
1468 ///
1469 ///         Self-target guard: no subcall may target this
    kernel. This is a defensive
```

```
1470 ///         measure — re-entry through public functions is already blocked by
1471 ///         nonReentrant, but rejecting kernel-targeted subcalls eliminates the
1472 ///         entire class of self-targeted attacks at the dispatch boundary.
1473 ///
1474 /// @dev    CONFIG SEQUENCING: Dispatch authorizes by permission address only, not by
1475 ///         configuration state. To atomically tighten a permission's rules, use
1476 ///         replacePermission rather than reconfiguring in place. An in-place configure
1477 ///         is subject to a front-run window while the transaction is pending.
1478 ///
1479 /// @param  account     The registered Safe account to execute through.
1480 /// @param  permission  The batch-aware permission that authorises this batch.
1481 ///                     Must be registered on the account and implement IBatchPermission.
1482 /// @param  calls       Ordered subcall sequence (length 1..MAX_BATCH_LENGTH).
1483 /// @param  managerSig  EIP-712 signature over DispatchBatch struct by the account's manager.
1484 /// @param  deadline    Unix timestamp after which the signature is invalid.
1485  function dispatchBatch(
1486  address account,
1487  address permission,
1488  Call[] calldata calls,
1489  bytes calldata managerSig,
1490  uint256 deadline
1491 ) external nonReentrant whenNotPaused {
1492 // 1. Account / session / deadline validation
1493 _requireRegistered(account);
1494  AccountConfig storage cfg = configs[account];
1495  if (!cfg.sessionActive) revert SessionInactive(account);
1496  if (block.timestamp > deadline) revert DeadlineExpired(deadline, block.timestamp);
1497
1498 // 2. Batch length bounds
1499  uint256 len = calls.length;
1500  if (len == 0) revert EmptyBatch();
1501  if (len > MAX_BATCH_LENGTH) revert BatchTooLong(len);
1502
```

```
1503 // 3. Permission must be registered for this account
1504  if (_permissionIndex[account][permission] == 0) revert
     PermissionNotRegistered(permission);
1505
1506 // 4. Verify manager signature over the canonical callsHash
1507  bytes32 callsHash = keccak256(abi.encode(calls));
1508  uint256 nonce     = batchNonces[account];
1509  bytes32 digest    = _hashTypedDataV4(keccak256(abi.encode(
1510  DISPATCH_BATCH_TYPEHASH,
1511  account,
1512  permission,
1513  callsHash,
1514  nonce,
1515  deadline
1516 )));
1517  if (!_recoverOrERC1271(cfg.manager, digest, managerSig))
     revert InvalidManagerSignature();
1518 // NOTE: Nonce is incremented here, but any subsequent
     revert will roll back this write;
1519 // failed attempts do not consume the nonce. To enforce
     single-use semantics, convert
1520 // post-nonce failures to non-reverting denials or add an
     explicit cancel-nonce operation.
1521  batchNonces[account] = nonce + 1;
1522
1523 // 5. Permission must implement IBatchPermission. Detection
     via staticcall
1524 //    is stricter than try/catch — guarantees no state
     mutation regardless of
1525 //    what the target claims about its function modifiers.
1526 {
1527 (bool detOk, bytes memory detRet) = permission.stat-
     iccall{gas: BATCH_DETECT_GAS_CAP}(
1528  abi.encodeWithSelector(IBatchPermission.isBatch-
     Permission.selector)
1529 );
1530  if (!detOk || detRet.length < 32 || abi.decode(detRet,
     (uint256)) != 1) {
1531  revert PermissionNotBatchAware(permission);
1532 }
1533 }
1534
1535 // 6. Pre-flight: no subcall may target the zero address
     or the kernel itself
1536  for (uint256 i = 0; i < len;) {
1537  if (calls[i].target == address(0))    revert
     BatchZeroTarget(i);
```

```
1538  if (calls[i].target == address(this)) revert
      KernelSelfTarget(i);
1539  if (calls[i].target == account)       revert
      AccountSelfTarget();
1540  unchecked { ++i; }
1541 }
1542
1543 // 7. Build BatchContext and evaluate the batch permission
1544  BatchContext memory ctx = BatchContext({
1545  account:        account,
1546  manager:        cfg.manager,
1547  submitter:      msg.sender,
1548  permission:     permission,
1549  batchHash:      callsHash,
1550  blockTimestamp: block.timestamp,
1551  blockNumber:    block.number,
1552  configEpoch:    registrationEpoch[account][permission]
1553 });
1554  if (!_evaluateBatchPermission(permission, calls, ctx))
      revert BatchPermissionDenied();
1555
1556 // 8. Execute each subcall in order via Safe module CALL
      (operation = 0).
1557 //    Any failure reverts the entire transaction, rolling
      back earlier subcalls.
1558  for (uint256 i = 0; i < len;) {
1559  Call calldata c = calls[i];
1560  bool ok = ISafe(account).execTransactionFromMod-
      ule(c.target, c.value, c.data, 0);
1561  if (!ok) revert BatchSubcallFailed(i, c.target);
1562  unchecked { ++i; }
1563 }
1564
1565  emit BatchDispatched(account, permission, callsHash, len);
1566 }
1567
1568 /// @dev Invoke `evaluateBatch` on the named permission via
      staticcall with the
1569 ///      batch gas cap. Returns false on revert, OOG, or
      malformed return data.
1570 ///      Pattern mirrors `_evaluatePermission` for consistency.
1571  function _evaluateBatchPermission(
1572  address permission,
1573  Call[] calldata calls,
1574  BatchContext memory ctx
1575 ) internal view returns (bool) {
1576
```

```
     bytes memory callData = abi.encodeCall(IBatchPermis-
    sion.evaluateBatch, (calls, ctx));
1577 (bool success, bytes memory ret) = permission.static-
    call{gas: BATCH_EVAL_GAS_CAP}(callData);
1578  if (!success || ret.length < 32) return false;
1579  return abi.decode(ret, (uint256)) == 1;
1580 }
1581
1582 //
    ------------------------------------------------------------------------
1583 // 3c. Permission introspection views
1584 //
    ------------------------------------------------------------------------
1585
1586 /// @notice Return the full permission list for an account
    with enriched metadata.
1587 /// @dev    Reads IPermissionIntrospection and IBatchPermis-
    sion.isBatchPermission()
1588 ///         on each registered permission via try/catch. A
    non-implementing permission
1589 ///         returns zero/false for the corresponding fields.
1590 ///
1591 ///         `hasIntrospection` is true when `permissionId()`
    succeeds AND returns
1592 ///         a non-zero value (a zero permissionId indicates a
    non-compliant or unset
1593 ///         implementation; see IPermissionIntrospection.per-
    missionId()).
1594 ///
1595 ///         Intended for off-chain use only. Each permission
    may make up to 3 external
1596 ///         view calls; call with a generous gas limit on
    accounts with many permissions.
1597 ///         MUST NOT be called from dispatch — use
    isPermissionRegistered() for O(1) checks.
1598 ///
1599 /// @dev    External calls to user-deployed permission contracts
    are NOT gas-capped here.
1600 ///         Off-chain consumers (indexers, dashboards) should
    apply their own gas limit or
1601 ///         timeout when calling this function over RPC; a
    buggy or malicious permission can
1602 ///         consume large amounts of gas or return large data.
    The kernel omits a cap because
1603 ///         this is a view with no on-chain impact; the
    dispatch-path already gas-caps each
1604
```

```
///         permission evaluation via the protocol's per-permission gas isolation guarantee.
/// @param  account The Safe account to query.
/// @return infos   Array of PermissionInfo, one per registered permission, in registration order.
 function getPermissionsWithInfo(address account) external view returns (PermissionInfo[] memory infos) {
 address[] storage perms = _permissions[account];
 uint256 len = perms.length;
 infos = new PermissionInfo[](len);
 for (uint256 i; i < len; i++) {
 address perm = perms[i];
 infos[i].permission = perm;
 try IBatchPermission(perm).isBatchPermission() returns (bool b) {
 infos[i].isBatch = b;
} catch {}
 try IPermissionIntrospection(perm).permissionId() returns (bytes32 pid) {
 if (pid != bytes32(0)) {
 infos[i].hasIntrospection = true;
 infos[i].permissionId = pid;
 try IPermissionIntrospection(perm).permission-Version() returns (bytes32 pv) {
 infos[i].permissionVersion = pv;
} catch {}
}
} catch {}
}
}

/// @notice Simulate whether a batch dispatch would be approved without executing it.
/// @dev    Runs the same validation as dispatchBatch EXCEPT signature verification
///         (no sig available in a simulation context) and Safe module execution.
///         Useful for off-chain pre-flight checks (keeper bots, UI previews).
///
///         Return values:
///           approved = true  'batch would pass evaluateBatch
///           approved = false 'batch would be denied; `reason` has a short descriptor
///
///         Does NOT check: session active, deadline, or sig.
///         Callers must verify those separately.
```

```
1640 ///         DOES check: account registration and permission
     registration; returns
1641 ///         (false, "AccountNotRegistered") or (false,
     "PermissionNotRegistered") accordingly.
1642 ///
1643 /// @param  account    The registered Safe account.
1644 /// @param  permission The batch-aware permission to evaluate.
1645 /// @param  calls      The call sequence to evaluate.
1646 /// @return approved   True if the batch permission would
     authorise the call sequence.
1647 /// @return reason     Short denial reason string; empty if
     approved.
1648  function previewBatch(
1649  address account,
1650  address permission,
1651  Call[] calldata calls
1652 ) external view returns (bool approved, string memory reason)
     {
1653  if (calls.length == 0)              return (false,
     "EmptyBatch");
1654  if (calls.length > MAX_BATCH_LENGTH) return (false,
     "BatchTooLong");
1655  if (!registered[account])           return (false,
     "AccountNotRegistered");
1656  if (_permissionIndex[account][permission] == 0) return
     (false, "PermissionNotRegistered");
1657
1658  for (uint256 i; i < calls.length; i++) {
1659  if (calls[i].target == address(0))    return (false,
     "BatchZeroTarget");
1660  if (calls[i].target == address(this)) return (false,
     "KernelSelfTarget");
1661  if (calls[i].target == account)       return (false,
     "AccountSelfTarget");
1662 }
1663
1664 (bool detOk, bytes memory detRet) = permission.static-
     call{gas: BATCH_DETECT_GAS_CAP}(
1665  abi.encodeWithSelector(IBatchPermission.isBatchPer-
     mission.selector)
1666 );
1667  if (!detOk || detRet.length < 32 || abi.decode(detRet,
     (uint256)) != 1) {
1668  return (false, "PermissionNotBatchAware");
1669 }
1670
1671  bytes32 callsHash = keccak256(abi.encode(calls));
```

```
1672  BatchContext memory ctx = BatchContext({
1673  account:        account,
1674  manager:        configs[account].manager,
1675  submitter:      address(0),   // no submitter
     in simulation; permissions enforcing submitter whitelists will
     return false
1676  permission:     permission,
1677  batchHash:      callsHash,
1678  blockTimestamp: block.timestamp,
1679  blockNumber:    block.number,
1680  configEpoch:    registrationEpoch[account][permission]
1681 });
1682
1683  if (!_evaluateBatchPermission(permission, calls, ctx)) {
1684  return (false, "BatchPermissionDenied");
1685 }
1686  return (true, "");
1687 }
1688
1689 //
     ------------------------------------------------------------------------
1690 // 4. Fee accounting
1691 //
     ------------------------------------------------------------------------
1692
1693 /// @notice Collect fees earned by the manager. The kernel
     validates the requested amount
1694 ///         against the registered fee policy and enforces
     the protocol/distributor split.
1695 /// @dev    TRUST ASSUMPTION: `currentNav` is provided by the
     manager and is not verified
1696 ///         on-chain. The fee policy is the sole guard against
     inflated NAV inputs.
1697 ///
1698 /// @dev    DENOMINATION WARNING: `currentNav` and `feeToken`
     must use consistent units.
1699 ///         The fee policy computes maxFee from currentNav —
     if NAV is USD-denominated
1700 ///         but feeToken is WETH, the fee ceiling will be
     wildly incorrect.
1701 ///         Deployers must ensure their fee policy validates
     or denominates in feeToken units.
1702 ///         The actual tokens transferred equal `grossFee` —
     not a function of `currentNav` —
1703 ///         but a dishonest manager could inflate `currentNav`
     to unlock a larger `maxFee`
1704
```

```
///         ceiling and then pass a correspondingly large `grossFee`. Deployers must use a
1705 ///         fee policy that validates NAV independently if the manager is not trusted.
1706 ///
1707 /// @dev Fee collection is atomic: any failed transfer reverts the entire call.
1708 ///      ETH mode (feeToken == address(0)) requires all recipients to be payable.
1709 ///      To avoid DoS on non-payable recipients, prefer ERC-20 fee tokens.
1710 /// @param  account    The registered Safe account from which fees are collected.
1711 /// @param  grossFee   Requested fee amount. Must not exceed the policy's computed maximum.
1712 /// @param  currentNav Current net asset value reported by the manager.
1713 /// @param  feeToken   ERC-20 token for fee payment; address(0) = native ETH.
1714 /// @dev DENOMINATION WARNING: `grossFee` and `currentNav` must be expressed in the same
1715 ///      token units as the fee token (or ETH wei if feeToken==address(0)). Mixing
1716 ///      denominations between grossFee and currentNav will silently produce incorrect fee
1717 ///      calculations inside the policy.
1718  function collectFees(
1719  address account,
1720  uint256 grossFee,
1721  uint256 currentNav,
1722  address feeToken
1723 ) external nonReentrant whenNotPaused {
1724 // W1: reject Safe-fallback-relayed calls. collectFees also accepts msg.sender == account,
1725 // so a fallbackHandler==kernel Safe could otherwise force its own fee outflows. A direct
1726 // call is exactly selector + 4 static args (see COLLECT_FEES_CALLDATA_LEN).
1727  if (msg.data.length != COLLECT_FEES_CALLDATA_LEN) revert UnexpectedCalldataLength();
1728 _requireRegistered(account);
1729  AccountConfig storage cfg = configs[account];
1730  if (!cfg.sessionActive) revert SessionInactive(account);
1731 // Fee collection is a fund-moving operation, so it is restricted to the manager or
1732 // the Safe account itself (the latter covers non-forwarding ERC-1271 managers, and is
```

```
1733 // the owner-controlled backstop). The permissionSigner
     manages the permission registry
1734 // and never moves funds, so it is intentionally excluded.
1735  if (msg.sender != cfg.manager && msg.sender != account) {
1736  revert NotManager(msg.sender, cfg.manager);
1737 }
1738  address feePolicy = cfg.feePolicy;
1739  if (feePolicy == address(0)) revert FeePolicyNotSet();
1740  if (grossFee == 0) revert ZeroFee();
1741  if (feeToken != cfg.feeAsset) revert FeeTokenMismatch(fee-
     Token, cfg.feeAsset);
1742
1743 // Recipient is always pulled from the policy — prevents
     manager from redirecting fees.
1744  address recipient = IFeePolicy(feePolicy).feeRecipient();
1745  if (recipient == address(0)) revert ZeroAddress();
1746
1747 (uint256 maxFee, address distributor, uint256 distribu-
     torBps) =
1748  IFeePolicy(feePolicy).computeFee(account, currentNav);
1749  if (grossFee > maxFee) revert FeeTooLarge(grossFee, maxFee);
1750  if (distributorBps > 10_000) revert DistributorBp-
     sTooLarge(distributorBps);
1751
1752 // Prevent payout targets from being the kernel itself:
     ETH transfers would revert
1753 // (no receive/fallback) blocking all collections; ERC-20
     transfers would permanently
1754 // trap tokens since the kernel has no sweep mechanism.
1755  if (treasury == address(this) || distributor == address(this)
     || recipient == address(this)) {
1756  revert ZeroAddress();
1757 }
1758
1759  uint256 protocolCut    = Math.mulDiv(grossFee,
     governance.currentProtocolCutBps(), 10_000);
1760  uint256 remainder      = grossFee - protocolCut;
1761  uint256 distributorCut = Math.mulDiv(remainder,
     distributorBps, 10_000);
1762 // If the policy returns a non-zero distributorBps but a
     zero distributor address,
1763 // fold the distributor share into managerTake rather than
     silently dropping it.
1764  if (distributor == address(0)) distributorCut = 0;
1765  uint256 managerTake    = remainder - distributorCut;
1766
1767
```

```
    // Record state update BEFORE external transfers (CEI
    pattern).
1768 // This prevents a policy that reverts after transfers
    from leaving funds extracted
1769 // but state un-updated, which would allow a second
    collection over the same period.
1770  IFeePolicy(feePolicy).recordCollection(account, grossFee,
    currentNav);
1771
1772  if (feeToken == address(0)) {
1773  if (protocolCut    > 0) _safeTransferETH(account,
    treasury,    protocolCut);
1774  if (distributorCut > 0) _safeTransferETH(account,
    distributor, distributorCut);
1775  if (managerTake    > 0) _safeTransferETH(account,
    recipient,   managerTake);
1776 } else {
1777  if (protocolCut    > 0) _safeTransferERC20(account,
    feeToken, treasury,    protocolCut);
1778  if (distributorCut > 0) _safeTransferERC20(account,
    feeToken, distributor, distributorCut);
1779  if (managerTake    > 0) _safeTransferERC20(account,
    feeToken, recipient,   managerTake);
1780 }
1781
1782  emit FeesCollected(account, feeToken, grossFee, protocolCut,
    distributorCut, managerTake);
1783 }
1784
1785 /// @dev Execute a native ETH transfer out of the Safe via
    module call.
1786 /// @dev Reverts the entire fee collection if the Safe call
    returns false.
1787 ///      Intentional design: partial payouts would leave split
    invariants broken.
1788 ///      Use ERC-20 tokens where recipient payability cannot
    be guaranteed.
1789  function _safeTransferETH(address account, address to, uint256
    value) internal {
1790  if (!ISafe(account).execTransactionFromModule(to, value,
    "", 0)) revert FeeTransferFailed();
1791 }
1792
1793 /// @dev Execute an ERC-20 transfer out of the Safe via module
    call, verifying BOTH the
1794 ///      Safe module's success bool AND the token's own return
    value (SafeERC20 semantics).
```

```
1795 ///      The module-only bool is true whenever the inner `transfer` does not revert, so a
1796 ///      token that returns `false` without reverting — or returns malformed data — would
1797 ///      otherwise be recorded as a collected fee while no tokens moved. This reverts on
1798 ///      that case. A compliant token that returns nothing (non-standard, e.g. some USDT
1799 ///      deployments) is tolerated as success, matching SafeERC20.
1800 ///      The return value is decoded as a uint256 compared to 1 (rather than abi.decode to
1801 ///      bool) so a non-canonical word reverts cleanly with FeeTransferFailed instead of a
1802 ///      decode panic, consistent with the permission-evaluation decode elsewhere.
1803 /// @dev OUT OF SCOPE: fee-on-transfer shortfall. A fee-on-transfer token returns `true`
1804 ///      and does move tokens, just fewer than `amount`; this check does not guarantee the
1805 ///      recipient received `amount`. That risk is bounded by the account's feeAsset choice.
1806  function _safeTransferERC20(address account, address token, address to, uint256 amount) internal {
1807  bytes memory data = abi.encodeCall(IERC20.transfer, (to, amount));
1808 (bool ok, bytes memory ret) = ISafe(account).execTransac-tionFromModuleReturnData(token, 0, data, 0);
1809  if (!ok) revert FeeTransferFailed();
1810  if (ret.length != 0 && (ret.length < 32 || abi.decode(ret, (uint256)) != 1)) revert FeeTransferFailed();
1811 }
1812
1813 //
     ------------------------------------------------------------------------
1814 // 5. Principal tracking
1815 //
     ------------------------------------------------------------------------
1816
1817 /// @notice Record a deposit into the account. Only the permissionSigner may call.
1818 /// @dev    These values are informational. They are not used to constrain fee
1819 ///         collection on-chain, but may be used by off-chain tooling and future policy
1820 ///         contracts to derive context-aware fee limits.
1821 /// @param  account The registered Safe account.
```

```
1822 /// @param  amount  Deposit amount to record (in the account's
     base currency units).
1823  function recordDeposit(address account, uint256 amount)
     external nonReentrant {
1824 _requireRegistered(account);
1825  if (msg.sender != configs[account].permissionSigner) revert
     NotPermissionSigner();
1826  uint256 newDeposits = cumulativeDeposits[account] +=
     amount;
1827  emit DepositRecorded(account, amount, newDeposits);
1828 }
1829
1830 /// @notice Record a withdrawal from the account. Only the
     permissionSigner may call.
1831 /// @param  account The registered Safe account.
1832 /// @param  amount  Withdrawal amount to record (in the
     account's base currency units).
1833  function recordWithdrawal(address account, uint256 amount)
     external nonReentrant {
1834 _requireRegistered(account);
1835  if (msg.sender != configs[account].permissionSigner) revert
     NotPermissionSigner();
1836  uint256 newWithdrawals = cumulativeWithdrawals[account]
     += amount;
1837  emit WithdrawalRecorded(account, amount, newWithdrawals);
1838 }
1839
1840 //
     ------------------------------------------------------------------------
1841 // Public helpers
1842 //
     ------------------------------------------------------------------------
1843
1844 /// @notice Compute the EIP-712 digest for a given struct
     hash.
1845 /// @dev    Exposed for off-chain tooling, frontends, and
     tests.
1846 /// @param  structHash The EIP-712 struct hash (output of
     `keccak256(abi.encode(TYPEHASH, ...))`).
1847 /// @return            The final EIP-712 digest including
     domain separator.
1848  function hashTypedDataV4(bytes32 structHash) external view
     returns (bytes32) {
1849  return _hashTypedDataV4(structHash);
1850 }
1851
1852
```

```
    //
    ------------------------------------------------------------------------
1853 // Internal helpers
1854 //
    ------------------------------------------------------------------------
1855
1856 /// @dev Reverts with AccountNotRegistered when the account
    has not been registered.
1857  function _requireRegistered(address account) internal view {
1858  if (!registered[account]) revert AccountNotRegistered(ac-
    count);
1859 }
1860
1861 /// @dev Return the flat registration fee for a permission.
1862  function _calcPermissionFee() internal view returns (uint256)
    {
1863  return governance.permissionRegistrationFee();
1864 }
1865
1866 /// @dev Forward `fee` to the treasury and refund any ETH
    overpayment to msg.sender.
1867 ///      All callers (registerPermission, replacePermission,
    registerPermissions) complete
1868 ///      every state mutation (nonce increment, permission
    array update) before invoking
1869 ///      this function. The refund callback to msg.sender
    therefore occurs after all
1870 ///      state is settled; re-entry into any nonReentrant
    function is blocked. Any
1871 ///      re-entry into non-guarded functions (revokeSession,
    activateSession) still
1872 ///      requires a valid permissionSigner signature, preventing
    unauthorised mutations.
1873  function _collectRegistrationFee(uint256 fee) internal {
1874  if (fee > 0) {
1875 (bool ok,) = treasury.call{value: fee}("");
1876  if (!ok) revert FeeTransferFailed();
1877 }
1878  uint256 excess = msg.value - fee;
1879  if (excess > 0) {
1880 (bool ok,) = msg.sender.call{value: excess}("");
1881  if (!ok) revert FeeTransferFailed();
1882 }
1883 }
1884
1885 /// @dev Remove a permission from the account's list using
    swap-and-pop to preserve
```

```
1886 ///      packed storage. Updates `_permissionIndex` for the
     swapped element.
1887  function _removePermission(address account, address permis-
     sion) internal {
1888  uint256 idx = _permissionIndex[account][permission];
1889  if (idx == 0) revert PermissionNotRegistered(permission);
1890  address[] storage perms = _permissions[account];
1891  uint256 lastIdx = perms.length - 1;
1892  if (idx - 1 != lastIdx) {
1893  address last = perms[lastIdx];
1894  perms[idx - 1] = last;
1895 _permissionIndex[account][last] = idx;
1896 }
1897  perms.pop();
1898  delete _permissionIndex[account][permission];
1899 }
1900
1901 /// @dev Remove every permission registered for an account,
     clearing both the ordered
1902 ///      list and the index mapping, and emitting
     `PermissionRevoked` for each. Used by
1903 ///      `setManager` to reset mandates on signer rotation.
     Bounded by
1904 ///      governance.maxPermissionsPerAccount(). Unlike
     `_removePermission`, this does not
1905 ///      swap-and-pop per element: it reads each entry, clears
     its index, then deletes the
1906 ///      whole array in one shot — so the array is never
     mutated mid-iteration.
1907  function _clearPermissions(address account) internal {
1908  address[] storage perms = _permissions[account];
1909  uint256 len = perms.length;
1910  for (uint256 i; i < len;) {
1911  address perm = perms[i];
1912  delete _permissionIndex[account][perm];
1913  registrationEpoch[account][perm] += 1;
1914  emit PermissionRevoked(account, perm);
1915  unchecked { ++i; }
1916 }
1917  delete _permissions[account];
1918 }
1919
1920 /// @dev EIP-712-compliant encoding of `address[]`:
1921 ///      keccak256 of the concatenation of each address
     zero-padded to 32 bytes,
1922 ///      which is the ABI encoding of `address[]` per EIP-712
     §4.
```

```
1923 /// @param  arr The address array to hash.
1924 /// @return     The EIP-712 hash of the array.
1925  function _hashAddressArray(address[] calldata arr) internal
     pure returns (bytes32) {
1926  bytes32[] memory buf = new bytes32[](arr.length);
1927  for (uint256 i; i < arr.length; i++) {
1928  buf[i] = bytes32(uint256(uint160(arr[i])));
1929 }
1930  return keccak256(abi.encodePacked(buf));
1931 }
1932
1933 /// @dev Verify an EIP-712 permissionSigner signature against
     a struct hash.
1934 ///      Supports both EOA (ECDSA) and smart-contract (ERC-1271)
     signers.
1935  function _verifySignerSig(address account, bytes32 structHash,
     bytes memory sig) internal view {
1936  bytes32 digest = _hashTypedDataV4(structHash);
1937  if (!_recoverOrERC1271(configs[account].permissionSigner,
     digest, sig)) revert InvalidSignerSignature();
1938 }
1939
1940 /// @dev Verify a signature against `expected`, supporting
     both ECDSA and ERC-1271.
1941 ///      ECDSA is tried first regardless of code presence;
     this handles EIP-7702 accounts
1942 ///      that install transient code but do not implement
     ERC-1271.  If ECDSA recovery
1943 ///      succeeds and the recovered address matches, the
     signature is accepted immediately.
1944 ///      Otherwise, falls back to ERC-1271 when `expected` is
     a contract.
1945 /// @param  expected Address expected to have produced the
     signature.
1946 /// @param  digest   EIP-712 digest to verify against.
1947 /// @param  sig      Signature bytes.
1948 /// @return          True if the signature is valid for
     `expected`.
1949  function _recoverOrERC1271(address expected, bytes32 digest,
     bytes memory sig) internal view returns (bool) {
1950 (address recovered, ECDSA.RecoverError err,) =
     ECDSA.tryRecover(digest, sig);
1951  if (err == ECDSA.RecoverError.NoError && recovered ==
     expected) return true;
1952  if (expected.code.length > 0) {
1953  try IERC1271(expected).isValidSignature(digest, sig)
     returns (bytes4 magic) {
```

```
1954  return magic == ERC1271_MAGIC;
1955 } catch {
1956  return false;
1957 }
1958 }
1959  return false;
1960 }
1961 }
```

```
 1  // SPDX-License-Identifier: GPL-2.0-or-later
 2  pragma solidity 0.8.26;
 3
 4  import {IPermission, Context}              from
    "../interfaces/IPermission.sol";
 5  import {IBatchPermission, Call, BatchContext} from "../inter-
    faces/IBatchPermission.sol";
 6  import {IPermissionIntrospection}          from "../inter-
    faces/IPermissionIntrospection.sol";
 7  import {IFeePolicy}                         from
    "../interfaces/IFeePolicy.sol";
 8  import {SailGovernance}          from "../governance/SailGover-
    nance.sol";
 9  import {ECDSA}                   from "@openzeppelin/con-
    tracts/utils/cryptography/ECDSA.sol";
10  import {EIP712}                  from "@openzeppelin/con-
    tracts/utils/cryptography/EIP712.sol";
11  import {ReentrancyGuard}         from "@openzeppelin/con-
    tracts/utils/ReentrancyGuard.sol";
12  import {IERC1271}                from "@openzeppelin/contracts/in-
    terfaces/IERC1271.sol";
13  import {IERC20}                  from "@openzeppelin/contracts/to-
    ken/ERC20/IERC20.sol";
14  import {Math}                    from "@openzeppelin/con-
    tracts/utils/math/Math.sol";
15
16  /// @dev Minimal Safe factory interface — used only in `createAc-
    count`.
17  interface ISafeFactory {
18   function createProxyWithNonce(address singleton, bytes
    calldata initializer, uint256 saltNonce)
19   external
20   returns (address proxy);
21
22  /// @notice The proxy creation bytecode the factory deploys
    (excludes the appended
```

```
23  ///         singleton constructor arg). For Safe v1.4.1 this
    is `type(SafeProxy).creationCode`.
24  /// @dev    The kernel uses this to predict the CREATE2 proxy
    address locally (see
25  ///         `createAccount`), so it can adopt an already-deployed
    proxy (idempotency /
26  ///         front-run resilience) without a factory-side
    predictor. Safe v1.4.1's factory
27  ///         exposes `proxyCreationCode()` but NOT a view
    address predictor —
28  ///         `calculateCreateProxyWithNonceAddress` was a
    revert-based simulator removed after v1.3.0.
29   function proxyCreationCode() external pure returns (bytes
    memory);
30  }
31
32  /// @dev Minimal Safe module interface — used for executing
    transactions and fee transfers.
33  /// @dev DEPLOY ASSUMPTION: only Safe v1.4.1-style proxies may be
    governance-codehash-allowlisted —
34  ///     i.e. proxies that (a) intercept masterCopy() (0xa619486e)
    from storage slot 0 in their own
35  ///     fallback, (b) expose nonce(), and (c) implement Safe-core
    checkSignatures(bytes32,bytes,bytes).
36  ///     registerAccount's #9 singleton pin and #4 owner-signature
    gate rely on all three.
37  interface ISafe {
38   function execTransactionFromModule(address to, uint256 value,
    bytes calldata data, uint8 operation)
39   external
40   returns (bool success);
41
42   function execTransactionFromModuleReturnData(address to,
    uint256 value, bytes calldata data, uint8 operation)
43   external
44   returns (bool success, bytes memory returnData);
45
46   function isModuleEnabled(address module) external view returns
    (bool);
47
48  /// @notice The Safe's transaction nonce. Incremented by
    `execTransaction` (before the inner
49  ///         call runs) and never by `setup` — so a value of 0
    proves the Safe has not yet
50  ///         executed an owner-approved transaction (used to
    reject setup-time registration).
51   function nonce() external view returns (uint256);
```

```
52
53  /// @notice The Safe singleton (master copy) the proxy delegates
    to. On a genuine SafeProxy
54  ///         v1.4.1 this is intercepted by the proxy's own
    fallback (selector 0xa619486e) and
55  ///         read from storage slot 0, so it cannot be forged
    by a hostile singleton.
56   function masterCopy() external view returns (address);
57
58  /// @notice Validate `signatures` over `dataHash` against the
    Safe's owner set and threshold.
59  ///         Safe-core method (not the fallback handler's
    ERC-1271 entrypoint), so it carries no
60  ///         dependency on which fallbackHandler is configured.
    Reverts (GS0xx) on any failure;
61  ///         returns nothing on success. `data` is only consulted
    for legacy contract-signature
62  ///         (v==0) entries (requires keccak256(data)==dataHash);
    empty for EOA / approved-hash.
63   function checkSignatures(bytes32 dataHash, bytes calldata
    data, bytes calldata signatures) external view;
64  }
65
66  /// @title  SailKernel
67  /// @notice Central execution kernel for the Sail protocol.
68  ///
69  ///         The kernel manages a registry of Safe accounts, each
    with a set of
70  ///         permission contracts. When a manager submits a signed
    transaction, the
71  ///         kernel:
72  ///           1. Verifies the manager's EIP-712 signature and
    nonce.
73  ///           2. Evaluates the named registered permission (only
    the selected permission must return true).
74  ///           3. Executes the transaction via Safe's module
    interface.
75  ///
76  ///         Key design properties:
77  ///           • Deny-by-default: accounts with no registered
    permissions cannot dispatch.
78  ///           • Permission cap: governance-tunable limit (1–100)
    per account to bound
79  ///             gas usage in the dispatch loop; read from
    SailGovernance at registration time.
80  ///           • Salt binding: `createAccount` binds the CREATE2
    salt to msg.sender to
```

```
81  ///           prevent front-running attacks on Safe registration.
82  ///           • Two-tier nonces: manager nonces gate dispatch;
    signer nonces gate
83  ///             permission-registry operations, preventing
    cross-operation replay.
84  ///         • ERC-1271 support: both manager and permissionSigner
    may be smart contracts.
85  ///
86  /// @custom:security-contact security@sail.money
87  contract SailKernel is EIP712, ReentrancyGuard {
88  //
    ------------------------------------------------------------------------
89  // Constants
90  //
    ------------------------------------------------------------------------
91
92  /// @notice Gas budget allocated to each permission's `evaluate`
    staticcall.
93  ///         A revert or gas exhaustion within a permission is
    treated as a false return.
94  ///         Increased to 150_000 to accommodate templates
    (e.g. GMXPerpPermission,
95  ///         AzuroPredictionPermission, LimitlessPrediction-
    Permission) that use
96  ///         `try this._decode*(...)` external calls within
    their evaluate paths.
97   uint256 public constant PERMISSION_GAS_CAP = 150_000;
98
99  /// @notice Maximum number of subcalls in a single batch
    dispatch.
100 /// @dev    Bounds gas consumption in the subcall execution
    loop. A manager that needs
101 ///         more than this can split into multiple consecutive
    batch dispatches.
102  uint256 public constant MAX_BATCH_LENGTH = 16;
103
104 /// @notice Gas budget allocated to a batch permission's
    `evaluateBatch` staticcall.
105 /// @dev    Higher than PERMISSION_GAS_CAP because the batch
    evaluator must inspect
106 ///         every subcall's calldata; a revert, OOG, or malformed
    return is treated
107 ///         as a false return (fail-closed).
108  uint256 public constant BATCH_EVAL_GAS_CAP = 1_000_000;
109
110 /// @dev Gas budget for the `isBatchPermission()` type-detec-
    tion staticcall. A view
```

```
111 ///      function returning a bool should comfortably fit; a
    larger budget would
112 ///      enlarge the attack surface without benefit.
113  uint256 private constant BATCH_DETECT_GAS_CAP = 20_000;
114
115 /// @dev Large nonce step applied to managerNonces/batchNonces
    when a signer restriction op
116 ///      (revokePermission, replacePermission, revokeSession,
    revokePermissions) takes effect.
117 ///      Invalidates any outstanding manager-signed dispatches
    that pre-date the restriction
118 ///      without requiring separate nonce-rotation messages
    (Octane finding #7 related).
119  uint256 private constant NONCE_EPOCH_INCREMENT = 1 << 128;
120
121 /// @dev ERC-1271 magic value returned by `isValidSignature`
    for a valid signature.
122  bytes4  private constant ERC1271_MAGIC           = 0x1626ba7e;
123
124 /// @dev Exact calldata length of `setManager(address)`:
    selector (4) + 1 static arg (32).
125 ///      Used by the W1 fallback-relay guard — a Safe
    FallbackManager relay appends the
126 ///      original caller's 20 bytes, so any deviation from
    this length flags a relayed call.
127  uint256 private constant SET_MANAGER_CALLDATA_LEN  = 36;
128
129 /// @dev Exact calldata length of `collectFees(ad-
    dress,uint256,uint256,address)`:
130 ///      selector (4) + 4 static args (128). See
    `SET_MANAGER_CALLDATA_LEN` for the W1 rationale.
131  uint256 private constant COLLECT_FEES_CALLDATA_LEN = 132;
132
133 //
    ------------------------------------------------------------------------
134 // EIP-712 type hashes
135 //
    ------------------------------------------------------------------------
136
137 /// @notice EIP-712 type hash for manager dispatch authorisa-
    tion.
138 ///         Type string: "Dispatch(address account,address
    permission,address target,uint256 value,bytes32 dataHash,uint256
    nonce,uint256 deadline)"
139 /// @dev BREAKING CHANGE from v1: `permission` field added.
    Any pre-signed dispatch
140
```

```
///      messages created against the v1 typehash are permanently invalid after this upgrade.
///      Off-chain systems (keeper bots, SDK integrations) must re-generate signatures.
 bytes32 public constant DISPATCH_TYPEHASH = keccak256(
"Dispatch(address account,address permission,address target,uint256 value,bytes32 dataHash,uint256 nonce,uint256 deadline)"
);

/// @notice EIP-712 type hash for single-permission registration.
///         Type string: "RegisterPermission(address account,address permission,uint256 nonce,uint256 deadline)"
 bytes32 public constant REGISTER_PERMISSION_TYPEHASH = keccak256(
"RegisterPermission(address account,address permission,uint256 nonce,uint256 deadline)"
);

/// @notice EIP-712 type hash for single-permission revocation.
///         Type string: "RevokePermission(address account,address permission,uint256 nonce,uint256 deadline)"
 bytes32 public constant REVOKE_PERMISSION_TYPEHASH = keccak256(
"RevokePermission(address account,address permission,uint256 nonce,uint256 deadline)"
);

/// @notice EIP-712 type hash for atomic permission replacement.
///         Type string: "ReplacePermission(address account,address oldPermission,address newPermission,uint256 nonce,uint256 deadline)"
 bytes32 public constant REPLACE_PERMISSION_TYPEHASH = keccak256(
"ReplacePermission(address account,address oldPermission,address newPermission,uint256 nonce,uint256 deadline)"
);

/// @notice EIP-712 type hash for atomic batch permission replacement.
///         Type string: "ReplacePermissions(address account,address[] oldPermissions,address[] newPermissions,uint256 nonce,uint256 deadline)"
 bytes32 public constant REPLACE_PERMISSIONS_TYPEHASH = keccak256(
"ReplacePermissions(address account,address[] oldPermissions,address[] newPermissions,uint256 nonce,uint256 deadline)"
```

```
168  );
169
170  /// @notice EIP-712 type hash for session revocation.
171  ///         Type string: "RevokeSession(address account,uint256
     nonce,uint256 deadline)"
172   bytes32 public constant REVOKE_SESSION_TYPEHASH = keccak256(
173  "RevokeSession(address account,uint256 nonce,uint256
     deadline)"
174  );
175
176  /// @notice EIP-712 type hash for session re-activation.
177  ///        Type string: "ActivateSession(address account,uint256
     nonce,uint256 deadline)"
178   bytes32 public constant ACTIVATE_SESSION_TYPEHASH = keccak256(
179  "ActivateSession(address account,uint256 nonce,uint256
     deadline)"
180  );
181
182  /// @notice EIP-712 type hash for fee policy updates.
183  ///         Type string: "SetFeePolicy(address account,address
     newFeePolicy,address feeAsset,uint256 nonce,uint256 deadline)"
184   bytes32 public constant SET_FEE_POLICY_TYPEHASH = keccak256(
185  "SetFeePolicy(address account,address newFeePolicy,address
     feeAsset,uint256 nonce,uint256 deadline)"
186  );
187
188  /// @notice EIP-712 type hash for batch permission registration.
189  ///         Type string: "RegisterPermissions(address
     account,address[] permissions,uint256 nonce,uint256 deadline)"
190  ///         The `address[]` field is encoded as keccak256 of
     the ABI-packed padded addresses
191  ///         per EIP-712 §4 (see `_hashAddressArray`).
192   bytes32 public constant REGISTER_PERMISSIONS_TYPEHASH =
     keccak256(
193  "RegisterPermissions(address account,address[] permis-
     sions,uint256 nonce,uint256 deadline)"
194  );
195
196  /// @notice EIP-712 type hash for batch permission revocation.
197  ///         Type string: "RevokePermissions(address
     account,address[] permissions,uint256 nonce,uint256 deadline)"
198   bytes32 public constant REVOKE_PERMISSIONS_TYPEHASH =
     keccak256(
199  "RevokePermissions(address account,address[] permis-
     sions,uint256 nonce,uint256 deadline)"
200  );
201
```

```
202  /// @notice EIP-712 type hash for manager batch-dispatch
     authorisation.
203  ///         Type string: "DispatchBatch(address account,address
     permission,bytes32 callsHash,uint256 nonce,uint256 deadline)"
204  ///         `callsHash` = keccak256(abi.encode(calls)) — see
     `dispatchBatch` for the encoding.
205   bytes32 public constant DISPATCH_BATCH_TYPEHASH = keccak256(
206  "DispatchBatch(address account,address permission,bytes32
     callsHash,uint256 nonce,uint256 deadline)"
207  );
208
209  /// @notice EIP-712 type hash for owner-authorised self-reg-
     istration via `registerAccount`.
210  ///         Type string: "RegisterAccount(address account,ad-
     dress permissionSigner,address manager,address feePolicy,address
     feeAsset,uint256 deadline)"
211  /// @dev    The owner-set+threshold signature over this struct
     is the robust gate for the
212  ///         self-registration path: it cannot be produced by
     a Safe.setup delegatecall helper
213  ///         (which has no owner keys). No nonce field —
     registration is one-shot (the
214  ///         `registered[account]` latch is never cleared, so
     a replayed signature reverts with
215  ///         AccountAlreadyRegistered) and the EIP-712 domain
     pins chainId against cross-chain replay.
216   bytes32 public constant REGISTER_ACCOUNT_TYPEHASH = keccak256(
217  "RegisterAccount(address account,address permission-
     Signer,address manager,address feePolicy,address feeAsset,uint256
     deadline)"
218  );
219
220  //
     ------------------------------------------------------------------------
221  // Account state
222  //
     ------------------------------------------------------------------------
223
224  /// @notice Per-account configuration set at registration and
     updatable via signed ops.
225   struct AccountConfig {
226  /// @dev Address whose signatures authorise permission-reg-
     istry operations.
227   address permissionSigner;
228  /// @dev Address whose signatures authorise dispatch calls.
229   address manager;
230
```

```
    /// @dev Fee policy contract; address(0) means no fee
    policy is configured.
231  address feePolicy;
232 /// @dev Canonical fee settlement token; address(0) =
    native ETH.
233  address feeAsset;
234 /// @dev When false, all dispatch calls for this account
    are blocked.
235  bool    sessionActive;
236 }
237
238 /// @notice Enriched permission descriptor returned by
    getPermissionsWithInfo.
239  struct PermissionInfo {
240 /// @dev Contract address of the permission.
241  address permission;
242 /// @dev True if the permission implements IBatchPermis-
    sion.isBatchPermission().
243  bool    isBatch;
244 /// @dev True if the permission implements IPermissionIn-
    trospection.
245  bool    hasIntrospection;
246 /// @dev From IPermissionIntrospection.permissionId();
    bytes32(0) if not supported.
247  bytes32 permissionId;
248 /// @dev From IPermissionIntrospection.permissionVersion();
    bytes32(0) if not supported.
249  bytes32 permissionVersion;
250 }
251
252 /// @notice Per-account configuration.
253  mapping(address account => AccountConfig)                    pub-
    lic  configs;
254
255 /// @notice Whether an account has been registered with the
    kernel.
256  mapping(address account => bool)                           pub-
    lic  registered;
257
258 /// @dev Ordered list of permission addresses per account.
    Maintained as a packed array
259 ///      with swap-and-pop removal to keep indices compact.
    Max length: governance.maxPermissionsPerAccount().
260  mapping(address account => address[])                      pri-
    vate _permissions;
261
262
```

```
    /// @dev Index-plus-one of each permission in `_permissions[ac-
    count]`.
263 ///      Zero means the permission is not registered. Stored
    as index+1 to distinguish
264 ///      "registered at slot 0" from "not registered".
265  mapping(address account => mapping(address permission =>
    uint256)) private _permissionIndex;
266
267 /// @notice Per-account nonces for manager dispatch signatures.
268 ///         Separate from signerNonces to prevent cross-operation
    replay.
269  mapping(address account => uint256) public managerNonces;
270
271 /// @notice Per-account nonces for manager batch-dispatch
    signatures.
272 /// @dev    Lives in a separate namespace from `managerNonces`
    so that single
273 ///         dispatches and batch dispatches cannot replay
    across each other.
274 ///         Consuming a batch nonce does not advance dispatch
    nonces, and vice versa.
275  mapping(address account => uint256) public batchNonces;
276
277 /// @notice Per-account nonces for permissionSigner operations
278 ///         (register, revoke, replace, session, feePolicy).
279 /// @dev    All signer operations share a single nonce counter
    per account.
280 ///         Concurrent independent operations must use batch
    variants (registerPermissions,
281 ///         revokePermissions) rather than multiple single-op
    calls. Submitting two
282 ///         single-op calls simultaneously will cause one to
    fail due to nonce conflict.
283  mapping(address account => uint256) public signerNonces;
284
285 /// @notice Per-(account, permission) registration epoch. A
    plain monotonic counter, bumped
286 ///         every time a permission LEAVES an account's registry
    (revoke, the removed side of a
287 ///         replace, or a manager-rotation clear). It is NOT
    bumped on registration: a freshly
288 ///         registered permission keeps its epoch so the
    configure-then-register flow (see
289 ///         MandateFactory) stamps the same epoch the dispatch
    later reads.
290 /// @dev    Pushed into Context/BatchContext at dispatch time
    so a ConfigurablePermission can
```

```
291 ///         compare it against the epoch it stamped at
    configure() time and fail closed when a
292 ///         stale configuration survives a revoke 're-register
    cycle (Octane #2 / #8). Distinct
293 ///         from the packed manager/batch nonce epochs
    (NONCE_EPOCH_INCREMENT) — this is a clean
294 ///         standalone +1 counter per (account, permission).
295  mapping(address account => mapping(address permission =>
    uint256)) public registrationEpoch;
296
297 //
    ------------------------------------------------------------------------
298 // Principal tracking
299 //
    ------------------------------------------------------------------------
300
301 /// @notice Cumulative deposit amount recorded for each account
    (informational).
302 ///         Written by the permissionSigner via `recordDeposit`.
303  mapping(address account => uint256) public cumulativeDeposits;
304
305 /// @notice Cumulative withdrawal amount recorded for each
    account (informational).
306 ///         Written by the permissionSigner via
    `recordWithdrawal`.
307  mapping(address account => uint256) public cumulativeWith-
    drawals;
308
309 /// @dev Records the single fee asset a policy instance has
    been bound to for an account.
310 ///      `bound` is an explicit flag (not asset != 0) because
    address(0) is a valid fee
311 ///      asset (native ETH), so it cannot double as the "unbound"
    sentinel.
312  struct PolicyAssetBinding {
313  bool    bound;
314  address asset;
315 }
316
317 /// @notice Per-(account, policy) binding pinning a fee-policy
    instance to the single fee
318 ///         asset it was first used with for that account.
    Reusing the SAME policy instance
319 ///         with a DIFFERENT asset would leave the policy's
    persisted high-water mark in
320 ///         stale units, inflating the next performance fee
    on a one-time over-collection.
```

```
321 ///         To change the fee asset, point the account at a
    fresh policy instance, which
322 ///         carries its own fresh per-account state.
323  mapping(address account => mapping(address policy => PolicyAs-
    setBinding)) private _policyAssetBinding;
324
325 //
    ----------------------------------------------------------------------
326 // Protocol references
327 //
    ----------------------------------------------------------------------
328
329 /// @notice The governance contract that stores fee parameters
    and the protocol cut.
330  SailGovernance public immutable governance;
331
332 /// @notice Runtime codehash of the audited, immutable
    SafeModuleEnabler this kernel pins as
333 ///         the ONLY permissible Safe.setup delegatecall `to`
    target (W2). Captured at
334 ///         construction from the deployed helper, so it
    matches the launch build by
335 ///         construction — no hand-copied literal, no
    recompile-mismatch risk. `createAccount`
336 ///         requires every non-zero `setupTarget` to carry
    exactly this codehash, so a
337 ///         governance mistake allowlisting a mutable/look-alike
    helper cannot open the
338 ///         setup-delegatecall takeover surface.
339  bytes32        public immutable EXPECTED_SETUP_CODEHASH;
340
341 /// @notice Address that receives the protocol's share of
    collected fees.
342  address        public treasury;
343
344 //
    ----------------------------------------------------------------------
345 // Events
346 //
    ----------------------------------------------------------------------
347
348 /// @notice Emitted when a new account is registered.
349 /// @param  account         The Safe address that was registered.
350 /// @param  permissionSigner Address authorised to manage
    permissions.
351 /// @param  manager          Address authorised to dispatch
    transactions.
```

```
352   event AccountRegistered(address indexed account, address
      indexed permissionSigner, address indexed manager);
353
354   /// @notice Emitted when a permission is added to an account.
355   /// @param  account    The Safe account.
356   /// @param  permission The permission contract address.
357   event PermissionRegistered(address indexed account, address
      indexed permission);
358
359   /// @notice Emitted when a permission is removed from an
      account.
360   /// @param  account    The Safe account.
361   /// @param  permission The permission contract address that
      was removed.
362   event PermissionRevoked(address indexed account, address
      indexed permission);
363
364   /// @notice Emitted when a permission is atomically replaced.
365   /// @param  account        The Safe account.
366   /// @param  oldPermission  Permission that was removed.
367   /// @param  newPermission  Permission that was added in its
      place.
368   event PermissionReplaced(address indexed account, address
      indexed oldPermission, address indexed newPermission);
369
370   /// @notice Emitted when the manager session is suspended for
      an account.
371   /// @param  account The Safe account whose session was revoked.
372   event SessionRevoked(address indexed account);
373
374   /// @notice Emitted when a previously revoked session is
      re-activated.
375   /// @param  account The Safe account whose session was
      activated.
376   event SessionActivated(address indexed account);
377
378   /// @notice Emitted when the fee policy for an account is
      updated.
379   /// @param  account       The Safe account.
380   /// @param  newFeePolicy  The new fee policy contract address
      (address(0) = cleared).
381   event FeePolicyUpdated(address indexed account, address indexed
      newFeePolicy);
382
383   /// @notice Emitted when an account's manager (delegated
      signer) is rotated.
384
```

```
    /// @dev    Rotation clears the account's permission set, so
    a `ManagerChanged` is
385 ///         accompanied by a `PermissionRevoked` for each
    previously-registered mandate.
386 /// @param  account    The Safe account whose manager changed.
387 /// @param  oldManager The previous manager address.
388 /// @param  newManager The new manager address.
389  event ManagerChanged(address indexed account, address indexed
    oldManager, address indexed newManager);
390
391 /// @notice Emitted on each successful dispatch.
392 /// @dev    `dataHash` is keccak256(calldata) — the raw bytes
    are recoverable from the tx.
393 /// @param  account     The Safe account that executed the
    transaction.
394 /// @param  permission  The registered permission that
    authorised the call.
395 /// @param  target      The call target.
396 /// @param  selector    Leading 4 bytes of calldata; bytes4(0)
    if calldata is shorter than 4 bytes.
397 /// @param  value       Native ETH forwarded with the call
    (wei).
398  event Dispatched(
399  address indexed account,
400  address indexed permission,
401  address target,
402  bytes4  selector,
403  uint256 value
404 );
405
406 /// @notice Emitted on each successful batch dispatch.
407 /// @param  account    The Safe account that executed the
    batch.
408 /// @param  permission The batch-aware permission that
    authorised the batch.
409 /// @param  batchHash  keccak256(abi.encode(calls)) — stable
    identifier for the call sequence.
410 /// @param  callCount  Number of subcalls in the batch.
411  event BatchDispatched(
412  address indexed account,
413  address indexed permission,
414  bytes32 batchHash,
415  uint256 callCount
416 );
417
418 /// @notice Emitted on each successful fee collection.
419 /// @param  account        The Safe account that was charged.
```

```
420 /// @param  feeToken       ERC-20 token used for fee payment;
    address(0) = native ETH.
421 /// @param  grossFee       Total fee collected.
422 /// @param  protocolCut    Portion forwarded to the treasury.
423 /// @param  distributorCut Portion forwarded to the fee policy's
    distributor.
424 /// @param  managerTake    Remainder forwarded to the manager's
    recipient.
425  event FeesCollected(
426  address indexed account,
427  address indexed feeToken,
428  uint256 grossFee,
429  uint256 protocolCut,
430  uint256 distributorCut,
431  uint256 managerTake
432 );
433
434 /// @notice Emitted when a deposit is recorded for an account.
435 /// @param  account    The Safe account.
436 /// @param  amount     Amount deposited in this call.
437 /// @param  cumulative Running cumulative deposit total after
    this call.
438  event DepositRecorded(address indexed account, uint256 amount,
    uint256 cumulative);
439
440 /// @notice Emitted when a withdrawal is recorded for an
    account.
441 /// @param  account    The Safe account.
442 /// @param  amount     Amount withdrawn in this call.
443 /// @param  cumulative Running cumulative withdrawal total
    after this call.
444  event WithdrawalRecorded(address indexed account, uint256
    amount, uint256 cumulative);
445
446 /// @notice Emitted when the treasury address is updated.
447 /// @param  oldTreasury Previous treasury address.
448 /// @param  newTreasury New treasury address.
449  event TreasuryUpdated(address indexed oldTreasury, address
    indexed newTreasury);
450
451 //
    ------------------------------------------------------------------------
452 // Errors
453 //
    ------------------------------------------------------------------------
454
455
```

```
    /// @dev Thrown by `_registerAccount` when the account is
    already registered.
456  error AccountAlreadyRegistered(address account);
457
458 /// @dev Thrown by `_requireRegistered` when the account has
    not been registered.
459  error AccountNotRegistered(address account);
460
461 /// @dev Thrown by `dispatch` when `sessionActive` is false
    for the account.
462  error SessionInactive(address account);
463
464 /// @dev Thrown when `block.timestamp` exceeds the provided
    deadline.
465  error DeadlineExpired(uint256 deadline, uint256 current);
466
467 /// @dev Thrown when an EIP-712 manager signature cannot be
    verified.
468  error InvalidManagerSignature();
469
470 /// @dev Thrown when an EIP-712 permissionSigner signature
    cannot be verified.
471  error InvalidSignerSignature();
472
473 /// @dev Thrown when a permission returns false (or reverts)
    during dispatch.
474  error PermissionDenied(address permission);
475
476 /// @dev Thrown when `ISafe.execTransactionFromModule` returns
    false.
477  error SafeExecutionFailed();
478
479 /// @dev Thrown by `registerPermission` / `registerPermissions`
    when the permission
480 ///      is already in the account's permission set.
481  error PermissionAlreadyRegistered(address permission);
482
483 /// @dev Thrown by operations that require a permission to be
    registered when it is not.
484  error PermissionNotRegistered(address permission);
485
486 /// @dev Thrown when adding permissions would exceed
    governance.maxPermissionsPerAccount().
487  error TooManyPermissions(address account, uint256 limit);
488
489 /// @dev Thrown when the ETH sent with a registration call is
    below the required fee.
```

```
490   error InsufficientFee(uint256 required, uint256 provided);
491
492   /// @dev Thrown by `collectFees` when no fee policy is set
      for the account.
493   error FeePolicyNotSet();
494
495   /// @dev Thrown by `collectFees` when `grossFee` exceeds the
      policy's computed maximum.
496   error FeeTooLarge(uint256 requested, uint256 maxAllowed);
497
498   /// @dev Thrown when an ETH or ERC-20 fee transfer via the
      Safe fails.
499   error FeeTransferFailed();
500
501   /// @dev Thrown when the caller of a manager-only function is
      not the account's manager.
502   error NotManager(address caller, address expected);
503
504   /// @dev Thrown by `onlyGovernance` when caller is not the
      current governance address.
505   error NotGovernance();
506
507   /// @dev Thrown by `onlyTimelock` when caller is not the
      governance 48-hour timelock.
508   error NotTimelock();
509
510   /// @dev Thrown when a caller other than the account's
      permissionSigner invokes a guarded op.
511   error NotPermissionSigner();
512
513   /// @dev Thrown when a required address argument is the zero
      address.
514   error ZeroAddress();
515
516   /// @dev Thrown by `setManager` when the new manager equals
      the current one — a no-op
517   ///      rotation would needlessly clear mandates and bump
      the nonce epoch.
518   error ManagerUnchanged();
519
520   /// @dev Thrown by permission registration when the supplied
      address has no deployed code.
521   error NotAContract(address addr);
522
523   /// @dev Thrown by `collectFees` when `distributorBps` returned
      by the policy exceeds 10 000.
524   error DistributorBpsTooLarge(uint256 bps);
```

```
525
526 /// @dev Thrown by `collectFees` when the fee token does not
    match the account's configured canonical asset.
527  error FeeTokenMismatch(address provided, address expected);
528
529 /// @dev Thrown by `collectFees` when `grossFee` is zero.
530  error ZeroFee();
531
532 /// @dev Thrown by `dispatch` / `collectFees` when the protocol
    is paused.
533  error ProtocolPaused();
534
535 /// @dev Thrown when `createAccount` deploys a Safe that does
    not have this kernel enabled as a module.
536  error ModuleNotEnabled();
537
538 /// @dev Thrown by `registerAccount` when the caller Safe has
    not finalized setup (nonce == 0),
539 ///      blocking a setup-delegatecall helper from registering
    attacker-chosen principals (Octane #4).
540  error SetupNotFinalized();
541
542 /// @dev Thrown by Safe-authorized functions (`registerAc-
    count`, `setManager`, `collectFees`) when
543 ///      msg.data is not the exact static length — a Safe
    fallback relay appends the caller's 20
544 ///      bytes, so a length mismatch flags a fallbackHan-
    dler-relayed call (Octane W1).
545  error UnexpectedCalldataLength();
546
547 /// @dev Thrown by `createAccount` when the provided Safe
    factory is not in governance's trusted allowlist.
548  error UntrustedFactory(address factory);
549
550 /// @dev Thrown by `createAccount` when the provided Safe
    singleton is not in governance's trusted allowlist.
551  error UntrustedSingleton(address singleton);
552
553 /// @dev Thrown by `createAccount` when `safeInitializer` is
    too short to contain the
554 ///      Safe.setup `to` field (selector + owners offset +
    threshold + to = 100 bytes).
555  error InvalidInitializer();
556
557 /// @dev Thrown by `createAccount` when the Safe.setup
    delegatecall `to` target is not
558 ///      in governance's trusted module-setup allowlist.
```

```
559   error UntrustedModuleSetup(address setup);
560
561   /// @dev Thrown by `createAccount` when the Safe.setup
      delegatecall `to` target's runtime
562   ///      codehash does not match the immutable, audited
      SafeModuleEnabler pinned at deploy
563   ///      (`EXPECTED_SETUP_CODEHASH`). Defends against a
      governance mistake allowlisting a
564   ///      MUTABLE helper at a trusted address: even an
      address-allowlisted target is rejected
565   ///      unless its deployed bytecode is byte-identical to
      the pinned immutable helper (W2).
566   error UntrustedModuleSetupCodehash(address setup);
567
568   /// @dev Thrown by `registerAccount` when the caller's runtime
      codehash is not in
569   ///      governance's trusted Safe-proxy-codehash allowlist.
570   error UntrustedProxyCodehash(bytes32 codehash);
571
572   /// @dev Thrown by `dispatchBatch` when the calls array is
      empty.
573   error EmptyBatch();
574
575   /// @dev Thrown by `dispatchBatch` when the calls array exceeds
      MAX_BATCH_LENGTH.
576   error BatchTooLong(uint256 length);
577
578   /// @dev Thrown by `dispatchBatch` when the named permission
      does not implement IBatchPermission
579   ///      (detected via try/catch on `isBatchPermission()`).
580   error PermissionNotBatchAware(address permission);
581
582   /// @dev Thrown by `dispatchBatch` when the named permission's
      `evaluateBatch`
583   ///      returns false, reverts, runs out of gas, or returns
      malformed data.
584   error BatchPermissionDenied();
585
586   /// @dev Thrown by `dispatchBatch` when one of the batched
      subcalls reverts
587   ///      (Safe's execTransactionFromModule returns false).
588   error BatchSubcallFailed(uint256 index, address target);
589
590   /// @dev Thrown by `dispatchBatch` when a subcall targets the
      kernel itself —
591   ///      a defensive guard preventing self-targeted reentrancy
      attempts.
```

```
592   error KernelSelfTarget(uint256 index);
593
594   /// @dev Thrown by `dispatchBatch` when a subcall targets the
      zero address.
595   error BatchZeroTarget(uint256 index);
596
597   /// @dev Thrown by `dispatch`/`dispatchBatch` when a call
      targets the Safe account itself.
598   ///      Prevents module-triggered self-calls that satisfy
      Safe's onlySelf guard and could
599   ///      enable owner/threshold changes, module manipulation,
      or guard/fallback overwrites.
600   error AccountSelfTarget();
601
602   /// @dev Thrown by `_registerAccount` or `setFeePolicy` when
      the provided fee policy is not
603   ///      in governance's trusted allowlist.  Prevents
      upgradeable/metamorphic policies.
604   error UntrustedFeePolicy(address policy);
605
606   /// @dev Thrown by `setFeePolicy` when a fee-policy instance
      already used for an account
607   ///      with one fee asset is reused with a different asset.
      Reusing an instance across
608   ///      denominations would leave its persisted high-water
      mark in stale units. To change
609   ///      the fee asset, point the account at a fresh policy
      instance.
610   error FeePolicyAssetMismatch(address policy, address expected,
      address provided);
611
612   /// @dev Thrown by `replacePermissions` when `oldPermissions`
      and `newPermissions` have different lengths.
613   error ArrayLengthMismatch();
614
615   //
      ------------------------------------------------------------------------
616   // Constructor
617   //
      ------------------------------------------------------------------------
618
619   /// @notice Deploy the kernel with a governance contract,
      treasury, and the Safe.setup helper.
620   /// @param  _governance    Address of the deployed SailGovernance
      contract.
621   /// @param  _treasury      Address that will receive the
      protocol's share of fees.
```

```
622  /// @param  _setupEnabler  The deployed, immutable SafeMod-
     uleEnabler. Its runtime codehash is
623  ///                       captured into `EXPECTED_SETUP_CODEHASH`
     and pinned as the only
624  ///                       permissible Safe.setup delegatecall
     target (W2). MUST be the genuine
625  ///                       immutable helper at launch; it must
     already be deployed when the
626  ///                       kernel is constructed so its
     codehash can be read here.
627   constructor(address _governance, address _treasury, address
     _setupEnabler) EIP712("SailKernel", "1") {
628   if (_governance == address(0) || _treasury == address(0)
     || _treasury == address(this)) revert ZeroAddress();
629   governance             = SailGovernance(_governance);
630   treasury               = _treasury;
631   EXPECTED_SETUP_CODEHASH = _setupEnabler.codehash;
632  }
633
634  //
     ------------------------------------------------------------------------
635  // Modifiers
636  //
     ------------------------------------------------------------------------
637
638  /// @dev Reverts with NotGovernance when caller is not the
     current governance address.
639   modifier onlyGovernance() {
640   if (msg.sender != governance.governance()) revert
     NotGovernance();
641  _;
642  }
643
644  /// @dev Reverts with NotTimelock when caller is not the
     governance timelock.
645  ///      Used for high-impact setters that must observe the
     48-hour delay.
646   modifier onlyTimelock() {
647   if (msg.sender != address(governance.timelock())) revert
     NotTimelock();
648  _;
649  }
650
651  /// @dev Reverts with ProtocolPaused when governance.isPaused()
     returns true.
652   modifier whenNotPaused() {
653   if (governance.isPaused()) revert ProtocolPaused();
```

```
654  _;
655  }
656
657  //
     ------------------------------------------------------------------------
658  // Governance
659  //
     ------------------------------------------------------------------------
660
661  /// @notice Update the treasury address that receives the
     protocol's fee share.
662  /// @dev    Enforces the 48-hour timelock so that a compromised
     governance key cannot
663  ///         instantly redirect all protocol fee flows. Schedule
     via governance.timelock().
664  /// @param  newTreasury New treasury address. Must not be the
     zero address.
665   function setTreasury(address newTreasury) external onlyTime-
     lock {
666   if (newTreasury == address(0)) revert ZeroAddress();
667   if (newTreasury == address(this)) revert ZeroAddress();
668   address old = treasury;
669   treasury = newTreasury;
670   emit TreasuryUpdated(old, newTreasury);
671  }
672
673  //
     ------------------------------------------------------------------------
674  // 1. Account instantiation
675  //
     ------------------------------------------------------------------------
676
677  /// @notice Deploy a new Safe via factory and register it with
     the kernel in one transaction.
678  /// @dev    The salt passed to the factory is derived from
     `keccak256(saltNonce, msg.sender,
679  ///         permissionSigner, manager, feePolicy)` so that a
     front-runner supplying different
680  ///         principals lands at a different CREATE2 address
     and cannot squat the registration
681  ///         (Octane #4 / #16). The Safe.setup delegatecall
     `to` target embedded in
682  ///         `safeInitializer` is validated against the trusted
     module-setup allowlist, removing
683  ///         the arbitrary-delegatecall surface (Octane #1).
     If a proxy already exists at the
684
```

```
    ///         predicted address (legitimate pre-deploy or retry)
    the factory call is skipped.
685 /// @param  safeFactory       Address of the Safe proxy factory
    contract.
686 /// @param  safeSingleton     Address of the Safe singleton
    (implementation) contract.
687 /// @param  safeInitializer   Calldata for the Safe's `setup`
    call during deployment.
688 /// @param  saltNonce         Caller-chosen nonce; combined
    with msg.sender + principals into the salt.
689 /// @param  permissionSigner  Address that will sign
    permission-registry operations.
690 /// @param  manager           Address that will sign dispatch
    calls.
691 /// @param  feePolicy         Fee policy contract; address(0)
    = no fee policy.
692 /// @param  feeAsset          Canonical fee settlement token;
    address(0) = native ETH.
693 /// @return account           Address of the deployed (or
    pre-existing) Safe proxy.
694  function createAccount(
695  address safeFactory,
696  address safeSingleton,
697  bytes calldata safeInitializer,
698  uint256 saltNonce,
699  address permissionSigner,
700  address manager,
701  address feePolicy,
702  address feeAsset
703 ) external returns (address account) {
704  if (!governance.trustedSafeFactory(safeFactory))     revert
    UntrustedFactory(safeFactory);
705  if (!governance.trustedSafeSingleton(safeSingleton)) revert
    UntrustedSingleton(safeSingleton);
706
707 // Enforce that the delegatecall target inside Safe.setup
    is an allowlisted helper.
708 // setup() ABI layout: selector(4) + owners_offset(32) +
    threshold(32) + to(32) + ...
709 // 'to' is at bytes [68:100].
710  if (safeInitializer.length < 100) revert InvalidInitial-
    izer();
711  address setupTarget = ad-
    dress(uint160(uint256(bytes32(safeInitializer[68:100]))));
712 // address(0) means no delegatecall (vanilla Safe.setup)
    — always safe, no allowlist check needed.
713
```

```
     if (setupTarget != address(0) && !governance.trustedMod-
    uleSetup(setupTarget)) revert UntrustedModuleSetup(setupTarget);
714  // W2: address-allowlisting alone is not enough — pin the
    target's runtime codehash to the
715  // audited immutable SafeModuleEnabler captured at deploy.
    This converts the operational rule
716  // ("only ever allowlist an IMMUTABLE helper") into a code
    guarantee: a governance mistake
717  // allowlisting a mutable/upgradeable look-alike at a
    trusted address cannot satisfy the pin.
718  // Gated on setupTarget != address(0) exactly like the
    address check, so the no-setup path
719  // (vanilla Safe.setup) is unaffected.
720   if (setupTarget != address(0) && setupTarget.codehash !=
    EXPECTED_SETUP_CODEHASH) {
721   revert UntrustedModuleSetupCodehash(setupTarget);
722  }
723
724   uint256 boundSalt = uint256(keccak256(abi.encode(saltNonce,
    msg.sender, permissionSigner, manager, feePolicy)));
725
726  // Predict the proxy address with the same CREATE2 formula
    SafeProxyFactory uses, so a
727  // proxy already deployed at that address (a retry, or a
    same-config front-runner) is
728  // adopted instead of reverting. Computed locally rather
    than via a factory predictor:
729  // Safe v1.4.1's factory exposes no view predictor (only
    proxyCreationCode()).
730  //   salt         = keccak256(keccak256(initializer),
    saltNonce)
731  //   deploymentData = proxyCreationCode() ++
    uint256(uint160(singleton))
732   bytes32 create2Salt = keccak256(abi.encodePacked(kec-
    cak256(safeInitializer), boundSalt));
733   bytes32 initCodeHash = keccak256(
734   abi.encodePacked(ISafeFactory(safeFactory).proxyCre-
    ationCode(), uint256(uint160(safeSingleton)))
735  );
736   address predicted = address(
737   uint160(uint256(keccak256(abi.encodePacked(bytes1(0xff),
    safeFactory, create2Salt, initCodeHash))))
738  );
739   if (predicted.code.length == 0) {
740   account = ISafeFactory(safeFactory).createProxyWith-
    Nonce(safeSingleton, safeInitializer, boundSalt);
741  } else {
```

```
742   account = predicted;
743  }
744
745   if (!ISafe(account).isModuleEnabled(address(this))) revert
     ModuleNotEnabled();
746
747  // Parity with registerAccount: the resulting proxy must
     carry an allowlisted Safe-proxy
748  // runtime codehash. The trusted factory + CREATE2 prediction
     path already implies this,
749  // so the check is redundant for a legitimately-created
     proxy — it makes the codehash trust
750  // anchor explicit and symmetric across both account-entry
     paths.
751   bytes32 accountCodehash;
752   assembly { accountCodehash := extcodehash(account) }
753   if (!governance.trustedSafeProxyCodehash(accountCodehash))
     revert UntrustedProxyCodehash(accountCodehash);
754
755  _registerAccount(account, permissionSigner, manager,
     feePolicy, feeAsset);
756  }
757
758  /// @notice Register an existing Safe that has already added
     this kernel as a module.
759  /// @dev    MUST be called by the Safe itself (msg.sender ==
     Safe). The caller must be a
760  ///         genuine Safe proxy (verified by runtime codehash
     against the trusted allowlist,
761  ///         blocking arbitrary-contract self-registration —
     finding #4a) and must already
762  ///         have this kernel enabled as a module.
763  ///
764  ///         #4 OWNER AUTHORISATION (the robust gate):
     registration is bound to an EIP-712
765  ///         signature over the principals, verified through
     the Safe's own owner-set+threshold
766  ///         check (`checkSignatures`). This defeats the
     Safe.setup-delegatecall attack that a
767  ///         view-only heuristic could not: during setup every
     Safe-storage signal (nonce, module
768  ///         state, slot-0 singleton) is attacker-forgeable,
     but a setup helper cannot produce the
769  ///         owners' signatures over this digest. If an attacker
     rewrites the owner set to sign with
770  ///         their own key, they own the Safe — there is no
     victim. (The same setup-controlling
```

```
771  ///         attacker could also enable a draining module of
     their own, so this boundary is the most
772  ///         a registration check can defend; see the PR notes.)
     The `nonce()==0` check below is
773  ///         kept purely as cheap defense-in-depth on top of
     the signature.
774  ///
775  ///         createAccount does NOT carry an owner signature:
     it routes through the internal
776  ///         `_registerAccount` (never this public function),
     is kernel-orchestrated, validates the
777  ///         singleton/factory/setup-target against governance
     allowlists, and binds the principals
778  ///         into the CREATE2 salt — so it is protected by
     construction and unaffected by this gate.
779  ///
780  ///         INVOCATION: owners sign the RegisterAccount digest
     off-chain; an owner-approved Safe
781  ///         `execTransaction` then calls this function (so
     msg.sender == the Safe, satisfying the
782  ///         codehash gate) carrying that signature. msg.sender
     == account is preserved.
783  /// @dev    `checkSignatures` is called with empty `data`,
     which suits EOA and approved-hash owners.
784  ///         A Safe owner that is itself a contract relying on
     the legacy v==0 contract-signature path
785  ///         would require non-empty `data` (keccak256(data)==di-
     gest) — a nested-Safe-owner edge case.
786  /// @param  permissionSigner  Address that will sign
     permission-registry operations.
787  /// @param  manager           Address that will sign dispatch
     calls.
788  /// @param  feePolicy         Fee policy contract; address(0)
     = no fee policy.
789  /// @param  feeAsset          Canonical fee settlement token;
     address(0) = native ETH.
790  /// @param  deadline          Unix timestamp after which the
     owner signature is invalid.
791  /// @param  ownerSig          Safe owner-set+threshold
     signature(s) over the RegisterAccount digest.
792   function registerAccount(
793   address permissionSigner,
794   address manager,
795   address feePolicy,
796   address feeAsset,
797   uint256 deadline,
798   bytes calldata ownerSig
```

```
799  ) external {
800  // No exact-length W1 guard here (cf. setManager/collect-
     Fees): `ownerSig` is dynamic, so an
801  // exact length is undefined and a minimum-length check
     would not catch a +20-byte fallback
802  // relay. The owner-signature requirement closes the W1
     fallback vector for this function
803  // directly — a relay cannot produce the owners' signature
     over the digest.
804
805  // 1. Codehash (cheap/static; uses extcodehash, not an
     ISafe method call). Pins genuine
806  //    SafeProxy bytecode before any call into the proxy.
807   bytes32 codehash;
808   assembly { codehash := extcodehash(caller()) }
809   if (!governance.trustedSafeProxyCodehash(codehash)) revert
     UntrustedProxyCodehash(codehash);
810
811  // 2. #9 trusted singleton — checked BEFORE any other
     ISafe method runs. The codehash above
812  //    only pins proxy bytecode; a genuine proxy can still
     delegate to a hostile singleton that
813  //    forges module execution/return data. masterCopy()
     is the one ISafe call that may precede
814  //    this check: the proxy answers 0xa619486e from storage
     slot 0 in its own fallback, so a
815  //    malicious singleton cannot forge it. Confirming the
     singleton here means nonce(),
816  //    isModuleEnabled(), and checkSignatures() below are
     never invoked on an unverified singleton.
817   address singleton = ISafe(msg.sender).masterCopy();
818   if (!governance.trustedSafeSingleton(singleton)) revert
     UntrustedSingleton(singleton);
819
820  // 3. #4 defense-in-depth: setup() never bumps the Safe
     nonce; execTransaction increments it
821  //    before the inner call runs. A value of 0 therefore
     flags a not-yet-finalized Safe. This is
822  //    forgeable by a setup-delegatecall helper (nonce
     lives in slot 5), so it is NOT the gate —
823  //    the owner signature below is. Kept as a cheap early
     reject.
824   if (ISafe(msg.sender).nonce() == 0) revert SetupNotFinal-
     ized();
825
826  // 4. Module must be enabled.
827
```

```
     if (!ISafe(msg.sender).isModuleEnabled(address(this)))
    revert ModuleNotEnabled();
828
829  // 5. #4 owner authorisation (the robust gate). Bind the
    principals to an owner-set+threshold
830  //    signature so a setup helper — which holds no owner
    keys — cannot register. chainId is in
831  //    the EIP-712 domain; no nonce is needed because
    registration is one-shot (`registered[]`
832  //    never clears, so a replay reverts AccountAlreadyReg-
    istered in _registerAccount).
833   if (block.timestamp > deadline) revert DeadlineExpired(dead-
    line, block.timestamp);
834   bytes32 digest = _hashTypedDataV4(keccak256(abi.encode(
835   REGISTER_ACCOUNT_TYPEHASH,
836   msg.sender,
837   permissionSigner,
838   manager,
839   feePolicy,
840   feeAsset,
841   deadline
842  )));
843  // Reverts (GS0xx) unless `ownerSig` satisfies the Safe's
    owner set + threshold.
844   ISafe(msg.sender).checkSignatures(digest, "", ownerSig);
845
846  _registerAccount(msg.sender, permissionSigner, manager,
    feePolicy, feeAsset);
847  }
848
849  /// @dev Shared registration logic for `createAccount` and
    `registerAccount`.
850   function _registerAccount(address account, address permission-
    Signer, address manager, address feePolicy, address feeAsset)
851   internal
852  {
853   if (registered[account]) revert AccountAlreadyRegis-
    tered(account);
854   if (permissionSigner == address(0) || manager == address(0))
    revert ZeroAddress();
855   if (feePolicy != address(0) && !governance.trustedFeePol-
    icy(feePolicy)) revert UntrustedFeePolicy(feePolicy);
856   registered[account] = true;
857   configs[account] = AccountConfig({
858   permissionSigner: permissionSigner,
859   manager:          manager,
860   feePolicy:        feePolicy,
```

```
861   feeAsset:         feeAsset,
862   sessionActive:    true
863  });
864   emit AccountRegistered(account, permissionSigner, manager);
865  }
866
867  /// @notice Rotate the manager (delegated signer) for an
     account, clearing every
868  ///         attached mandate in the same transaction.
869  /// @dev    AUTHORIZATION: MUST be called by the Safe itself
     (`msg.sender == account`).
870  ///         Authorization therefore flows through the Safe's
     own owner threshold — the
871  ///         custody anchor — and replay protection comes from
     the Safe's nonce, so no
872  ///         kernel signature or nonce is needed. This mirrors
     `registerAccount`'s
873  ///         `msg.sender == Safe` trust model. `_requireRegistered`
     ensures only an
874  ///         already-registered account (a genuine Safe proxy,
     vetted at registration)
875  ///         can reach this path, and an account can only rotate
     its own manager.
876  ///
877  ///         MANDATE RESET: A rotated signer must never silently
     inherit authority the
878  ///         owner approved for the old one. Rather than leave
     mandates attached but
879  ///         inert, this clears the permission set outright
     (fail-closed: subsequent
880  ///         dispatches revert with `PermissionNotRegistered`
     until the owner re-approves
881  ///         each mandate via the normal `registerPermission(s)`
     flow — which binds them
882  ///         to the new manager). A `PermissionRevoked` is
     emitted per cleared mandate.
883  ///
884  ///         IN-FLIGHT OPS: `managerNonces`, `batchNonces`,
     and `signerNonces` are all
885  ///         bumped by `NONCE_EPOCH_INCREMENT` so any dispatch
     the old manager pre-signed,
886  ///         and any permission-signer op pre-signed against
     the old epoch, is invalidated —
887  ///         consistent with every other mandate-mutating op.
888  ///
889  ///         PAUSE: Intentionally exempt from `whenNotPaused`
     — losing the agent key is
```

```
890  ///         exactly the kind of incident during which recovery
     must remain possible, and
891  ///         rotation moves no funds. (See `revokeSession`/`re-
     vokePermission`.)
892  /// @param  newManager New address authorised to sign
     dispatches. Must be non-zero and
893  ///                    different from the current manager.
894   function setManager(address newManager) external nonReentrant
     {
895  // W1: reject Safe-fallback-relayed calls. A direct call
     is exactly selector + 1 static arg;
896  // a fallback relay appends the caller's 20 bytes (see
     SET_MANAGER_CALLDATA_LEN).
897   if (msg.data.length != SET_MANAGER_CALLDATA_LEN) revert
     UnexpectedCalldataLength();
898   address account = msg.sender;
899  _requireRegistered(account);
900   if (newManager == address(0)) revert ZeroAddress();
901   address oldManager = configs[account].manager;
902   if (newManager == oldManager) revert ManagerUnchanged();
903
904   configs[account].manager = newManager;
905  _clearPermissions(account);
906   managerNonces[account] += NONCE_EPOCH_INCREMENT;
907   batchNonces[account]   += NONCE_EPOCH_INCREMENT;
908   signerNonces[account]  += NONCE_EPOCH_INCREMENT;
909
910   emit ManagerChanged(account, oldManager, newManager);
911  }
912
913  /// @notice The address currently authorised to sign dispatches
     for an account.
914  /// @param  account The Safe account to query.
915  /// @return         The account's manager (delegated signer);
     address(0) if unregistered.
916   function getManager(address account) external view returns
     (address) {
917   return configs[account].manager;
918  }
919
920  //
     ----------------------------------------------------------------------
921  // 2. Permission registry
922  //
     ----------------------------------------------------------------------
923
924  /// @notice Register a single permission for an account.
```

```
925  ///         Requires a permissionSigner EIP-712 signature and
     an ETH registration fee.
926  /// @dev    PERMISSION TRUST: The kernel binds authorization
     to a permission address only.
927  ///         If a registered permission is upgradeable or uses
     a proxy, a change to its
928  ///         implementation requires no new kernel signature.
     Operators are responsible for
929  ///         registering only non-upgradeable or audited
     permission contracts.
930  ///         See Sail Protocol whitepaper §8.2.
931  /// @param  account    The registered Safe account.
932  /// @param  permission Address of the permission contract to
     register.
933  /// @param  deadline   Unix timestamp after which the signature
     is invalid.
934  /// @param  sig        EIP-712 signature over RegisterPermission
     struct by permissionSigner.
935   function registerPermission(address account, address permis-
     sion, uint256 deadline, bytes calldata sig)
936   external
937   payable
938   nonReentrant
939   whenNotPaused
940  {
941  _requireRegistered(account);
942   if (block.timestamp > deadline) revert DeadlineExpired(dead-
     line, block.timestamp);
943   if (permission == address(0)) revert ZeroAddress();
944   if (permission.code.length == 0) revert NotAContract(per-
     mission);
945   if (_permissionIndex[account][permission] != 0) revert
     PermissionAlreadyRegistered(permission);
946   uint256 limit = governance.maxPermissionsPerAccount();
947   if (_permissions[account].length >= limit)
948   revert TooManyPermissions(account, limit);
949
950   uint256 nonce = signerNonces[account];
951  _verifySignerSig(
952   account,
953   keccak256(abi.encode(REGISTER_PERMISSION_TYPEHASH,
     account, permission, nonce, deadline)),
954   sig
955  );
956   signerNonces[account] = nonce + 1;
957
958   uint256 fee = _calcPermissionFee();
```

```
959   if (msg.value < fee) revert InsufficientFee(fee, msg.value);
960
961   _permissions[account].push(permission);
962   _permissionIndex[account][permission] = _permissions[ac-
      count].length; // stored as index + 1
963
964   _collectRegistrationFee(fee);
965   emit PermissionRegistered(account, permission);
966   }
967
968   /// @notice Revoke a single permission from an account.
969   ///         Requires a permissionSigner EIP-712 signature.
970   /// @dev    Intentionally exempt from whenNotPaused — users
      must be able to revoke
971   ///         permissions even during a protocol pause to reduce
      their exposure.
972   /// @param  account    The registered Safe account.
973   /// @param  permission Address of the permission contract to
      revoke.
974   /// @param  deadline   Unix timestamp after which the signature
      is invalid.
975   /// @param  sig        EIP-712 signature over RevokePermission
      struct by permissionSigner.
976   function revokePermission(address account, address permission,
      uint256 deadline, bytes calldata sig) external nonReentrant {
977   _requireRegistered(account);
978   if (block.timestamp > deadline) revert DeadlineExpired(dead-
      line, block.timestamp);
979   uint256 nonce = signerNonces[account];
980   _verifySignerSig(
981   account,
982   keccak256(abi.encode(REVOKE_PERMISSION_TYPEHASH,
      account, permission, nonce, deadline)),
983   sig
984   );
985   signerNonces[account] = nonce + 1;
986   _removePermission(account, permission);
987   registrationEpoch[account][permission] += 1;
988   managerNonces[account] += NONCE_EPOCH_INCREMENT;
989   batchNonces[account]   += NONCE_EPOCH_INCREMENT;
990   emit PermissionRevoked(account, permission);
991   }
992
993   /// @notice Atomically replace one permission with another in
      a single signed operation.
994   ///         Requires a permissionSigner EIP-712 signature and
      an ETH registration fee.
```

```
995  /// @dev    PERMISSION TRUST: The kernel binds authorization
     to a permission address only.
996  ///         If a registered permission is upgradeable or uses
     a proxy, a change to its
997  ///         implementation requires no new kernel signature.
     Operators are responsible for
998  ///         registering only non-upgradeable or audited
     permission contracts.
999  ///         See Sail Protocol whitepaper §8.2.
1000 /// @param  account        The registered Safe account.
1001 /// @param  oldPermission  Permission to remove.
1002 /// @param  newPermission  Permission to add in its place.
1003 /// @param  deadline       Unix timestamp after which the
     signature is invalid.
1004 /// @param  sig          EIP-712 signature over ReplacePermission
     struct by permissionSigner.
1005  function replacePermission(
1006  address account,
1007  address oldPermission,
1008  address newPermission,
1009  uint256 deadline,
1010  bytes calldata sig
1011 ) external payable nonReentrant whenNotPaused {
1012 _requireRegistered(account);
1013  if (block.timestamp > deadline) revert DeadlineExpired(dead-
     line, block.timestamp);
1014  if (newPermission == address(0)) revert ZeroAddress();
1015  if (newPermission.code.length == 0) revert NotAContract(new-
     Permission);
1016  if (_permissionIndex[account][newPermission] != 0) revert
     PermissionAlreadyRegistered(newPermission);
1017
1018  uint256 nonce = signerNonces[account];
1019 _verifySignerSig(
1020  account,
1021  keccak256(abi.encode(REPLACE_PERMISSION_TYPEHASH,
     account, oldPermission, newPermission, nonce, deadline)),
1022  sig
1023 );
1024  signerNonces[account] = nonce + 1;
1025
1026  uint256 idx = _permissionIndex[account][oldPermission];
1027  if (idx == 0) revert PermissionNotRegistered(oldPermission);
1028
1029  uint256 fee = _calcPermissionFee();
1030  if (msg.value < fee) revert InsufficientFee(fee, msg.value);
1031
```

```
1032 _permissions[account][idx - 1] = newPermission;
1033  delete _permissionIndex[account][oldPermission];
1034 _permissionIndex[account][newPermission] = idx;
1035 // Bump the REMOVED side only. newPermission is register-like
     (its config is applied just
1036 // before this call in the bundled flow); bumping it would
     strand that fresh config.
1037  registrationEpoch[account][oldPermission] += 1;
1038
1039 _collectRegistrationFee(fee);
1040  managerNonces[account] += NONCE_EPOCH_INCREMENT;
1041  batchNonces[account]   += NONCE_EPOCH_INCREMENT;
1042  emit PermissionReplaced(account, oldPermission, newPer-
     mission);
1043 }
1044
1045 /// @notice Atomically replace N permissions with N new ones
     in a single signed operation.
1046 ///         Eliminates the front-running overlap window that
     arises when separate
1047 ///         `registerPermissions` + `revokePermissions` calls
     are used for NN migrations.
1048 ///         Requires a permissionSigner EIP-712 signature and
     a fee per swap.
1049 /// @dev    PERMISSION TRUST: The kernel binds authorization
     to a permission address only.
1050 ///         If a registered permission is upgradeable or uses
     a proxy, a change to its
1051 ///         implementation requires no new kernel signature.
     Operators are responsible for
1052 ///         registering only non-upgradeable or audited
     permission contracts.
1053 ///         See Sail Protocol whitepaper §8.2.
1054 /// @param  account         The registered Safe account.
1055 /// @param  oldPermissions  Permissions to remove (parallel
     to `newPermissions`).
1056 /// @param  newPermissions  Permissions to add in their place.
1057 /// @param  deadline        Unix timestamp after which the
     signature is invalid.
1058 /// @param  sig             EIP-712 signature over
     ReplacePermissions struct by permissionSigner.
1059  function replacePermissions(
1060  address account,
1061  address[] calldata oldPermissions,
1062  address[] calldata newPermissions,
1063  uint256 deadline,
1064  bytes calldata sig
```

```
) external payable nonReentrant whenNotPaused {
 if (oldPermissions.length != newPermissions.length) revert ArrayLengthMismatch();
 if (oldPermissions.length == 0) {
 if (msg.value > 0) _collectRegistrationFee(0); // refunds full msg.value to caller
 return;
}
_requireRegistered(account);
 if (block.timestamp > deadline) revert DeadlineExpired(deadline, block.timestamp);

 uint256 nonce = signerNonces[account];
_verifySignerSig(
 account,
 keccak256(abi.encode(
 REPLACE_PERMISSIONS_TYPEHASH,
 account,
_hashAddressArray(oldPermissions),
_hashAddressArray(newPermissions),
 nonce,
 deadline
)),
 sig
);
 signerNonces[account] = nonce + 1;

 uint256 fee = _calcPermissionFee() * oldPermissions.length;
 if (msg.value < fee) revert InsufficientFee(fee, msg.value);

 for (uint256 i; i < oldPermissions.length; i++) {
 address oldPerm = oldPermissions[i];
 address newPerm = newPermissions[i];
 if (newPerm == address(0)) revert ZeroAddress();
 if (newPerm.code.length == 0) revert NotAContract(newPerm);
 uint256 idx = _permissionIndex[account][oldPerm];
 if (idx == 0) revert PermissionNotRegistered(oldPerm);
 if (_permissionIndex[account][newPerm] != 0) revert PermissionAlreadyRegistered(newPerm);
_permissions[account][idx - 1] = newPerm;
 delete _permissionIndex[account][oldPerm];
_permissionIndex[account][newPerm] = idx;
 registrationEpoch[account][oldPerm] += 1; // bump the removed side only (see replacePermission)
 emit PermissionReplaced(account, oldPerm, newPerm);
}
```

```
1106
1107 _collectRegistrationFee(fee);
1108  managerNonces[account] += NONCE_EPOCH_INCREMENT;
1109  batchNonces[account]   += NONCE_EPOCH_INCREMENT;
1110 }
1111
1112 /// @notice Suspend the manager session for an account. All
     `dispatch` calls will
1113 ///         revert with `SessionInactive` until `activateSession`
     is called.
1114 /// @param  account  The registered Safe account.
1115 /// @param  deadline Unix timestamp after which the signature
     is invalid.
1116 /// @param  sig      EIP-712 signature over RevokeSession
     struct by permissionSigner.
1117  function revokeSession(address account, uint256 deadline,
     bytes calldata sig) external nonReentrant {
1118 _requireRegistered(account);
1119  if (block.timestamp > deadline) revert DeadlineExpired(dead-
     line, block.timestamp);
1120  uint256 nonce = signerNonces[account];
1121 _verifySignerSig(
1122  account,
1123  keccak256(abi.encode(REVOKE_SESSION_TYPEHASH, account,
     nonce, deadline)),
1124  sig
1125 );
1126  signerNonces[account] = nonce + 1;
1127  configs[account].sessionActive = false;
1128  managerNonces[account] += NONCE_EPOCH_INCREMENT;
1129  batchNonces[account]   += NONCE_EPOCH_INCREMENT;
1130  emit SessionRevoked(account);
1131 }
1132
1133 /// @notice Re-activate a previously suspended session.
     Requires a fresh permissionSigner
1134 ///         signature to prove the key is still under the
     operator's control.
1135 /// @param  account  The registered Safe account.
1136 /// @param  deadline Unix timestamp after which the signature
     is invalid.
1137 /// @param  sig      EIP-712 signature over ActivateSession
     struct by permissionSigner.
1138  function activateSession(address account, uint256 deadline,
     bytes calldata sig) external nonReentrant {
1139 _requireRegistered(account);
1140
```

```
     if (block.timestamp > deadline) revert DeadlineExpired(dead-
    line, block.timestamp);
1141  uint256 nonce = signerNonces[account];
1142 _verifySignerSig(
1143  account,
1144  keccak256(abi.encode(ACTIVATE_SESSION_TYPEHASH, account,
    nonce, deadline)),
1145  sig
1146 );
1147  signerNonces[account] = nonce + 1;
1148  configs[account].sessionActive = true;
1149 // Rotate manager/batch nonce epochs on reactivation so
    any dispatch the manager
1150 // pre-signed while the session was suspended (current
    epoch) cannot execute once
1151 // the session is live again. Mirrors revokeSession's
    epoch bump — a session cycle
1152 // is a clean kill switch for outstanding signatures in
    both directions.
1153  managerNonces[account] += NONCE_EPOCH_INCREMENT;
1154  batchNonces[account]   += NONCE_EPOCH_INCREMENT;
1155  emit SessionActivated(account);
1156 }
1157
1158 /// @notice Replace the fee policy for an account. Requires a
    permissionSigner signature.
1159 ///         Setting `newFeePolicy = address(0)` clears the
    policy and blocks fee collection.
1160 /// @param  account      The registered Safe account.
1161 /// @param  newFeePolicy New fee policy contract address;
    address(0) = no fee policy.
1162 /// @param  feeAsset     Canonical fee settlement token for
    the new policy; address(0) = native ETH.
1163 /// @param  deadline     Unix timestamp after which the
    signature is invalid.
1164 /// @param  sig          EIP-712 signature over SetFeePolicy
    struct by permissionSigner.
1165  function setFeePolicy(address account, address newFeePolicy,
    address feeAsset, uint256 deadline, bytes calldata sig) external
    nonReentrant {
1166 _requireRegistered(account);
1167 // Allow clearing (newFeePolicy == 0) during pause so
    permissionSigner can disarm a compromised policy
1168 // before the pause lifts and collectFees becomes callable
    again.  Block non-zero updates while paused.
1169  if (governance.isPaused() && newFeePolicy != address(0))
    revert ProtocolPaused();
```

```
1170  if (newFeePolicy != address(0) && !governance.trustedFeeP-
     olicy(newFeePolicy)) revert UntrustedFeePolicy(newFeePolicy);
1171  if (block.timestamp > deadline) revert DeadlineExpired(dead-
     line, block.timestamp);
1172  uint256 nonce = signerNonces[account];
1173 _verifySignerSig(
1174  account,
1175  keccak256(abi.encode(SET_FEE_POLICY_TYPEHASH, account,
     newFeePolicy, feeAsset, nonce, deadline)),
1176  sig
1177 );
1178  signerNonces[account] = nonce + 1;
1179
1180 // Pin a policy instance to the single fee asset it is
     used with for this account, so the
1181 // policy's persisted (per-account) high-water mark can
     never be mixed across denominations.
1182 // First lazily bind the currently configured policy (e.g.
     one set at registration) to its
1183 // current asset, so a later swap to a different asset on
     the SAME instance is caught even
1184 // on the first setFeePolicy call.
1185  address currentPolicy = configs[account].feePolicy;
1186  if (currentPolicy != address(0)) {
1187  PolicyAssetBinding storage current = _policyAssetBind-
     ing[account][currentPolicy];
1188  if (!current.bound) { current.bound = true; current.asset
     = configs[account].feeAsset; }
1189 }
1190 // Then enforce the binding for the incoming policy: bind
     on first use, else require a match.
1191  if (newFeePolicy != address(0)) {
1192  PolicyAssetBinding storage binding = _policyAssetBind-
     ing[account][newFeePolicy];
1193  if (!binding.bound) { binding.bound = true; binding.asset
     = feeAsset; }
1194  else if (binding.asset != feeAsset) revert
     FeePolicyAssetMismatch(newFeePolicy, binding.asset, feeAsset);
1195 }
1196
1197  configs[account].feePolicy = newFeePolicy;
1198  configs[account].feeAsset  = (newFeePolicy == address(0))
     ? address(0) : feeAsset;
1199  emit FeePolicyUpdated(account, newFeePolicy);
1200
1201 // Lifecycle hook (after all kernel state writes — CEI):
     tell the new policy it has been
```

```
1202 // attached so it can re-anchor its per-account accounting. This closes the detach/reattach
1203 // over-collection (the same instance would otherwise bill management fees over the dormant
1204 // interval). The call passes ONLY `account` — the kernel never learns NAV/valuation; that
1205 // stays the policy/manager's responsibility. The new policy is governance-trusted (checked
1206 // above) and setFeePolicy is nonReentrant, so the external call is safe here.
1207  if (newFeePolicy != address(0)) IFeePolicy(newFeePoli-cy).onAttach(account);
1208 }
1209
1210 /// @notice Register multiple permissions atomically. One signer nonce is consumed for the
1211 ///         entire batch; the total fee equals the sum of individual permission fees.
1212 ///         Reverts atomically if any permission in the batch is already registered,
1213 ///         the cap would be exceeded, the fee is insufficient, or the signature is invalid.
1214 /// @dev    The `permissions` array is EIP-712 encoded as keccak256 of the ABI-packed
1215 ///         zero-padded addresses (see `_hashAddressArray`). Empty arrays are a no-op
1216 ///         and do not consume a nonce.
1217 /// @dev    PERMISSION TRUST: The kernel binds authorization to a permission address only.
1218 ///         If a registered permission is upgradeable or uses a proxy, a change to its
1219 ///         implementation requires no new kernel signature. Operators are responsible for
1220 ///         registering only non-upgradeable or audited permission contracts.
1221 ///         See Sail Protocol whitepaper §8.2.
1222 /// @param  account     The registered Safe account.
1223 /// @param  permissions Addresses of permission contracts to register.
1224 /// @param  deadline    Unix timestamp after which the signature is invalid.
1225 /// @param  sig         EIP-712 signature over RegisterPermissions struct by permissionSigner.
1226  function registerPermissions(
1227  address account,
1228  address[] calldata permissions,
1229  uint256 deadline,
```

```
1230  bytes calldata sig
1231 ) external payable nonReentrant whenNotPaused {
1232  if (permissions.length == 0) {
1233  if (msg.value > 0) _collectRegistrationFee(0); //
     refunds full msg.value to caller
1234  return;
1235 }
1236 _requireRegistered(account);
1237  if (block.timestamp > deadline) revert DeadlineExpired(dead-
     line, block.timestamp);
1238
1239 // Enforce the cap before consuming the nonce to avoid
     nonce burns on revert.
1240  uint256 limit = governance.maxPermissionsPerAccount();
1241  if (_permissions[account].length + permissions.length >
     limit)
1242  revert TooManyPermissions(account, limit);
1243
1244  uint256 nonce = signerNonces[account];
1245 _verifySignerSig(
1246  account,
1247  keccak256(abi.encode(
1248  REGISTER_PERMISSIONS_TYPEHASH,
1249  account,
1250 _hashAddressArray(permissions),
1251  nonce,
1252  deadline
1253 )),
1254  sig
1255 );
1256  signerNonces[account] = nonce + 1;
1257
1258 // Compute total fee before any state changes
1259  uint256 totalFee = _calcPermissionFee() * permis-
     sions.length;
1260  if (msg.value < totalFee) revert InsufficientFee(totalFee,
     msg.value);
1261
1262 // Add all permissions atomically — reverts if any duplicate
     or invalid address found
1263  for (uint256 i; i < permissions.length; i++) {
1264  address perm = permissions[i];
1265  if (perm == address(0)) revert ZeroAddress();
1266  if (perm.code.length == 0) revert NotAContract(perm);
1267  if (_permissionIndex[account][perm] != 0) revert
     PermissionAlreadyRegistered(perm);
1268 _permissions[account].push(perm);
```

```
1269 _permissionIndex[account][perm] = _permissions[ac-
     count].length;
1270  emit PermissionRegistered(account, perm);
1271 }
1272
1273 _collectRegistrationFee(totalFee);
1274 }
1275
1276 /// @notice Revoke multiple permissions atomically. One signer
     nonce is consumed for
1277 ///         the entire batch; no ETH fee is required.
1278 ///         Empty arrays are a no-op and do not consume a
     nonce.
1279 /// @dev    Intentionally exempt from whenNotPaused — users
     must be able to revoke
1280 ///         permissions even during a protocol pause to reduce
     their exposure.
1281 /// @param  account     The registered Safe account.
1282 /// @param  permissions Addresses of permission contracts to
     revoke.
1283 /// @param  deadline    Unix timestamp after which the signature
     is invalid.
1284 /// @param  sig         EIP-712 signature over RevokePermissions
     struct by permissionSigner.
1285  function revokePermissions(
1286  address account,
1287  address[] calldata permissions,
1288  uint256 deadline,
1289  bytes calldata sig
1290 ) external nonReentrant {
1291  if (permissions.length == 0) return;
1292 _requireRegistered(account);
1293  if (block.timestamp > deadline) revert DeadlineExpired(dead-
     line, block.timestamp);
1294
1295  uint256 nonce = signerNonces[account];
1296 _verifySignerSig(
1297  account,
1298  keccak256(abi.encode(
1299  REVOKE_PERMISSIONS_TYPEHASH,
1300  account,
1301 _hashAddressArray(permissions),
1302  nonce,
1303  deadline
1304 )),
1305  sig
1306 );
```

```
1307  signerNonces[account] = nonce + 1;
1308
1309  for (uint256 i; i < permissions.length; i++) {
1310 _removePermission(account, permissions[i]);
1311  registrationEpoch[account][permissions[i]] += 1;
1312  emit PermissionRevoked(account, permissions[i]);
1313 }
1314  managerNonces[account] += NONCE_EPOCH_INCREMENT;
1315  batchNonces[account]   += NONCE_EPOCH_INCREMENT;
1316 }
1317
1318 /// @notice Return the full list of registered permission
     addresses for an account.
1319 /// @param  account The Safe account to query.
1320 /// @return         Array of registered permission contract
     addresses.
1321  function getPermissions(address account) external view returns
     (address[] memory) {
1322  return _permissions[account];
1323 }
1324
1325 /// @notice Check whether a specific permission is registered
     for an account.
1326 /// @param  account    The Safe account to query.
1327 /// @param  permission The permission contract address to look
     up.
1328 /// @return            True if the permission is currently
     registered.
1329  function isPermissionRegistered(address account, address
     permission) external view returns (bool) {
1330  return _permissionIndex[account][permission] != 0;
1331 }
1332
1333 //
     ------------------------------------------------------------------------
1334 // 3. Manager dispatch
1335 //
     ------------------------------------------------------------------------
1336
1337 /// @notice Verify a manager signature, evaluate the named
     permission, and
1338 ///         execute the transaction via the Safe module
     interface.
1339 ///
1340 // SELECTIVE authorization semantics: the manager signature
     names one
1341
```

```
    // registered permission, and only that permission evaluates
    the call.
1342 // Changed from the prior conjunctive (AND) model where all
    registered
1343 // permissions had to approve. The new model enables
    multi-permission SMAs
1344 // where unrelated permissions (e.g., Uniswap, Aave, Transfer)
    coexist
1345 // on one account without falsely denying each other's calls.
    Layered
1346 // defense via permission composition is not supported here —
    a separate
1347 // guard mechanism may be added later if needed.
1348 ///
1349 /// @dev    The manager nonce is incremented before external
    interaction; however, any revert
1350 ///         will roll back this write, so failed attempts do
    not consume the nonce. To achieve
1351 ///         strict single-use semantics, convert post-nonce
    failures to non-reverting denials
1352 ///         or add an explicit cancel-nonce operation. The
    permission is evaluated via
1353 ///         staticcall with PERMISSION_GAS_CAP gas; a revert
    or gas exhaustion inside a
1354 ///         permission is treated as denial. The named permission
    must be pre-registered
1355 ///         on the account; this is the permissionSigner's
    trust anchor.
1356 /// @dev    CONFIG SEQUENCING: Dispatch authorizes by permission
    address only, not by
1357 ///         configuration state. To atomically tighten a
    permission's rules, use
1358 ///         replacePermission rather than reconfiguring in
    place. An in-place configure
1359 ///         is subject to a front-run window while the
    transaction is pending.
1360 /// @param  account     The registered Safe account to execute
    through.
1361 /// @param  permission  The registered permission that must
    authorise this call.
1362 /// @param  target      Call target address.
1363 /// @param  value       Native ETH to forward with the call
    (wei).
1364 /// @param  data        Calldata for the target call.
1365 /// @param  managerSig  EIP-712 signature over Dispatch struct
    by the account's manager.
1366
```

```
    /// @param  deadline    Unix timestamp after which the signature
    is invalid.
1367  function dispatch(
1368  address account,
1369  address permission,
1370  address target,
1371  uint256 value,
1372  bytes calldata data,
1373  bytes calldata managerSig,
1374  uint256 deadline
1375 ) external nonReentrant whenNotPaused {
1376 _requireRegistered(account);
1377
1378  AccountConfig storage cfg = configs[account];
1379  if (!cfg.sessionActive) revert SessionInactive(account);
1380 // Deadline is a user-supplied expiry, intentionally
    compared against block.timestamp.
1381 // Matches the pattern used by every other deadline-checking
    entry point in this contract.
1382 // forge-lint: disable-next-line(block-timestamp)
1383  if (block.timestamp > deadline) revert DeadlineExpired(dead-
    line, block.timestamp);
1384
1385 // O(1) membership check — revert if the named permission
    is not registered.
1386  if (_permissionIndex[account][permission] == 0) revert
    PermissionNotRegistered(permission);
1387
1388  uint256 nonce   = managerNonces[account];
1389  bytes32 dataHash = keccak256(data);
1390  bytes32 digest   = _hashTypedDataV4(keccak256(abi.encode(
1391  DISPATCH_TYPEHASH,
1392  account,
1393  permission,
1394  target,
1395  value,
1396  dataHash,
1397  nonce,
1398  deadline
1399 )));
1400  if (!_recoverOrERC1271(cfg.manager, digest, managerSig))
    revert InvalidManagerSignature();
1401  managerNonces[account] = nonce + 1;
1402
1403 // Prevent module-triggered self-calls: a call targeting
    the Safe itself satisfies
1404
```

```
    // Safe's onlySelf guard, enabling enableModule/set-
    Guard/owner changes without permission.
1405  if (target == account) revert AccountSelfTarget();
1406 // Parity with dispatchBatch's per-subcall guards: a single
    dispatch may not target the
1407 // zero address or the kernel itself. Re-entry is already
    blocked by nonReentrant; rejecting
1408 // these targets closes the class at the dispatch boundary.
    Single dispatch has no subcall
1409 // index, so it reuses the batch errors with index 0.
1410  if (target == address(0))    revert BatchZeroTarget(0);
1411  if (target == address(this)) revert KernelSelfTarget(0);
1412
1413  bytes4 sel = data.length >= 4 ? bytes4(data[:4]) : bytes4(0);
1414  Context memory ctx = Context({
1415  account:        account,
1416  manager:        cfg.manager,
1417  submitter:      msg.sender,
1418  target:         target,
1419  selector:       sel,
1420  value:          value,
1421  blockTimestamp: block.timestamp,
1422  blockNumber:    block.number,
1423  configEpoch:    registrationEpoch[account][permission]
1424 });
1425  if (!_evaluatePermission(permission, data, ctx)) revert
    PermissionDenied(permission);
1426
1427  if (!ISafe(account).execTransactionFromModule(target,
    value, data, 0)) revert SafeExecutionFailed();
1428
1429  emit Dispatched(account, permission, target, sel, value);
1430 }
1431
1432 /// @dev Invoke a single permission via staticcall with the
    configured gas cap.
1433 ///      Returns false on revert, out-of-gas, or malformed
    return data.
1434  function _evaluatePermission(address permission, bytes
    calldata data, Context memory ctx)
1435  internal
1436  view
1437  returns (bool)
1438 {
1439  bytes memory callData = abi.encodeCall(IPermission.evaluate,
    (data, ctx));
1440
```

```
    (bool success, bytes memory ret) = permission.static-
    call{gas: PERMISSION_GAS_CAP}(callData);
1441  if (!success || ret.length < 32) return false;
1442 // Decode as uint256 to avoid abi.decode revert on
    non-canonical bool words (e.g. 0x02).
1443  return abi.decode(ret, (uint256)) == 1;
1444 }
1445
1446 //
    ------------------------------------------------------------------------
1447 // 3b. Batch dispatch
1448 //
    ------------------------------------------------------------------------
1449
1450 /// @notice Execute a sequence of Safe module calls as a single
    atomic transaction,
1451 ///         gated by ONE named batch-aware permission.
1452 ///
1453 /// @dev    DIVERGENCE FROM `dispatch`: only the named
    `permission` is evaluated.
1454 ///         Other IPermissions registered on the account are
    NOT consulted during
1455 ///         batch dispatch. The batch-aware permission owns
    full responsibility for
1456 ///         validating every subcall and any cross-call
    invariants (e.g. matching
1457 ///         approve/consume amounts, mandatory reset-to-zero
    cleanup).
1458 ///
1459 ///         The `permission` MUST still be registered on the
    account — registration
1460 ///         is the permissionSigner's trust anchor. Choosing
    it for a batch then
1461 ///         requires only the manager's signature.
1462 ///
1463 ///         Atomicity: if any subcall returns false from
    execTransactionFromModule,
1464 ///         the entire dispatch reverts, rolling back all
    prior subcalls in the batch.
1465 ///
1466 ///         Operation type: every subcall executes with
    `operation = 0` (CALL).
1467 ///         DELEGATECALL is never used. The kernel does not
    depend on Safe MultiSend.
1468 ///
1469 ///         Self-target guard: no subcall may target this
    kernel. This is a defensive
```

```
1470 ///         measure — re-entry through public functions is already blocked by
1471 ///         nonReentrant, but rejecting kernel-targeted subcalls eliminates the
1472 ///         entire class of self-targeted attacks at the dispatch boundary.
1473 ///
1474 /// @dev    CONFIG SEQUENCING: Dispatch authorizes by permission address only, not by
1475 ///         configuration state. To atomically tighten a permission's rules, use
1476 ///         replacePermission rather than reconfiguring in place. An in-place configure
1477 ///         is subject to a front-run window while the transaction is pending.
1478 ///
1479 /// @param  account     The registered Safe account to execute through.
1480 /// @param  permission  The batch-aware permission that authorises this batch.
1481 ///                     Must be registered on the account and implement IBatchPermission.
1482 /// @param  calls       Ordered subcall sequence (length 1..MAX_BATCH_LENGTH).
1483 /// @param  managerSig  EIP-712 signature over DispatchBatch struct by the account's manager.
1484 /// @param  deadline    Unix timestamp after which the signature is invalid.
1485  function dispatchBatch(
1486  address account,
1487  address permission,
1488  Call[] calldata calls,
1489  bytes calldata managerSig,
1490  uint256 deadline
1491 ) external nonReentrant whenNotPaused {
1492 // 1. Account / session / deadline validation
1493 _requireRegistered(account);
1494  AccountConfig storage cfg = configs[account];
1495  if (!cfg.sessionActive) revert SessionInactive(account);
1496  if (block.timestamp > deadline) revert DeadlineExpired(deadline, block.timestamp);
1497
1498 // 2. Batch length bounds
1499  uint256 len = calls.length;
1500  if (len == 0) revert EmptyBatch();
1501  if (len > MAX_BATCH_LENGTH) revert BatchTooLong(len);
1502
```

```
1503 // 3. Permission must be registered for this account
1504  if (_permissionIndex[account][permission] == 0) revert
     PermissionNotRegistered(permission);
1505
1506 // 4. Verify manager signature over the canonical callsHash
1507  bytes32 callsHash = keccak256(abi.encode(calls));
1508  uint256 nonce     = batchNonces[account];
1509  bytes32 digest    = _hashTypedDataV4(keccak256(abi.encode(
1510  DISPATCH_BATCH_TYPEHASH,
1511  account,
1512  permission,
1513  callsHash,
1514  nonce,
1515  deadline
1516 )));
1517  if (!_recoverOrERC1271(cfg.manager, digest, managerSig))
     revert InvalidManagerSignature();
1518 // NOTE: Nonce is incremented here, but any subsequent
     revert will roll back this write;
1519 // failed attempts do not consume the nonce. To enforce
     single-use semantics, convert
1520 // post-nonce failures to non-reverting denials or add an
     explicit cancel-nonce operation.
1521  batchNonces[account] = nonce + 1;
1522
1523 // 5. Permission must implement IBatchPermission. Detection
     via staticcall
1524 //    is stricter than try/catch — guarantees no state
     mutation regardless of
1525 //    what the target claims about its function modifiers.
1526 {
1527 (bool detOk, bytes memory detRet) = permission.stat-
     iccall{gas: BATCH_DETECT_GAS_CAP}(
1528  abi.encodeWithSelector(IBatchPermission.isBatch-
     Permission.selector)
1529 );
1530  if (!detOk || detRet.length < 32 || abi.decode(detRet,
     (uint256)) != 1) {
1531  revert PermissionNotBatchAware(permission);
1532 }
1533 }
1534
1535 // 6. Pre-flight: no subcall may target the zero address
     or the kernel itself
1536  for (uint256 i = 0; i < len;) {
1537  if (calls[i].target == address(0))    revert
     BatchZeroTarget(i);
```

```
1538  if (calls[i].target == address(this)) revert
      KernelSelfTarget(i);
1539  if (calls[i].target == account)       revert
      AccountSelfTarget();
1540  unchecked { ++i; }
1541 }
1542
1543 // 7. Build BatchContext and evaluate the batch permission
1544  BatchContext memory ctx = BatchContext({
1545  account:        account,
1546  manager:        cfg.manager,
1547  submitter:      msg.sender,
1548  permission:     permission,
1549  batchHash:      callsHash,
1550  blockTimestamp: block.timestamp,
1551  blockNumber:    block.number,
1552  configEpoch:    registrationEpoch[account][permission]
1553 });
1554  if (!_evaluateBatchPermission(permission, calls, ctx))
      revert BatchPermissionDenied();
1555
1556 // 8. Execute each subcall in order via Safe module CALL
      (operation = 0).
1557 //    Any failure reverts the entire transaction, rolling
      back earlier subcalls.
1558  for (uint256 i = 0; i < len;) {
1559  Call calldata c = calls[i];
1560  bool ok = ISafe(account).execTransactionFromMod-
      ule(c.target, c.value, c.data, 0);
1561  if (!ok) revert BatchSubcallFailed(i, c.target);
1562  unchecked { ++i; }
1563 }
1564
1565  emit BatchDispatched(account, permission, callsHash, len);
1566 }
1567
1568 /// @dev Invoke `evaluateBatch` on the named permission via
      staticcall with the
1569 ///      batch gas cap. Returns false on revert, OOG, or
      malformed return data.
1570 ///      Pattern mirrors `_evaluatePermission` for consistency.
1571  function _evaluateBatchPermission(
1572  address permission,
1573  Call[] calldata calls,
1574  BatchContext memory ctx
1575 ) internal view returns (bool) {
1576
```

```
     bytes memory callData = abi.encodeCall(IBatchPermis-
    sion.evaluateBatch, (calls, ctx));
1577 (bool success, bytes memory ret) = permission.static-
    call{gas: BATCH_EVAL_GAS_CAP}(callData);
1578  if (!success || ret.length < 32) return false;
1579  return abi.decode(ret, (uint256)) == 1;
1580 }
1581
1582 //
    ------------------------------------------------------------------------
1583 // 3c. Permission introspection views
1584 //
    ------------------------------------------------------------------------
1585
1586 /// @notice Return the full permission list for an account
    with enriched metadata.
1587 /// @dev    Reads IPermissionIntrospection and IBatchPermis-
    sion.isBatchPermission()
1588 ///         on each registered permission via try/catch. A
    non-implementing permission
1589 ///         returns zero/false for the corresponding fields.
1590 ///
1591 ///         `hasIntrospection` is true when `permissionId()`
    succeeds AND returns
1592 ///         a non-zero value (a zero permissionId indicates a
    non-compliant or unset
1593 ///         implementation; see IPermissionIntrospection.per-
    missionId()).
1594 ///
1595 ///         Intended for off-chain use only. Each permission
    may make up to 3 external
1596 ///         view calls; call with a generous gas limit on
    accounts with many permissions.
1597 ///         MUST NOT be called from dispatch — use
    isPermissionRegistered() for O(1) checks.
1598 ///
1599 /// @dev    External calls to user-deployed permission contracts
    are NOT gas-capped here.
1600 ///         Off-chain consumers (indexers, dashboards) should
    apply their own gas limit or
1601 ///         timeout when calling this function over RPC; a
    buggy or malicious permission can
1602 ///         consume large amounts of gas or return large data.
    The kernel omits a cap because
1603 ///         this is a view with no on-chain impact; the
    dispatch-path already gas-caps each
1604
```

```
///         permission evaluation via the protocol's per-permission gas isolation guarantee.
/// @param  account The Safe account to query.
/// @return infos   Array of PermissionInfo, one per registered permission, in registration order.
 function getPermissionsWithInfo(address account) external view returns (PermissionInfo[] memory infos) {
 address[] storage perms = _permissions[account];
 uint256 len = perms.length;
 infos = new PermissionInfo[](len);
 for (uint256 i; i < len; i++) {
 address perm = perms[i];
 infos[i].permission = perm;
 try IBatchPermission(perm).isBatchPermission() returns (bool b) {
 infos[i].isBatch = b;
} catch {}
 try IPermissionIntrospection(perm).permissionId() returns (bytes32 pid) {
 if (pid != bytes32(0)) {
 infos[i].hasIntrospection = true;
 infos[i].permissionId = pid;
 try IPermissionIntrospection(perm).permission-Version() returns (bytes32 pv) {
 infos[i].permissionVersion = pv;
} catch {}
}
} catch {}
}
}

/// @notice Simulate whether a batch dispatch would be approved without executing it.
/// @dev    Runs the same validation as dispatchBatch EXCEPT signature verification
///         (no sig available in a simulation context) and Safe module execution.
///         Useful for off-chain pre-flight checks (keeper bots, UI previews).
///
///         Return values:
///           approved = true  'batch would pass evaluateBatch
///           approved = false 'batch would be denied; `reason` has a short descriptor
///
///         Does NOT check: session active, deadline, or sig.
///         Callers must verify those separately.
```

```
1640 ///         DOES check: account registration and permission
     registration; returns
1641 ///         (false, "AccountNotRegistered") or (false,
     "PermissionNotRegistered") accordingly.
1642 ///
1643 /// @param  account    The registered Safe account.
1644 /// @param  permission The batch-aware permission to evaluate.
1645 /// @param  calls      The call sequence to evaluate.
1646 /// @return approved   True if the batch permission would
     authorise the call sequence.
1647 /// @return reason     Short denial reason string; empty if
     approved.
1648  function previewBatch(
1649  address account,
1650  address permission,
1651  Call[] calldata calls
1652 ) external view returns (bool approved, string memory reason)
     {
1653  if (calls.length == 0)              return (false,
     "EmptyBatch");
1654  if (calls.length > MAX_BATCH_LENGTH) return (false,
     "BatchTooLong");
1655  if (!registered[account])           return (false,
     "AccountNotRegistered");
1656  if (_permissionIndex[account][permission] == 0) return
     (false, "PermissionNotRegistered");
1657
1658  for (uint256 i; i < calls.length; i++) {
1659  if (calls[i].target == address(0))    return (false,
     "BatchZeroTarget");
1660  if (calls[i].target == address(this)) return (false,
     "KernelSelfTarget");
1661  if (calls[i].target == account)       return (false,
     "AccountSelfTarget");
1662 }
1663
1664 (bool detOk, bytes memory detRet) = permission.static-
     call{gas: BATCH_DETECT_GAS_CAP}(
1665  abi.encodeWithSelector(IBatchPermission.isBatchPer-
     mission.selector)
1666 );
1667  if (!detOk || detRet.length < 32 || abi.decode(detRet,
     (uint256)) != 1) {
1668  return (false, "PermissionNotBatchAware");
1669 }
1670
1671  bytes32 callsHash = keccak256(abi.encode(calls));
```

```
1672  BatchContext memory ctx = BatchContext({
1673  account:        account,
1674  manager:        configs[account].manager,
1675  submitter:      address(0),   // no submitter
     in simulation; permissions enforcing submitter whitelists will
     return false
1676  permission:     permission,
1677  batchHash:      callsHash,
1678  blockTimestamp: block.timestamp,
1679  blockNumber:    block.number,
1680  configEpoch:    registrationEpoch[account][permission]
1681 });
1682
1683  if (!_evaluateBatchPermission(permission, calls, ctx)) {
1684  return (false, "BatchPermissionDenied");
1685 }
1686  return (true, "");
1687 }
1688
1689 //
     ------------------------------------------------------------------------
1690 // 4. Fee accounting
1691 //
     ------------------------------------------------------------------------
1692
1693 /// @notice Collect fees earned by the manager. The kernel
     validates the requested amount
1694 ///         against the registered fee policy and enforces
     the protocol/distributor split.
1695 /// @dev    TRUST ASSUMPTION: `currentNav` is provided by the
     manager and is not verified
1696 ///         on-chain. The fee policy is the sole guard against
     inflated NAV inputs.
1697 ///
1698 /// @dev    DENOMINATION WARNING: `currentNav` and `feeToken`
     must use consistent units.
1699 ///         The fee policy computes maxFee from currentNav —
     if NAV is USD-denominated
1700 ///         but feeToken is WETH, the fee ceiling will be
     wildly incorrect.
1701 ///         Deployers must ensure their fee policy validates
     or denominates in feeToken units.
1702 ///         The actual tokens transferred equal `grossFee` —
     not a function of `currentNav` —
1703 ///         but a dishonest manager could inflate `currentNav`
     to unlock a larger `maxFee`
1704
```

```
    ///         ceiling and then pass a correspondingly large
    `grossFee`. Deployers must use a
1705 ///         fee policy that validates NAV independently if
    the manager is not trusted.
1706 ///
1707 /// @dev Fee collection is atomic: any failed transfer reverts
    the entire call.
1708 ///      ETH mode (feeToken == address(0)) requires all
    recipients to be payable.
1709 ///      To avoid DoS on non-payable recipients, prefer ERC-20
    fee tokens.
1710 /// @param  account    The registered Safe account from which
    fees are collected.
1711 /// @param  grossFee   Requested fee amount. Must not exceed
    the policy's computed maximum.
1712 /// @param  currentNav Current net asset value reported by
    the manager.
1713 /// @param  feeToken   ERC-20 token for fee payment; address(0)
    = native ETH.
1714 /// @dev DENOMINATION WARNING: `grossFee` and `currentNav`
    must be expressed in the same
1715 ///      token units as the fee token (or ETH wei if
    feeToken==address(0)). Mixing
1716 ///      denominations between grossFee and currentNav will
    silently produce incorrect fee
1717 ///      calculations inside the policy.
1718  function collectFees(
1719  address account,
1720  uint256 grossFee,
1721  uint256 currentNav,
1722  address feeToken
1723 ) external nonReentrant whenNotPaused {
1724 // W1: reject Safe-fallback-relayed calls. collectFees
    also accepts msg.sender == account,
1725 // so a fallbackHandler==kernel Safe could otherwise force
    its own fee outflows. A direct
1726 // call is exactly selector + 4 static args (see
    COLLECT_FEES_CALLDATA_LEN).
1727  if (msg.data.length != COLLECT_FEES_CALLDATA_LEN) revert
    UnexpectedCalldataLength();
1728 _requireRegistered(account);
1729  AccountConfig storage cfg = configs[account];
1730  if (!cfg.sessionActive) revert SessionInactive(account);
1731 // Fee collection is a fund-moving operation, so it is
    restricted to the manager or
1732 // the Safe account itself (the latter covers non-forwarding
    ERC-1271 managers, and is
```

```
1733 // the owner-controlled backstop). The permissionSigner
     manages the permission registry
1734 // and never moves funds, so it is intentionally excluded.
1735  if (msg.sender != cfg.manager && msg.sender != account) {
1736  revert NotManager(msg.sender, cfg.manager);
1737 }
1738  address feePolicy = cfg.feePolicy;
1739  if (feePolicy == address(0)) revert FeePolicyNotSet();
1740  if (grossFee == 0) revert ZeroFee();
1741  if (feeToken != cfg.feeAsset) revert FeeTokenMismatch(fee-
     Token, cfg.feeAsset);
1742
1743 // Recipient is always pulled from the policy — prevents
     manager from redirecting fees.
1744  address recipient = IFeePolicy(feePolicy).feeRecipient();
1745  if (recipient == address(0)) revert ZeroAddress();
1746
1747 (uint256 maxFee, address distributor, uint256 distribu-
     torBps) =
1748  IFeePolicy(feePolicy).computeFee(account, currentNav);
1749  if (grossFee > maxFee) revert FeeTooLarge(grossFee, maxFee);
1750  if (distributorBps > 10_000) revert DistributorBp-
     sTooLarge(distributorBps);
1751
1752 // Prevent payout targets from being the kernel itself:
     ETH transfers would revert
1753 // (no receive/fallback) blocking all collections; ERC-20
     transfers would permanently
1754 // trap tokens since the kernel has no sweep mechanism.
1755  if (treasury == address(this) || distributor == address(this)
     || recipient == address(this)) {
1756  revert ZeroAddress();
1757 }
1758
1759  uint256 protocolCut    = Math.mulDiv(grossFee,
     governance.currentProtocolCutBps(), 10_000);
1760  uint256 remainder      = grossFee - protocolCut;
1761  uint256 distributorCut = Math.mulDiv(remainder,
     distributorBps, 10_000);
1762 // If the policy returns a non-zero distributorBps but a
     zero distributor address,
1763 // fold the distributor share into managerTake rather than
     silently dropping it.
1764  if (distributor == address(0)) distributorCut = 0;
1765  uint256 managerTake    = remainder - distributorCut;
1766
1767
```

```
    // Record state update BEFORE external transfers (CEI
    pattern).
1768 // This prevents a policy that reverts after transfers
    from leaving funds extracted
1769 // but state un-updated, which would allow a second
    collection over the same period.
1770  IFeePolicy(feePolicy).recordCollection(account, grossFee,
    currentNav);
1771
1772  if (feeToken == address(0)) {
1773  if (protocolCut    > 0) _safeTransferETH(account,
    treasury,    protocolCut);
1774  if (distributorCut > 0) _safeTransferETH(account,
    distributor, distributorCut);
1775  if (managerTake    > 0) _safeTransferETH(account,
    recipient,   managerTake);
1776 } else {
1777  if (protocolCut    > 0) _safeTransferERC20(account,
    feeToken, treasury,    protocolCut);
1778  if (distributorCut > 0) _safeTransferERC20(account,
    feeToken, distributor, distributorCut);
1779  if (managerTake    > 0) _safeTransferERC20(account,
    feeToken, recipient,   managerTake);
1780 }
1781
1782  emit FeesCollected(account, feeToken, grossFee, protocolCut,
    distributorCut, managerTake);
1783 }
1784
1785 /// @dev Execute a native ETH transfer out of the Safe via
    module call.
1786 /// @dev Reverts the entire fee collection if the Safe call
    returns false.
1787 ///      Intentional design: partial payouts would leave split
    invariants broken.
1788 ///      Use ERC-20 tokens where recipient payability cannot
    be guaranteed.
1789  function _safeTransferETH(address account, address to, uint256
    value) internal {
1790  if (!ISafe(account).execTransactionFromModule(to, value,
    "", 0)) revert FeeTransferFailed();
1791 }
1792
1793 /// @dev Execute an ERC-20 transfer out of the Safe via module
    call, verifying BOTH the
1794 ///      Safe module's success bool AND the token's own return
    value (SafeERC20 semantics).
```

```
1795 ///      The module-only bool is true whenever the inner `transfer` does not revert, so a
1796 ///      token that returns `false` without reverting — or returns malformed data — would
1797 ///      otherwise be recorded as a collected fee while no tokens moved. This reverts on
1798 ///      that case. A compliant token that returns nothing (non-standard, e.g. some USDT
1799 ///      deployments) is tolerated as success, matching SafeERC20.
1800 ///      The return value is decoded as a uint256 compared to 1 (rather than abi.decode to
1801 ///      bool) so a non-canonical word reverts cleanly with FeeTransferFailed instead of a
1802 ///      decode panic, consistent with the permission-evaluation decode elsewhere.
1803 /// @dev OUT OF SCOPE: fee-on-transfer shortfall. A fee-on-transfer token returns `true`
1804 ///      and does move tokens, just fewer than `amount`; this check does not guarantee the
1805 ///      recipient received `amount`. That risk is bounded by the account's feeAsset choice.
1806  function _safeTransferERC20(address account, address token, address to, uint256 amount) internal {
1807  bytes memory data = abi.encodeCall(IERC20.transfer, (to, amount));
1808 (bool ok, bytes memory ret) = ISafe(account).execTransac-tionFromModuleReturnData(token, 0, data, 0);
1809  if (!ok) revert FeeTransferFailed();
1810  if (ret.length != 0 && (ret.length < 32 || abi.decode(ret, (uint256)) != 1)) revert FeeTransferFailed();
1811 }
1812
1813 //
     ------------------------------------------------------------------------
1814 // 5. Principal tracking
1815 //
     ------------------------------------------------------------------------
1816
1817 /// @notice Record a deposit into the account. Only the permissionSigner may call.
1818 /// @dev    These values are informational. They are not used to constrain fee
1819 ///         collection on-chain, but may be used by off-chain tooling and future policy
1820 ///         contracts to derive context-aware fee limits.
1821 /// @param  account The registered Safe account.
```

```
1822 /// @param  amount  Deposit amount to record (in the account's
     base currency units).
1823  function recordDeposit(address account, uint256 amount)
     external nonReentrant {
1824 _requireRegistered(account);
1825  if (msg.sender != configs[account].permissionSigner) revert
     NotPermissionSigner();
1826  uint256 newDeposits = cumulativeDeposits[account] +=
     amount;
1827  emit DepositRecorded(account, amount, newDeposits);
1828 }
1829
1830 /// @notice Record a withdrawal from the account. Only the
     permissionSigner may call.
1831 /// @param  account The registered Safe account.
1832 /// @param  amount  Withdrawal amount to record (in the
     account's base currency units).
1833  function recordWithdrawal(address account, uint256 amount)
     external nonReentrant {
1834 _requireRegistered(account);
1835  if (msg.sender != configs[account].permissionSigner) revert
     NotPermissionSigner();
1836  uint256 newWithdrawals = cumulativeWithdrawals[account]
     += amount;
1837  emit WithdrawalRecorded(account, amount, newWithdrawals);
1838 }
1839
1840 //
     ------------------------------------------------------------------------
1841 // Public helpers
1842 //
     ------------------------------------------------------------------------
1843
1844 /// @notice Compute the EIP-712 digest for a given struct
     hash.
1845 /// @dev    Exposed for off-chain tooling, frontends, and
     tests.
1846 /// @param  structHash The EIP-712 struct hash (output of
     `keccak256(abi.encode(TYPEHASH, ...))`).
1847 /// @return            The final EIP-712 digest including
     domain separator.
1848  function hashTypedDataV4(bytes32 structHash) external view
     returns (bytes32) {
1849  return _hashTypedDataV4(structHash);
1850 }
1851
1852
```

```
    //
    --------------------------------------------------------------------
1853 // Internal helpers
1854 //
    --------------------------------------------------------------------
1855
1856 /// @dev Reverts with AccountNotRegistered when the account
    has not been registered.
1857  function _requireRegistered(address account) internal view {
1858  if (!registered[account]) revert AccountNotRegistered(ac-
    count);
1859 }
1860
1861 /// @dev Return the flat registration fee for a permission.
1862  function _calcPermissionFee() internal view returns (uint256)
    {
1863  return governance.permissionRegistrationFee();
1864 }
1865
1866 /// @dev Forward `fee` to the treasury and refund any ETH
    overpayment to msg.sender.
1867 ///      All callers (registerPermission, replacePermission,
    registerPermissions) complete
1868 ///      every state mutation (nonce increment, permission
    array update) before invoking
1869 ///      this function. The refund callback to msg.sender
    therefore occurs after all
1870 ///      state is settled; re-entry into any nonReentrant
    function is blocked. Any
1871 ///      re-entry into non-guarded functions (revokeSession,
    activateSession) still
1872 ///      requires a valid permissionSigner signature, preventing
    unauthorised mutations.
1873  function _collectRegistrationFee(uint256 fee) internal {
1874  if (fee > 0) {
1875 (bool ok,) = treasury.call{value: fee}("");
1876  if (!ok) revert FeeTransferFailed();
1877 }
1878  uint256 excess = msg.value - fee;
1879  if (excess > 0) {
1880 (bool ok,) = msg.sender.call{value: excess}("");
1881  if (!ok) revert FeeTransferFailed();
1882 }
1883 }
1884
1885 /// @dev Remove a permission from the account's list using
    swap-and-pop to preserve
```

```
1886 ///      packed storage. Updates `_permissionIndex` for the swapped element.
1887  function _removePermission(address account, address permission) internal {
1888  uint256 idx = _permissionIndex[account][permission];
1889  if (idx == 0) revert PermissionNotRegistered(permission);
1890  address[] storage perms = _permissions[account];
1891  uint256 lastIdx = perms.length - 1;
1892  if (idx - 1 != lastIdx) {
1893  address last = perms[lastIdx];
1894  perms[idx - 1] = last;
1895 _permissionIndex[account][last] = idx;
1896 }
1897  perms.pop();
1898  delete _permissionIndex[account][permission];
1899 }
1900
1901 /// @dev Remove every permission registered for an account, clearing both the ordered
1902 ///      list and the index mapping, and emitting `PermissionRevoked` for each. Used by
1903 ///      `setManager` to reset mandates on signer rotation. Bounded by
1904 ///      governance.maxPermissionsPerAccount(). Unlike `_removePermission`, this does not
1905 ///      swap-and-pop per element: it reads each entry, clears its index, then deletes the
1906 ///      whole array in one shot — so the array is never mutated mid-iteration.
1907  function _clearPermissions(address account) internal {
1908  address[] storage perms = _permissions[account];
1909  uint256 len = perms.length;
1910  for (uint256 i; i < len;) {
1911  address perm = perms[i];
1912  delete _permissionIndex[account][perm];
1913  registrationEpoch[account][perm] += 1;
1914  emit PermissionRevoked(account, perm);
1915  unchecked { ++i; }
1916 }
1917  delete _permissions[account];
1918 }
1919
1920 /// @dev EIP-712-compliant encoding of `address[]`:
1921 ///      keccak256 of the concatenation of each address zero-padded to 32 bytes,
1922 ///      which is the ABI encoding of `address[]` per EIP-712 §4.
```

```
1923 /// @param  arr The address array to hash.
1924 /// @return     The EIP-712 hash of the array.
1925  function _hashAddressArray(address[] calldata arr) internal
     pure returns (bytes32) {
1926  bytes32[] memory buf = new bytes32[](arr.length);
1927  for (uint256 i; i < arr.length; i++) {
1928  buf[i] = bytes32(uint256(uint160(arr[i])));
1929 }
1930  return keccak256(abi.encodePacked(buf));
1931 }
1932
1933 /// @dev Verify an EIP-712 permissionSigner signature against
     a struct hash.
1934 ///      Supports both EOA (ECDSA) and smart-contract (ERC-1271)
     signers.
1935  function _verifySignerSig(address account, bytes32 structHash,
     bytes memory sig) internal view {
1936  bytes32 digest = _hashTypedDataV4(structHash);
1937  if (!_recoverOrERC1271(configs[account].permissionSigner,
     digest, sig)) revert InvalidSignerSignature();
1938 }
1939
1940 /// @dev Verify a signature against `expected`, supporting
     both ECDSA and ERC-1271.
1941 ///      ECDSA is tried first regardless of code presence;
     this handles EIP-7702 accounts
1942 ///      that install transient code but do not implement
     ERC-1271.  If ECDSA recovery
1943 ///      succeeds and the recovered address matches, the
     signature is accepted immediately.
1944 ///      Otherwise, falls back to ERC-1271 when `expected` is
     a contract.
1945 /// @param  expected Address expected to have produced the
     signature.
1946 /// @param  digest   EIP-712 digest to verify against.
1947 /// @param  sig      Signature bytes.
1948 /// @return          True if the signature is valid for
     `expected`.
1949  function _recoverOrERC1271(address expected, bytes32 digest,
     bytes memory sig) internal view returns (bool) {
1950 (address recovered, ECDSA.RecoverError err,) =
     ECDSA.tryRecover(digest, sig);
1951  if (err == ECDSA.RecoverError.NoError && recovered ==
     expected) return true;
1952  if (expected.code.length > 0) {
1953  try IERC1271(expected).isValidSignature(digest, sig)
     returns (bytes4 magic) {
```

```
1954  return magic == ERC1271_MAGIC;
1955 } catch {
1956  return false;
1957 }
1958 }
1959  return false;
1960 }
1961 }
```

Severity

Impact Explanation: [Medium] Operational DoS at the account level: the old permission is removed or a new permission remains registered but unconfigured (deny-by-default), and replace rotates manager/batch nonces, invalidating queued manager-signed actions. Recovery is possible by reconfiguring and re-signing; no direct fund loss.

Likelihood Explanation: [Medium] Requires a realistic but specific state (a pending factory replace/attach broadcast to a public mempool) and the attacker paying the registration fee. Such broadcasts are common operationally, and sabotage can be a rational incentive; no complex synchronization or dependencies are required.

Exploitation

Exploitation Scenarios:

Scenario 1.
Front-run MandateFactory.replace: An attacker copies kernelReplaceSig from a pending replace transaction and calls SailKernel.replacePermission first, removing oldPermission, installing newPermission, and rotating manager/batch nonce epochs. The victim's replace then reverts on its kernel step, rolling back configure(). Final state: newPermission registered but unconfigured (deny-by-default), oldPermission removed, queued manager-signed actions invalidated.
Preconditions / Assumptions
(a). Account is registered in SailKernel and has oldPermission registered
(b). governance.isPaused() == false
(c). newPermission has deployed code and is not already registered for the account
(d). PermissionSigner has signed a valid ReplacePermission message with current signerNonces[account] and an unexpired deadline
(e). Operator broadcasts MandateFactory.replace with configureSig and kernelReplaceSig to a public mempool
(f). Attacker can observe the mempool and is willing to pay

the registration fee

Scenario 2.
Front-run MandateFactory.attach: An attacker copies the kernelSig for registerPermission from a pending attach and calls SailKernel.registerPermission first. The victim's attach then reverts on its kernel step, rolling back configure(). Final state: the permission is registered but unconfigured (deny-by-default). No nonce-epoch rotation here.
Preconditions / Assumptions
(a). Account is registered in SailKernel
(b). governance.isPaused() == false
(c). template has deployed code and is not already registered for the account
(d). PermissionSigner has signed a valid RegisterPermission message with current signerNonces[account] and an unexpired deadline
(e). Operator broadcasts MandateFactory.attach with configureSig and kernelSig to a public mempool
(f). Attacker can observe the mempool and is willing to pay the registration fee

Scenario 3.
Invalidate queued manager operations during replace: Same as the first scenario, but timed to coincide with a backlog of pre-signed Dispatch/DispatchBatch. The early replace rotates manager/batch nonces, invalidating all queued signatures, while leaving the newly registered permission unconfigured (deny-by-default) after the victim's replace reverts.
Preconditions / Assumptions
(a). All preconditions from Scenario 1
(b). There exist queued manager-signed Dispatch/DispatchBatch operations using the current manager/batch nonce epoch

Vulnerability #3

Stale per-account applied fee-rate snapshot on reattach in StandardFeePolicy.onAttach causes first-window fee mispricing

Severity: MEDIUM | Likelihood: MEDIUM | Impact: MEDIUM | Status: Resolved

Resolved in PR #72. StandardFeePolicy.onAttach now refreshes the per-account applied management and performance fee-rate snapshots to the current schedule, so the first post-reattach window is priced at the current rate (prospectively; the dormant interval is not billed and the first-window performance fee remains suppressed).

When a previously used StandardFeePolicy instance is reattached to an account, onAttach resets the accrual start time but does not refresh the per-account appliedManagementFeeBps/appliedPerformanceFeeBps snapshots. computeFee uses these stale snapshots for the entire first post-reattach window until recordCollection runs, so if global fee rates changed while detached, the first window is billed at an obsolete rate. This can overcharge after a rate decrease (or undercharge after an increase).

StandardFeePolicy tracks per-account appliedManagementFeeBps and appliedPerformanceFeeBps snapshots that define the fee rates used by computeFee until the next recordCollection updates them. onAttach(account) is invoked by the kernel when an account reattaches the same policy instance; it resets lastCollectionTimestamp[account] to now and sets pendingReanchor[account] = true (suppressing the first performance fee), but it does not update appliedManagementFeeBps/appliedPerformanceFeeBps. As a result, the first accrual window that begins at onAttach is priced using the old per-account snapshots. If the feeManager changed managementFeeBps/performanceFeeBps while the policy was detached, the first post-reattach window is billed at the stale rate. The kernel's collectFees enforces the policy's computeFee cap and calls recordCollection before moving funds; recordCollection then updates the applied* snapshots for future windows and enforces a minimum 1-day interval since lastCollectionTimestamp. This yields a concrete over-collection risk after a rate decrease (or under-collection after an increase) for the entire first post-reattach window, whose length can be arbitrarily long (manager-controlled) beyond the 1-day minimum. Performance fees are suppressed in that first window due to pendingReanchor; the issue primarily affects the management fee leg.

file: contracts/policies/StandardFeePolicy.sol

```
331    lastCollectionTimestamp[account] = block.timestamp;
332    pendingReanchor[account]         = true;


333  }
334  }
```

```
331    lastCollectionTimestamp[account] = block.timestamp;
332    pendingReanchor[account]         = true;
333 +  appliedManagementFeeBps[account]  = managementFeeBps;
334 +  appliedPerformanceFeeBps[account] = performanceFeeBps;
335  }
336  }
```

Severity

Impact Explanation: [Medium] Direct, material loss of fees due to mispricing of the first post-reattach window's management fee; this is a fee/yield overcharge relative to the newly configured schedule, not principal compromise.

Likelihood Explanation: [Medium] Requires a realistic but specific lifecycle (detach → rate change while detached → reattach same instance → collect after ≥1 day). No admin malice or broken integrations required; managers have incentive to request computeFee's maximum.

Exploitation

Exploitation Scenarios:

Scenario 1.
Single account: An account previously used the policy at rate M1, detaches, the feeManager lowers the policy's global management fee to M2 < M1, then the account reattaches the same instance. onAttach resets lastCollectionTimestamp but leaves appliedManagementFeeBps at M1. After at least 1 day, the manager calls collectFees, and computeFee charges management fees for the post-reattach window at M1 instead of M2, over-collecting for that window.
Preconditions / Assumptions
(a). Account previously used this StandardFeePolicy instance and had hwmSeeded[account] = true
(b). A last successful collection set appliedManagementFeeBps[account] = M1
(c). PermissionSigner detaches the policy for the account (setFeePolicy to another or zero)
(d). FeeManager lowers global managementFeeBps to M2 < M1 while detached
(e). PermissionSigner reattaches the same policy instance (kernel calls onAttach)
(f). onAttach resets lastCollectionTimestamp and sets pendingReanchor; applied* snapshots remain at M1
(g). At least 1 day elapses after onAttach
(h). Manager (or the account) calls collectFees requesting up to computeFee; sessionActive true; feeAsset matches

binding

Scenario 2.
Maximized impact: Same as above, but the manager delays the first collection well beyond 1 day after reattach. Because computeFee uses elapsed time at the stale M1 rate until recordCollection runs, the over-collection scales with elapsed and NAV for the entire delayed window.
Preconditions / Assumptions
(a). All preconditions of Scenario 1
(b). Manager waits significantly longer than 1 day after onAttach before calling collectFees to increase elapsed

Scenario 3.
Fleet-level: Multiple accounts that previously used the same policy detach, the feeManager lowers the global management fee, and later the accounts reattach the same instance. For each account, onAttach resets time but appliedManagementFeeBps remains at the old M1. The first post-reattach collections across the fleet are all computed at M1 rather than the new M2, aggregating a large over-collection across accounts.
Preconditions / Assumptions
(a). Multiple accounts previously used the same policy instance and had seeded HWM and appliedManagementFeeBps set at their last collections
(b). PermissionSigners detach the policy across those accounts
(c). FeeManager lowers global managementFeeBps while detached
(d). PermissionSigners reattach the same policy instance for those accounts (kernel calls onAttach per account)
(e). At least 1 day elapses for each account before first post-reattach fee collection
(f). Managers call collectFees per account, requesting up to computeFee

Vulnerability #4

Empty data passed to Safe.checkSignatures in SailKernel.registerAccount causes DoS/friction for contract-owner Safe self-registration

Severity: LOW | Likelihood: LOW | Impact: MEDIUM | Status: Resolved

Resolved in PR #69. registerAccount now passes the EIP-712 preimage (0x19 0x01 domainSeparator structHash) as the checkSignatures data argument, enabling Safe's v==0 contract-signature path (ERC-1271 / nested-Safe owners) to validate.

SailKernel.registerAccount always calls Safe.checkSignatures with data == "", which prevents Safe v1.4.1's v==0 contract-owner signature path from validating. EOAs and approved-hash owners still work, but Safes whose threshold requires a contract owner signature cannot self-register unless each such owner first on-chain approves the digest. In the worst case (non-transacting ERC-1271 owner), self-registration is impossible.

In SailKernel.registerAccount, the kernel validates owner authorization by calling Safe.checkSignatures(digest, "", ownerSig). On Safe v1.4.1, the v==0 contract-signature path (used when a Safe owner is a contract, e.g., another Safe or an ERC-1271 wallet) first requires keccak256(data) == dataHash before invoking owner.isValidSignature(data, contractSignature). Because Sail passes data == "" while dataHash is the non-empty EIP-712 digest, any v==0 entry in ownerSig fails with GS027. EOAs validate normally, and the approved-hash (v==1) path works if each contract owner pre-approves the digest via Safe.approveHash(digest). As a result, existing Safes whose owner threshold requires a contract-owner signature cannot self-register unless all such owners can and do on-chain approve the digest; if a required contract owner cannot originate approveHash (e.g., a verifier-only ERC-1271 contract), self-registration is impossible. This affects only the self-registration path; createAccount is unaffected. There is no funds impact.

file: contracts/core/SailKernel.sol

```
781  ///         `execTransaction` then calls this function (so
     msg.sender == the Safe, satisfying the
782  ///         codehash gate) carrying that signature. msg.sender
     == account is preserved.
783 -/// @dev    `checkSignatures` is called with empty `data`,
     which suits EOA and approved-hash owners.
```

```
784 -///         A Safe owner that is itself a contract relying on
    the legacy v==0 contract-signature path
785 -///         would require non-empty `data` (keccak256(data)==di-
    gest) — a nested-Safe-owner edge case.

786  /// @param  permissionSigner  Address that will sign
    permission-registry operations.
787  /// @param  manager           Address that will sign dispatch
    calls.

...

832  //    never clears, so a replay reverts AccountAlreadyReg-
    istered in _registerAccount).
833   if (block.timestamp > deadline) revert DeadlineExpired(dead-
    line, block.timestamp);
834 -  bytes32 digest = _hashTypedDataV4(keccak256(abi.encode(
835   REGISTER_ACCOUNT_TYPEHASH,
836   msg.sender,

...

840   feeAsset,
841   deadline
842 -)));



843  // Reverts (GS0xx) unless `ownerSig` satisfies the Safe's
    owner set + threshold.
844 -  ISafe(msg.sender).checkSignatures(digest, "", ownerSig);
845
846  _registerAccount(msg.sender, permissionSigner, manager,
    feePolicy, feeAsset);
```

```
781  ///         `execTransaction` then calls this function (so
    msg.sender == the Safe, satisfying the
782  ///         codehash gate) carrying that signature. msg.sender
    == account is preserved.
783 +/// @dev    `checkSignatures` is called with the EIP-712
    preimage as `data`
784 +///         (0x1901 || domainSeparator || structHash) so Safe
    v1.4.1's v==0
785 +///         contract-signature path can validate (requires
    keccak256(data)==digest).
```

```
786 +///         EOAs and approved-hash owners remain supported.
787  /// @param  permissionSigner  Address that will sign
     permission-registry operations.
788  /// @param  manager           Address that will sign dispatch
     calls.

...

833  //    never clears, so a replay reverts AccountAlreadyReg-
     istered in _registerAccount).
834   if (block.timestamp > deadline) revert DeadlineExpired(dead-
     line, block.timestamp);
835 + bytes32 structHash = keccak256(abi.encode(
836   REGISTER_ACCOUNT_TYPEHASH,
837   msg.sender,

...

841   feeAsset,
842   deadline
843 +));
844 + bytes32 digest = _hashTypedDataV4(structHash);
845 +// EIP-712 preimage enables Safe v1.4.1 v==0
     contract-signature path.
846 + bytes memory preimage = abi.encodePacked(bytes1(0x19),
     bytes1(0x01), _domainSeparatorV4(), structHash);
847  // Reverts (GS0xx) unless `ownerSig` satisfies the Safe's
     owner set + threshold.
848 + ISafe(msg.sender).checkSignatures(digest, preimage,
     ownerSig);
849
850  _registerAccount(msg.sender, permissionSigner, manager,
     feePolicy, feeAsset);
```

Severity

Impact Explanation: [Medium] For affected configurations, an important non-core function (self-registration of pre-existing Safes) is broken (Scenario 1) or requires additional on-chain coordination (Scenario 2). No funds or core invariants are impacted.

Likelihood Explanation: [Low] The combined preconditions are uncommon: users must use registerAccount (not createAccount), have thresholds requiring contract-owner signatures, and (for the DoS case) have a required contract owner that cannot originate approveHash. These make the problematic state relatively rare.

Exploitation

Exploitation Scenarios:

Scenario 1.
Scenario 1 (DoS): An existing Safe (SafeTop) has owners [EOA_A, Contract_C] with threshold=2. Contract_C is an ERC-1271 verifier contract that cannot originate external calls, so it cannot call SafeTop.approveHash. The operator builds the RegisterAccount EIP-712 digest, aggregates EOA_A's ECDSA and a v==0 contract signature for Contract_C, and calls SailKernel.registerAccount via SafeTop.execTransaction. Safe.checkNSignatures enforces keccak256(data) == dataHash for v==0; since Sail passed data == "", verification reverts with GS027. Because Contract_C cannot use the approved-hash workaround, self-registration cannot succeed.
Preconditions / Assumptions
(a). Using SailKernel.registerAccount (self-registration of an existing Safe), not createAccount
(b). Safe module already enabled and Safe nonce() > 0
(c). Safe proxy codehash and singleton are governance-allowlisted
(d). Owner threshold requires at least one contract owner signature
(e). The required contract owner cannot originate approveHash (e.g., verifier-only ERC-1271 contract)

Scenario 2.
Scenario 2 (Friction): An existing Safe (SafeTop) has owners [EOA_A, SafeNested] with threshold=2, and SafeNested is itself a Safe capable of executing transactions. Attempting registerAccount with EOA_A's ECDSA and a v==0 contract signature for SafeNested fails (GS027). The operator then has SafeNested execTransaction to call SafeTop.approveHash(digest), rebuilding ownerSig to include a v==1 approved-hash entry for SafeNested. The second registerAccount call succeeds, but only after extra on-chain coordination.
Preconditions / Assumptions
(a). Using SailKernel.registerAccount (self-registration of an existing Safe), not createAccount
(b). Safe module already enabled and Safe nonce() > 0
(c). Safe proxy codehash and singleton are governance-allowlisted
(d). Owner threshold requires a contract owner (nested Safe) signature
(e). Nested Safe can execute a transaction to call approveHash on the top-level Safe

Vulnerability #5

Floor-based oracle min-out calculation in SwapPermission allows 1 base-unit slack, weakening slippage bounds

Severity: LOW | Likelihood: MEDIUM | Impact: LOW | Status: Acknowledged

Valid but won't patch; documented in SwapPermission and TEMPLATES.md. The slack is bounded to a single base unit and only material for very-low-decimal (especially 0-decimal) output tokens — negligible for typical 6–18-decimal tokens. The oracle band is a sanity bound; the manager-supplied amountOutMin is the primary slippage floor. Adding ceil-rounding to recover one base unit on exotic tokens isn't worth the added template complexity.

SwapPermission computes the oracle-constrained minimum-out using integer floor rounding twice and only denies when the computed floor is zero. This creates a bounded 1 base-unit slack in the manager's favor near integer boundaries. It is negligible for typical 6–18 decimal tokens but can be material for low-decimal (especially 0-decimal) outputs.

In SwapPermission._oracleCheck, the gate enforces amountOutMin against an oracle-derived floor computed as: expectedOut = floor(amountIn * price / 10^dec), and oracleMinOut = floor(expectedOut * (10_000 - maxSlippageBps) / 10_000). The check denies only when oracleMinOut == 0; otherwise it requires amountOutMin >= oracleMinOut. Because both operations floor, the effective enforced minimum is at most one base unit lower than a strict ceil-based interpretation would require (even at 0 bps). A manager can select boundary-sized trades (where the rational oracle quote is just below an integer) and set amountOutMin to that lower integer so the permission passes. The template appropriately fail-closes dust trades (when oracleMinOut truncates to 0), but does not prevent this one-unit slack for non-dust trades. For mainstream tokens (6–18 decimals) the slack is one micro-unit/wei and economically negligible; for low-decimal tokens (especially 0 decimals) the slack can be one whole token per trade. The permission constrains only the manager's amountOutMin; it does not verify realized output at evaluation time.

file: contracts/templates/SwapPermission.sol

```
278  // trades are denied under an oracle as a deliberate
     consequence.
279   if (oracleMinOut == 0) return false;
```

```
280   return amountOutMin >= oracleMinOut;
281  }
```

```
278  // trades are denied under an oracle as a deliberate
     consequence.
279   if (oracleMinOut == 0) return false;
280 +// Enforce a strict ceil-rounded minimum to eliminate
     1-unit slack near integer boundaries.
281 + oracleMinOut = Math.mulDiv(
282 + Math.mulDiv(amountIn, price, 10 ** uint256(dec),
     Math.Rounding.Ceil),
283 +10_000 - s.maxSlippageBps,
284 +10_000,
285 + Math.Rounding.Ceil
286 +);
287   return amountOutMin >= oracleMinOut;
288  }
```

Severity

Impact Explanation: [Low] Loss per trade is strictly bounded to 1 base unit of tokenOut and does not freeze funds, break invariants, or enable theft beyond permissions. It is negligible for typical 6–18 decimal tokens and only becomes noticeable for low-decimal outputs (e.g., 0 decimals).

Likelihood Explanation: [Medium] While low-decimal outputs are uncommon, they are plausible configuration choices. Once present, choosing boundary-sized trades and setting amountOutMin accordingly is trivial for an adversarial manager.

Exploitation

Exploitation Scenarios:

Scenario 1.
Scenario 1 (0 bps, 0-decimal output): The account configures SwapPermission with a fresh oracle and maxSlippageBps = 0; tokenOut has 0 decimals. The manager chooses amountIn so the oracle-implied output is ~1.99. They set amountOutMin = 1 and route the swap to return 1 token. Because expectedOut floors to 1, oracleMinOut = 1, the check passes and the trade executes with 1 tokenOut instead of enforcing a stricter integer of 2.
Preconditions / Assumptions
(a). Account registered and configured with SwapPermission; routers and tokens allowlisted; recipient pinned to the account; ctx.value = 0
(b). A valid price oracle is configured; maxPriceAgeSec > 0; oracle price is fresh and non-zero; oracle decimals dec ≤ 77
(c). maxSlippageBps = 0
(d). tokenOut has 0 decimals (base unit = 1 whole token)
(e). Manager is adversarial within permissions and can choose amountIn near integer boundaries and set amountOutMin
(f). The chosen route/path produces an execution that returns exactly 1 tokenOut; amountIn ≤ maxAmountPerTx

Scenario 2.
Scenario 2 (small bps, 0-decimal output): With maxSlippageBps = 1% and tokenOut having 0 decimals, the manager picks amountIn so the oracle-implied output is slightly above 2. expectedOut floors to 2, then oracleMinOut = floor(2 * 0.99) = 1. The manager sets amountOutMin = 1 and executes a route that returns 1 token; the permission passes even though a more conservative interpretation might expect a higher integer floor.
Preconditions / Assumptions
(a). Account registered and configured with SwapPermission; routers and tokens allowlisted; recipient pinned to the account; ctx.value = 0
(b). A valid price oracle is configured; maxPriceAgeSec > 0; oracle price is fresh and non-zero; oracle decimals dec ≤ 77
(c). maxSlippageBps is small (e.g., 1%)
(d). tokenOut has 0 decimals (base unit = 1 whole token)
(e). Manager selects amountIn so the oracle-quoted output is slightly above 2, sets amountOutMin = 1
(f). The chosen route returns exactly 1 tokenOut; amountIn ≤ maxAmountPerTx

Scenario 3.
Scenario 3 (accumulation via repetition on low-decimal output): With low-decimal tokenOut and small/zero bps, the manager repeatedly selects boundary trades so that expectedOut floors to N and oracleMinOut becomes N-1. They set amountOutMin = N-1 and route swaps to return exactly N-1. Each trade passes and the account consistently realizes a 1 base-unit shortfall per trade; losses accumulate linearly with repetitions.
Preconditions / Assumptions
(a). Account registered and configured with SwapPermission; routers and tokens allowlisted; recipient pinned to the account; ctx.value = 0
(b). A valid price oracle is configured; maxPriceAgeSec > 0; oracle price is fresh and non-zero; oracle decimals dec ≤ 77
(c). maxSlippageBps = 0 or small; per-transaction cap permits repeated small trades
(d). tokenOut has very low decimals (0 or 1) to make the 1 base-unit slack non-dust
(e). Manager repeatedly selects boundary-sized amountIn to keep expectedOut near integers so oracleMinOut = N-1 and sets amountOutMin = N-1 each time

(f). Each chosen route returns exactly N-1 base units; all trades respect maxAmountPerTx

Vulnerability #6

Missing validation of Aave interestRateMode/referralCode in BorrowPermission.evaluate causes manager-controlled debt type and referral attribution

Severity: LOW | Likelihood: MEDIUM | Impact: LOW | Status: Resolved

The interest-rate-mode portion was Resolved in PR #73 (Aave borrows constrained to variable rate). The referral-code portion is valid but won't be patched: a referral code is off-chain attribution on the account's own position with no effect on funds, principal, or the resulting position. Constraining it would add code to neutralize a marketing field and would prevent the template from participating in a venue referral program the account owner might want. Documented as a known boundary in BorrowPermission and TEMPLATES.md.

BorrowPermission's Aave borrow path decodes but does not validate interestRateMode or referralCode, allowing a malicious manager to choose stable vs variable debt and any referral code while still passing target/asset/amount/onBehalfOf/LTV checks. This can increase the account's interest costs or divert referral attribution/value and can also be used for minor griefing by selecting an unsupported mode.

In BorrowPermission's Aave branch, the calldata is decoded as (asset, amount, _, _, onBehalfOf), and the 3rd (interestRateMode) and 4th (referralCode) arguments are ignored. The permission enforces: (a) protocol target allowlist, (b) asset allowlist, (c) per-transaction amount cap, (d) onBehalfOf must equal the account, and (e) optional LTV ceiling via paired oracles with freshness checks. Because interestRateMode and referralCode are not constrained, a manager can: (1) choose stable vs variable debt product without violating the enforced checks, potentially increasing ongoing interest costs and risk dynamics; (2) set referralCode to direct attribution/off-chain rewards to themselves or an affiliate; and (3) select an unsupported mode to cause target-layer reverts (minor griefing). This does not bypass LTV or amount caps and does not redirect funds away from the account. Owners can mitigate by revoking the permission, rotating the manager, or revoking the session. A hardening fix is to add explicit checks for interestRateMode (e.g., variable-only) and a specific referralCode (e.g., 0 or a configured value).

file: contracts/templates/BorrowPermission.sol

```
196   if (ctx.selector == AAVE_BORROW) {
197   if (txData.length < LEN_AAVE) return false;
198 -(address asset, uint256 amount, , , address onBehalfOf)
      =
199   abi.decode(txData[4:], (address, uint256, uint256,
      uint16, address));



200   if (!isAllowedAsset[ctx.account][asset]) return false;
201   if (amount > s.maxAmountPerTx)           return false;
```

```
196   if (ctx.selector == AAVE_BORROW) {
197   if (txData.length < LEN_AAVE) return false;
198 +(address asset, uint256 amount, uint256 rateMode,
      uint16 referralCode, address onBehalfOf) =
199   abi.decode(txData[4:], (address, uint256, uint256,
      uint16, address));
200 +// Enforce variable-rate borrowing only (interestRateMode
      == 2) and no referral attribution (referralCode == 0).
201 + if (rateMode != 2)           return false;
202 + if (referralCode != 0)       return false;
203   if (!isAllowedAsset[ctx.account][asset]) return false;
204   if (amount > s.maxAmountPerTx)           return false;
```

Severity

Impact Explanation: [Low] No direct theft or frozen funds occur; LTV and amount caps remain enforced. The primary harm is potential increase in interest expense over time or diversion of referral attribution/value, which is bounded, reversible (repay/rotate/revoke), and contingent on external conditions.

Likelihood Explanation: [Medium] A malicious manager can readily exploit this when the Aave market state permits (e.g., stable enabled, referral programs active). While it depends on plausible external states, it requires no complex conditions beyond an adversarial manager and typical configuration.

Exploitation

Exploitation Scenarios:

Scenario 1.
Scenario 1 (manager forces stable-rate debt): The manager crafts Aave Pool.borrow(asset, amount, interestRateMode=1, referralCode=any, onBehalfOf=account) and dispatches via SailKernel using the registered BorrowPermission. BorrowPermission approves (it ignores rate/referral) while enforcing target/asset/amount/onBehalfOf and optional LTV. Aave executes, opening stable-rate debt for the account, potentially increasing interest costs versus variable-rate borrowing.
Preconditions / Assumptions
(a). Safe account is registered with SailKernel; BorrowPermission is registered for the account
(b). BorrowPermission configuration allowlists the canonical Aave v3 Pool target and the chosen asset
(c). maxAmountPerTx >= planned borrow amount; if oracles are set, both are configured and fresh so _ltvCheck passes
(d). On the selected Aave market, stable-rate borrowing is enabled for the asset
(e). Manager controls the delegated signer and session is active; kernel not paused; value=0

Scenario 2.
Scenario 2 (manager captures referral attribution): The manager submits Aave Pool.borrow with referralCode set to an affiliate they control (mode can be stable or variable), onBehalfOf=account. BorrowPermission approves and Aave processes the borrow, attributing referral to the attacker's chosen code, diverting attribution/off-chain rewards without affecting on-chain fund movements.
Preconditions / Assumptions
(a). Safe account is registered with SailKernel; BorrowPermission is registered for the account
(b). BorrowPermission configuration allowlists the canonical Aave v3 Pool target and the chosen asset
(c). maxAmountPerTx >= planned borrow amount; if oracles are set, both are configured and fresh so _ltvCheck passes
(d). Referral/attribution program exists and yields value or meaningful attribution
(e). Manager controls the delegated signer and session is active; kernel not paused; value=0

Scenario 3.
Scenario 3 (minor griefing with unsupported mode): The manager submits Aave Pool.borrow with an unsupported interestRateMode for the asset (e.g., stable disabled), onBehalfOf=account. BorrowPermission approves, but Aave reverts due to unsupported mode. This causes operational noise/grief with no on-chain loss.
Preconditions / Assumptions
(a). Safe account is registered with SailKernel; BorrowPermission is registered for the account
(b). BorrowPermission configuration allowlists the canonical Aave v3 Pool target and the chosen asset
(c). maxAmountPerTx >= planned borrow amount; if oracles are set, both are configured and fresh so _ltvCheck passes
(d). On the selected Aave market, the chosen interestRateMode (e.g., stable) is disabled for that asset
(e). Manager controls the delegated signer and session is active; kernel not paused; value=0

Vulnerability #7

pendingReanchor + zero management-fee snapshot in StandardFeePolicy reattach flow causes fee-collection deadlock and revenue loss

Severity: LOW | Likelihood: LOW | Impact: LOW | Status: Resolved

Resolved in PR #72. computeFee returns a minimal fee on the first post-reattach collection when the management leg is zero in a suppressed window, allowing the clearing collection to settle and recordCollection to clear the re-anchor latch. Documented boundary: an account holding no balance of the configured fee asset cannot complete the clearing collection.

Reattaching the same StandardFeePolicy instance to an account with a 0% per-account management-fee snapshot sets pendingReanchor and suppresses performance fees. Because computeFee returns 0, SailKernel.collectFees rejects all calls (ZeroFee or FeeTooLarge), making recordCollection unreachable and leaving pendingReanchor uncleared indefinitely. This deadlocks fee collection on that instance for the account until operators migrate to a fresh policy instance, causing manager and protocol fee loss. A related rounding case can transiently extend the performance-fee waiver window.

When a previously seeded account reattaches the same StandardFeePolicy instance via SailKernel.setFeePolicy, StandardFeePolicy.onAttach resets lastCollectionTimestamp to now and sets pendingReanchor = true. While pendingReanchor is true, computeFee suppresses the entire performance-fee leg and only returns the management-fee component, calculated using the per-account appliedManagementFeeBps snapshot. That snapshot only updates inside recordCollection (and at initial seeding). SailKernel.collectFees requires a nonzero grossFee and enforces grossFee 0 but very small and NAV is small, making 1day management accrual round to zero. Then collectFees may revert for multiple days, extending the period during which performance is suppressed; once management accrual becomes nonzero or NAV grows, the flag clears on the first successful collection. Operational mitigations include switching to a fresh policy instance and reseeding HWM, or ensuring a nonzero per-account management-fee snapshot at reattach.

file: contracts/policies/StandardFeePolicy.sol

```
270   );
271
```

```
272  // On the first collection after a reattach, charge no
     performance fee: the HWM is about to be
273  // re-anchored to max(priorHWM, currentNav) (see
     recordCollection), so any gain booked while
```

```
270  );
271
272 +// Allow a one-time minimal fee to clear the reanchor
     latch on zero-management schedules
273 +// after the minimum interval has elapsed. This enables
     recordCollection to run and
274 +// re-base HWM without requiring a non-zero appliedMan-
     agementFeeBps snapshot.
275 + if (pendingReanchor[account] && managementFee == 0 &&
     elapsed >= MIN_COLLECTION_INTERVAL) {
276 + return (1, _distributor, _distributorBps);
277 +}
278  // On the first collection after a reattach, charge no
     performance fee: the HWM is about to be
279  // re-anchored to max(priorHWM, currentNav) (see
     recordCollection), so any gain booked while
```

Severity

Impact Explanation: [Low] The issue denies or reduces fee collection for the manager and protocol on a per-instance, per-account basis, resulting in direct fee revenue loss. End-user principal funds are not at risk, and an operational workaround (switch to new policy instance and reseed HWM) exists.

Likelihood Explanation: [Low] The deadlock requires the specific operator lifecycle choice of reattaching the same policy instance when the per-account management-fee snapshot is exactly 0% (true 0/20), and the rounding extension requires small NAV and very low bps. These are plausible but operator-driven and relatively uncommon states; competent operators can avoid or quickly remediate them.

Exploitation

Exploitation Scenarios:

Scenario 1.
Scenario 1 (deadlock on 0/20): An account with StandardFeePolicy has hwmSeeded = true and appliedManagementFeeBps == 0 (true 0-and-20). The operator reattaches the same policy instance via SailKernel.setFeePolicy. onAttach sets lastCollectionTimestamp = now and pendingReanchor = true. After the minimum interval, the manager calls SailKernel.collectFees with a nonzero grossFee. StandardFeePolicy.computeFee returns 0 (performance suppressed; management snapshot 0), so the kernel rejects the call (FeeTooLarge/ZeroFee), never reaching recordCollection. pendingReanchor is never cleared and fee collection on this instance remains blocked indefinitely until operators migrate to a new instance and reseed HWM.
Preconditions / Assumptions
(a). StandardFeePolicy is in use and hwmSeeded[account] == true
(b). The per-account appliedManagementFeeBps[account] == 0 at the time of reattach (true 0/20 snapshot)
(c). Operator (permissionSigner) reattaches the SAME policy instance via SailKernel.setFeePolicy

(d). SailKernel.collectFees enforces grossFee != 0 and grossFee <= computeFee(account, currentNav)
(e). StandardFeePolicy.computeFee suppresses performance while pendingReanchor == true and uses the per-account appliedManagementFeeBps snapshot for management
(f). StandardFeePolicy.recordCollection is only callable via the kernel after a successful collectFees and cannot run if computeFee returns 0

Scenario 2.
Scenario 3 (rounding extends waiver window): An account reattaches the same policy instance with pendingReanchor = true and has a very low appliedManagementFeeBps snapshot and small NAV, so 1day management accrual rounds to 0. collectFees reverts (ZeroFee or FeeTooLarge), preventing recordCollection from clearing pendingReanchor. If NAV remains small, this can extend the performance-fee suppression window beyond the intended single period. Once elapsed grows or NAV increases enough that management accrual becomes nonzero, a subsequent collectFees succeeds, clears pendingReanchor, and ratchets HWM.
Preconditions / Assumptions
(a). StandardFeePolicy is in use and hwmSeeded[account] == true
(b). Operator reattaches the SAME policy instance, setting pendingReanchor = true
(c). The per-account appliedManagementFeeBps[account] is positive but very small, and currentNav is small enough that shortperiod management accrual rounds to zero
(d). SailKernel.collectFees enforces grossFee != 0 and grossFee <= computeFee(account, currentNav), so zero management accrual blocks collection until accrual becomes nonzero

Vulnerability #8

Lack of return-data verification in SailKernel.dispatch with Compound V2 borrow causes nonce burn and misleading success emission

Severity: INFORMATIONAL | Likelihood: LOW | Impact: LOW | Status: Acknowledged

Valid but won't patch; documented in KERNEL.md. Single dispatch checks only the boolean module-call success and does not inspect return data, so a venue that signals failure via a non-reverting status code (e.g. Compound V2) can report success on a no-op, consuming a nonce with no fund effect. Inspecting return data would require a kernel-level post-call hook, which we are deliberately not adding to keep the trusted core minimal; batch flows with postconditions are the recommended path for such venues.

SailKernel.dispatch checks only the Safe module call's boolean (raw CALL success) and does not inspect target return data. Compound V2 cToken.borrow(uint256) often fails by returning a non-zero status without reverting. BorrowPermission permits such borrows based on preconditions. As a result, a soft-failing borrow is treated as a successful dispatch: Dispatched is emitted and the manager nonce is consumed even though no borrow occurred. An attacker holding a valid manager signature can time submission to burn the nonce. No funds are lost; impact is minor operational griefing.

Safe v1.4.1's ModuleManager.execTransactionFromModule returns only whether the low-level CALL reverted. SailKernel.dispatch relies solely on this boolean and does not decode or validate the called contract's return data. BorrowPermission explicitly supports Compound V2's borrow(uint256), enforcing only preconditions (allowlists, per-tx cap, optional LTV) via staticcall and cannot observe post-call outcomes. Compound V2 commonly signals borrow failure by returning a non-zero error code without reverting. Consequently, when such a soft-fail occurs, execTransactionFromModule still returns true, SailKernel.dispatch emits Dispatched, and managerNonces is incremented, despite no actual borrow. An adversary who possesses a valid manager-signed dispatch can submit it during a soft-fail window to burn the nonce. This results in a misleading success event and minor operational friction (re-signing needed), but no loss of funds or broken invariants. Using dispatchBatch with a postcondition that depends on the borrowed funds reverts the whole batch on soft-failure and preserves the nonce.

file: contracts/core/SailKernel.sol

```
1425  if (!_evaluatePermission(permission, data, ctx)) revert
      PermissionDenied(permission);
1426
1427  if (!ISafe(account).execTransactionFromModule(target,
      value, data, 0)) revert SafeExecutionFailed();
```

```
1425  if (!_evaluatePermission(permission, data, ctx)) revert
      PermissionDenied(permission);
1426
1427+ // NOTE: Some protocols (e.g. Compound V2) may signal
      failure via non-reverting status codes.
1428+ // To eliminate such “silent success”, integrate an optional
      permission post-call validation:
1429+ // 1) when requested by the permission, use
      execTransactionFromModuleReturnData to capture returnData,
1430+ // 2) staticcall a permission-provided post-check over
      (txData, returnData, ctx), and
1431+ // 3) revert if the post-check denies. Until then, prefer
      batch flows with postconditions.
1432  if (!ISafe(account).execTransactionFromModule(target,
      value, data, 0)) revert SafeExecutionFailed();
```

Severity

Impact Explanation: [Low] No principal funds or core invariants are affected. Impact is limited to nonce consumption and a misleading success event, causing minor operational friction (re-signing).

Likelihood Explanation: [Low] Exploitation requires possession of a valid manager signature and timing against Compound's soft-fail windows; it is griefing with no direct profit and depends on conditions outside attacker control.

Exploitation

Exploitation Scenarios:

Scenario 1.
A malicious relayer holding a valid manager signature submits a single-call Compound V2 borrow when the market will soft-fail (return non-zero without revert). SailKernel.dispatch treats the call as successful, emits Dispatched, and consumes the manager nonce, even though no borrow occurred.
Preconditions / Assumptions
(a). Safe account registered with sessionActive = true
(b). BorrowPermission registered and configured to allow the specific Compound cToken and its underlying asset
(c). Per-transaction cap and optional LTV oracles configured so preconditions pass
(d). A valid EIP-712 manager signature for a single-call borrow dispatch exists
(e). Attacker/relayer possesses the signed payload
(f). At submission time, cToken.borrow(uint256) returns a non-zero status without reverting

Scenario 2.
An attacker front-runs a near-deadline signed borrow by submitting it during a soft-fail window. The nonce is consumed and Dispatched is emitted; the victim must re-sign and may miss a time-sensitive window.
Preconditions / Assumptions
(a). All preconditions from Scenario 1

(b). The signed dispatch has a near-expiring deadline
(c). The signature is exposed to a relayer or visible in the mempool for front-running
(d). Attacker times submission during a soft-fail window

Scenario 3.
A manager uses a single-call workflow (no batch post-conditions). An attacker times submission to a soft-fail period, causing a no-op that still consumes the nonce; a later operation depending on borrowed funds fails due to missing funds, requiring re-signing.
Preconditions / Assumptions
(a). All preconditions from Scenario 1
(b). The manager's intended workflow is single-call (no batch postconditions to force atomic revert on failure)
(c). Attacker times submission during a soft-fail window

Vulnerability #9

Early-flooring LTV math in BorrowPermission._ltvCheck causes denial of legitimate borrows under coarse oracle decimals

Severity: INFORMATIONAL | Likelihood: LOW | Impact: LOW | Status: Acknowledged

Valid but won't patch; documented in BorrowPermission. The flooring is fail-closed: it may reject a borrow that is marginally within the true LTV ceiling, but it never permits an over-LTV borrow. The recourse is to borrow slightly less. Rewriting the LTV math to recover the last fractional unit of headroom makes an over-cautious check less cautious for no security gain.

BorrowPermission._ltvCheck floors the LTV-applied collateral value before decimal rescaling and price division. With coarse collateral-oracle decimals and small collateral values, this can reduce the computed borrow headroom to zero, denying otherwise LTV-compliant borrows across Aave, Morpho, and Compound paths.

In BorrowPermission._ltvCheck, the maximum allowed borrow amount is computed as: scaledBound = floor(colValue * maxLtvBps / 10_000); then maxAmountAllowed = floor(scaledBound * 10^borDec / 10^colDec) / borPrice. Because the floor is applied before collapsing the collateral and borrow decimal systems and before dividing by borrow price, any fractional portion of collateral under the LTV can be truncated away. When the collateral oracle uses coarse precision (e.g., colDec = 0) and the account's collateral value is small in that unit system, scaledBound can become 0 even though a positive borrow would be within the true LTV bound after unit rescaling. The template allows oracle decimals up to 77 but imposes no lower bound; freshness and nonzero prices are enforced, so such inputs are valid. This results in a fail-closed denial of intended borrows (not a bypass). It affects Aave, Morpho, and Compound V2 borrow paths alike because they all route through _ltvCheck. A safer ordering that remains fail-closed would first rescale collateral into borrow units and then apply the LTV and price division, or use cross-multiplication with Math.mulDiv to minimize floors.

file: contracts/templates/BorrowPermission.sol

```
267 //       also closes the prior bug where pre-flooring the
    borrow value to whole numeraire
268 //       units (borrowScaled) collapsed any sub-1-unit
    borrow to 0 and passed ANY ceiling.
```

```
269 -//   (2) No collateral truncation. The LTV fraction is
    applied to the full-precision colValue
270 -//       mantissa FIRST; only then is the decimal scale
    collapsed by dividing by 10^colDec.
271 -//       Folding 10^colDec in before taking the fraction
    would floor away fractional
272 -//       collateral and over-block borrows that are genuinely
    within the ceiling.
273  // colValue != 0, borPrice != 0 and decimals <= 77 are
    all guarded above, so every divisor is
274  // non-zero and 10^colDec / 10^borDec cannot overflow;
    mulDiv carries each product at full
275  // precision in a 512-bit intermediate.
276 -  uint256 scaledBound        = Math.mulDiv(colValue,
    s.maxLtvBps, 10_000);
277 -  uint256 maxAmountAllowed = Math.mulDiv(scaledBound, 10 **
    uint256(borDec), 10 ** uint256(colDec)) / borPrice;
278   return amount <= maxAmountAllowed;
279  }
```

```
267  //       also closes the prior bug where pre-flooring the
    borrow value to whole numeraire
268  //       units (borrowScaled) collapsed any sub-1-unit
    borrow to 0 and passed ANY ceiling.
269 +//   (2) No avoidable truncation. Decimal scales are
    collapsed FIRST by rescaling colValue
270 +//       into borrow units, then the LTV fraction and
    price division are applied. This
271 +//       preserves fractional collateral information and
    avoids over-blocking while
272 +//       remaining fail-closed.
273  // colValue != 0, borPrice != 0 and decimals <= 77 are
    all guarded above, so every divisor is
274  // non-zero and 10^colDec / 10^borDec cannot overflow;
    mulDiv carries each product at full
275  // precision in a 512-bit intermediate.
276 +  uint256 scaledCol        = Math.mulDiv(colValue, 10 **
    uint256(borDec), 10 ** uint256(colDec));
277 +  uint256 maxAmountAllowed = Math.mulDiv(scaledCol,
    s.maxLtvBps, 10_000) / borPrice;
278   return amount <= maxAmountAllowed;
279  }
```

Severity

Impact Explanation: [Low] Per-account denial of intended borrow actions without fund loss or long-term lockup; does not break core protocol/network functionality or invariants.

Likelihood Explanation: [Low] Requires uncommon but permitted oracle conditions: coarse collateral-oracle decimals and small collateral values in that scale; not attacker-driven and outside attacker control.

Exploitation

Exploitation Scenarios:

Scenario 1.
Aave: With collateralOracle returning colValue=1, colDec=0 and maxLtvBps=5000 (50%), and borrowOracle returning borPrice=10^18, borDec=18 (1 base = 1 quote unit), any positive Aave borrow on an allowlisted asset/target is denied because scaledBound=floor(1*5000/10000)=0 => maxAmountAllowed=0, despite a true 0.5 quote-unit headroom.
Preconditions / Assumptions
(a). Account is registered in SailKernel; configured epoch matches (configCurrent).
(b). Aave protocol target and the borrow asset are allowlisted; ctx.value=0; amount maxAmountAllowed=0, although the true LTV would allow it.
Preconditions / Assumptions
(a). Account is registered; configured epoch matches.
(b). Morpho protocol target and the borrow asset are allowlisted; ctx.value=0; amount <= maxAmountPerTx; onBehalf=account; receiver=account.
(c). collateralOracle.getPrice(account, address(0)) returns (colValue=3, colDec=0, updatedAt fresh).
(d). borrowOracle.getPrice(asset, address(0)) returns (borPrice=10^18, borDec=18, updatedAt fresh).
(e). maxLtvBps=3300; decimals ≤ 77.
(f). Manager attempts a borrow near 1 quote unit that should be allowed under 33% LTV with full-precision math.

Scenario 3.
Compound V2: On the COMPOUND_BORROW path, the template resolves ICErc20(ctx.target).underlying() and applies the same check. Under colValue=1, colDec=0, maxLtvBps=5000 and borPrice=10^18, borDec=18, any positive underlying-denominated borrow amount is denied for the same early-flooring reason.
Preconditions / Assumptions
(a). Account is registered; configured epoch matches.
(b). Compound cToken target is allowlisted; underlying asset is allowlisted; ctx.value=0; amount <= maxAmountPerTx.
(c). Template successfully resolves ICErc20(ctx.target).underlying().
(d). collateralOracle.getPrice(account, address(0)) returns (colValue=1, colDec=0, updatedAt fresh).
(e). borrowOracle.getPrice(underlying, address(0)) returns (borPrice=10^18, borDec=18, updatedAt fresh).
(f). maxLtvBps=5000; decimals ≤ 77.
(g). Manager attempts a positive borrow within the true LTV bound after unit rescaling.

Warning #1

Predictable nonce-epoch bump in SailKernel dispatch/batch nonces allows pre-signed future-nonce messages to execute after a signer-restriction bump, causing fund exfiltration

Severity: MEDIUM | Likelihood: LOW | Impact: HIGH | Status: Resolved

Resolved in PR #70 for the emergency-revoke kill-switch: revokeSession now advances the signer nonce across a full epoch, so a permissionSigner operation pre-signed before a revoke (including a queued ActivateSession) is invalidated and the session cannot be silently re-enabled — upholding single-block revocability. The separate manager/batch predictable-epoch portion (a pre-signed future-nonce dispatch becoming valid after a later epoch bump) is acknowledged and not patched: it requires an already-adversarial manager who pre-signs the message, a subsequent epoch-bump event, and a permissively-configured batch permission, and it is bounded by the owner's single-block revoke. Note that a predictable post-bump nonce is deterministically computable under any fixed bump scheme, so canonicalization would add complexity without closing it.

SailKernel uses a fixed NONCE_EPOCH_INCREMENT (2^128) to invalidate prior signatures by bumping manager/batch nonces. Because the bump size is predictable and the EIP-712 typed data does not carry an explicit epoch, a manager can pre-sign a batch with nonce=current+2^128. After any signer-restriction operation that bumps by 2^128 (e.g., revoke/replace/revokePermissions/session toggle), that pre-signed batch's nonce matches and can execute, enabling token exfiltration via ApproveAndCallBatchPermission when configured in permissive mode.

The kernel advances managerNonces and batchNonces by a fixed step (NONCE_EPOCH_INCREMENT = 1 << 128) on operations such as revokePermission, replacePermissions, revokePermissions, revokeSession, and activateSession. EIP-712 message verification for dispatch and dispatchBatch uses the current on-chain nonce but does not include a separate epoch field. A manager (adversarial within permission bounds per trust assumptions) can therefore pre-sign a message using nonce = current + 2^128 (or higher multiples). After any signer-restriction bump that adds exactly 2^128 to the on-chain nonce, the pre-signed message's nonce aligns and becomes valid. If the targeted permission remains registered and configured and the session is active, the pre-signed message executes. Concretely, with ApproveAndCallBatchPermission configured with requireRecipientIsAccount=false, a three-call batch [approve, swapExactTokensForTokens(to=attacker), reset-approve] passes validation and executes via the Safe module, exfiltrating funds to the attacker's address while

resetting allowance. This behavior contradicts a strong reading of 'invalidate outstanding signatures' because explicitly future-nonced signatures survive a single epoch bump. While setManager or a revokeSession→activateSession cycle would close this window (and templates require config freshness when permissions are removed/re-added), a signer-restriction bump that leaves the batch permission intact unintentionally revalidates such time-bombs. The impact is direct fund loss bounded by configured per-token caps; likelihood is lowered by multiplicative constraints (need for a later bump event, defender not using setManager/session cycle, permissive template mode, and a priori pre-signed future nonce).

file: contracts/core/SailKernel.sol

```
118  ///      without requiring separate nonce-rotation messages (Octane finding #7 related).
119   uint256 private constant NONCE_EPOCH_INCREMENT = 1 << 128;
120
121  /// @dev ERC-1271 magic value returned by `isValidSignature` for a valid signature.
...
904   configs[account].manager = newManager;
905  _clearPermissions(account);
906 - managerNonces[account] += NONCE_EPOCH_INCREMENT;
907 - batchNonces[account] += NONCE_EPOCH_INCREMENT;
908   signerNonces[account]  += NONCE_EPOCH_INCREMENT;
```

```
118  ///      without requiring separate nonce-rotation messages (Octane finding #7 related).
119   uint256 private constant NONCE_EPOCH_INCREMENT = 1 << 128;
120 +// SECURITY NOTE:
121 +// To prevent pre-signed "future-nonce" (current + 2^128) messages from becoming valid after an epoch bump,
122 +// replace every linear epoch bump of managerNonces/batchNonces using `+= NONCE_EPOCH_INCREMENT`
123 +
```

```
    // with a canonicalized/unpredictable bump. Minimal,
    EIP-712-compatible option:
124 +//     nonce = ((nonce >> 128) + 1) << 128;  // zero lower
    128 bits on epoch change
125 +// Apply at all bump sites: revokePermission, replacePermis-
    sion, replacePermissions, revokePermissions,
126 +// revokeSession, activateSession, and setManager (for
    managerNonces/batchNonces). Optionally canonicalize signerNonces
    in setManager as well.
127
128  /// @dev ERC-1271 magic value returned by `isValidSignature`
    for a valid signature.

...

911   configs[account].manager = newManager;
912  _clearPermissions(account);
913 +// SECURITY FIX: canonicalize/refresh epoch for
    manager/batch nonces to block predictable future-nonce replays.
914 +// Example: managerNonces[account] = ((manager-
    Nonces[account] >> 128) + 1) << 128; batchNonces[account] =
    ((batchNonces[account] >> 128) + 1) << 128;
915   signerNonces[account]  += NONCE_EPOCH_INCREMENT;
```

Severity

Impact Explanation: [High] Successful exploitation causes direct, material loss of principal funds from the user's Safe by executing an approved batch that delivers output to an attacker-controlled address. The loss can be repeated if multiple future-nonce messages were pre-signed, bounded per execution by configured per-token caps.

Likelihood Explanation: [Low] The attack requires multiple constraints outside attacker control: (1) a later signer-restriction bump event of exactly 2^128; (2) defender not using setManager or a revokeSession→activateSession pattern; (3) the batch permission remaining registered and configured; (4) permissive template configuration (requireRecipientIsAccount=false). It also relies on prior creation of a future-nonce signature by the adversarial manager (or equivalent operator error in producing/sharing such a signature). These multiplicative conditions reduce overall likelihood.

Exploitation

Exploitation Scenarios:

Scenario 1.
Post-incident exfiltration via a time-bomb batch: An adversarial manager pre-signs a DispatchBatch with nonce=current+2^128 for ApproveAndCallBatchPermission (requireRecipientIsAccount=false), approving token T to router R, swapping T→U with to=attackerEOA, then resetting allowance. Later, the permissionSigner revokes/replaces an unrelated permission (or toggles session) causing batchNonces += 2^128, while the batch permission remains registered and configured and session remains active. Immediately after the bump, the attacker submits the pre-signed batch; EIP-712 verification now matches the on-chain nonce, evaluateBatch succeeds, and Safe executes the batch, delivering output to attackerEOA and resetting allowance. Multiple pre-signed sequential nonces (current+2^128+i) can be executed in sequence after the bump.
Preconditions / Assumptions

(a). Account is registered; sessionActive=true.
(b). ApproveAndCallBatchPermission is registered and configured; configuredEpoch equals the kernel's registrationEpoch for the permission (i.e., permission was not removed).
(c). ApproveAndCallBatchPermission has token T allowlisted with a sufficiently large maxApprovalAmount; spender/router R and consuming pair (R, swapExactTokensForTokens) are allowlisted; requireRecipientIsAccount=false.
(d). Pre-batch allowance for (T, R) is zero (enforced by the template).
(e). At T0, attacker (adversarial manager) pre-signs an EIP-712 batch with nonce = current batch nonce + 2^128 and a far-enough deadline.
(f). At T1, a signer-restriction operation occurs (e.g., revokePermission/replacePermissions/revokePermissions/revokeSession/activateSession) that bumps batchNonces += 2^128 while leaving the batch permission registered and configured and session active.
(g). Defender does not use setManager (which changes cfg.manager and clears permissions) nor a revokeSession→activateSession recovery pattern that blocks during suspension and double-bumps the nonce.
(h). Sufficient T balance is present in the Safe.

Warning #2

v==1 approved-hash shortcut in Safe.checkSignatures used by SailKernel.registerAccount causes unauthorized account registration and potential fund theft

Severity: MEDIUM | Likelihood: LOW | Impact: HIGH | Status: Resolved

Resolved in PR #69 (same change as #5). A bounded scan over the static signature region rejects any v==1 approved-hash entry before checkSignatures is called, so a setup-time delegate cannot authorize registration without a genuine owner signature. The scan is bounded to the static region so contract-signature tails are never misread.

SailKernel.registerAccount relies on Safe v1.4.1 checkSignatures to verify owner approval. Safe's v==1 "approved-hash" path accepts a signature when msg.sender equals the r-encoded owner or when approvedHashes is set. A malicious Safe.setup delegatecall helper can temporarily make the SailKernel contract an owner, set threshold=1, and call registerAccount with a single v==1 signature (r=kernel). Inside Safe, msg.sender is the kernel, so the signature is accepted without any real owner ECDSA. The helper then restores the owner set before setup returns. The Safe appears normal but is registered to Sail with attacker-controlled permissionSigner and manager, enabling later fund-draining via the Sail module.

SailKernel.registerAccount is intended to be robust against arbitrary setup delegates by requiring Safe owner-set approval via ISafe.checkSignatures over an EIP-712 RegisterAccount digest. However, Safe v1.4.1's checkNSignatures treats signatures with v==1 as "approved-hash" entries and accepts them whenever (a) the currentOwner (decoded from r) is an owner and (b) msg.sender equals that owner or approvedHashescurrentOwner is nonzero. During Safe.setup, the setupModules delegatecall target runs with full Safe context (msg.sender == Safe) and can modify Safe storage and call authorized functions. A malicious helper can: (1) enable the SailKernel module (enableModule(kernel)), (2) satisfy SailKernel's nonce()!=0 check (via execTransaction by a temporary owner or direct SSTORE), (3) temporarily add the SailKernel contract as an owner and set threshold=1, then (4) call SailKernel.registerAccount with a single v==1 signature whose r encodes the kernel address. Inside Safe.checkSignatures, msg.sender is the SailKernel (the caller), so the v==1 approval passes without any genuine owner ECDSA or approvedHashes entry. registerAccount then stores attacker-chosen permissionSigner and manager for the victim Safe. The helper can restore the original owners and threshold before setup returns, leaving the Safe appearing normal while the Sail registration is attacker-controlled. An alternative variant seeds approvedHashesrealOwner directly via storage writes during setup, avoid-

ing temporary kernel ownership. Post-registration, the attacker can register permissive permissions and dispatch arbitrary module calls to drain funds.

file: contracts/core/SailKernel.sol

```
841   deadline
842  )));
843  // Reverts (GS0xx) unless `ownerSig` satisfies the Safe's owner set + threshold.
844   ISafe(msg.sender).checkSignatures(digest, "", ownerSig);
```

```
841   deadline
842  )));
843 +// Disallow Safe v==1 approved-hash signatures (caller shortcut).
844 +// Scan concatenated 65-byte signature entries and reject any with v == 1.
845 +// v==0 (contract-signature) is not supported here as `data` is empty.
846 +{
847 + uint256 chunks = ownerSig.length / 65;
848 + for (uint256 i; i < chunks; ++i) {
849 + uint8 v;
850 + assembly { v := byte(0, calldataload(add(add(ownerSig.offset, mul(i, 65)), 64))) }
851 + if (v == 1) revert("ApprovedHashSigNotAllowed");
852 +}
853 +}
854  // Reverts (GS0xx) unless `ownerSig` satisfies the Safe's owner set + threshold.
855   ISafe(msg.sender).checkSignatures(digest, "", ownerSig);
```

Severity

Impact Explanation: [High] Once registered with attacker-controlled permissionSigner and manager, the attacker can register permissive permissions and dispatch arbitrary module calls, enabling direct theft of principal funds from the victim's Safe via the Sail module.

Likelihood Explanation: [Low] Exploitation requires the victim to initialize their Safe with a malicious setup delegatecall helper (user/integrator-side unsafe choice). While plausible, this is a significant constraint outside the attacker's full control and thus categorized as low likelihood.

Exploitation

Exploitation Scenarios:

Scenario 1.
Malicious setup helper temporarily adds the SailKernel contract as a Safe owner and sets threshold=1, enables the kernel module, ensures nonce!=0, then calls SailKernel.registerAccount with a single v==1 signature where r encodes the kernel. Safe.checkSignatures accepts the signature because msg.sender (the kernel) equals the r-encoded owner. The helper then restores owners/threshold; the Safe appears normal but is registered with attacker-controlled principals.
Preconditions / Assumptions
(a). Victim Safe is created externally (not via SailKernel.createAccount) and uses a malicious Safe.setup delegatecall helper during setupModules
(b). SafeProxy runtime codehash and singleton are genuine v1.4.1 (trusted by SailKernel)
(c). Attacker can execute arbitrary code in the Safe's context during setup (delegatecall)
(d). SailKernel module address is known to the helper

Scenario 2.
Malicious setup helper enables the kernel module, ensures nonce!=0, computes the RegisterAccount digest,

and directly SSTOREs approvedHashesrealOwner=1 in Safe storage. It then calls SailKernel.registerAccount with a single v==1 signature where r encodes the real owner. Safe.checkSignatures accepts due to approvedHashes. The helper can clear the mapping entry afterward.
Preconditions / Assumptions
(a). Victim Safe is created externally with a malicious setup delegatecall helper
(b). Helper can compute and write to Safe storage (approvedHashes mapping) during setup
(c). SafeProxy and singleton are official v1.4.1
(d). SailKernel module is enabled during setup

Scenario 3.
Malicious setup helper enables the kernel module, adds attacker EOA as a temporary owner with threshold=1, executes a cheap Safe.execTransaction signed by the attacker to bump nonce, then temporarily adds the kernel as owner (threshold=1) and calls SailKernel.registerAccount with a single v==1 signature (r=kernel). It restores owners/threshold afterward.
Preconditions / Assumptions
(a). Victim Safe is created externally with a malicious setup delegatecall helper
(b). Helper can add/remove temporary owners and set threshold during setup
(c). Helper can execute Safe.execTransaction using a temporary owner to bump nonce
(d). SailKernel module is enabled during setup